GIÁO TRÌNH TỰ HỌC ĐIỆN TỬ VÀ LẬP TRÌNH NHÚNG

ARDUINO TỪ TRONG CHĂN

Từ một chiếc LED đến hệ thống biết quan sát và phản hồi

Hiểu dòng điện — Hiểu chương trình — Tự tạo một vật biết hành động

Mạch điện, cảm biến, cơ cấu chấp hành, thuật toán thời gian thực và dự án tích hợp

Nguyễn Huỳnh Đăng Nhựt

Bản thảo hoàn chỉnh — 2026

Lời nói đầu

Một dòng Python có thể làm một con số xuất hiện trên màn hình. Một dòng Arduino có thể làm căn phòng sáng lên, một động cơ quay hoặc một cảm biến bắt đầu quan sát. Chính khoảnh khắc chương trình bước ra khỏi cửa sổ máy tính và chạm vào thế giới vật lý khiến điện tử nhúng vừa cuốn hút, vừa đòi hỏi trách nhiệm hơn.

Cuốn sách này được viết theo tinh thần của *Python từ trong chăn*: bắt đầu từ một hình dung gần gũi, đi đến khái niệm, tự lắp và chạy một ví dụ, rồi mổ xẻ điều đang thực sự xảy ra. Nhưng ở Arduino, "chạy được" chưa đủ. Mỗi bài còn phải trả lời dòng điện đi đâu, linh kiện chịu giới hạn nào, chuyện gì xảy ra khi dây lỏng và làm thế nào để thất bại mà không làm hỏng phần cứng.

Ta sẽ dùng Arduino Uno R3 làm bảng tham chiếu vì nó phổ biến và có tài liệu phong phú. Tư duy trong sách áp dụng rộng hơn, nhưng số chân, điện áp logic, độ phân giải ADC, chân PWM và khả năng cấp dòng khác nhau giữa các board. Mỗi khi chuyển sang Uno R4, Nano, ESP32 hoặc board 3,3 V, hãy mở đúng pinout và datasheet. Tên "Arduino" là một hệ sinh thái, không phải lời hứa rằng mọi phần cứng giống nhau.

Con đường học được chia làm sáu phần. Trước tiên, ta hiểu điện áp, dòng, điện trở, breadboard và vòng đời một sketch. Sau đó board học cách đọc nút, ánh sáng, nhiệt độ; tạo phản hồi bằng LED, còi, servo và động cơ. Phần giữa là bước ngoặt: rời khỏi `delay()`, dùng `millis()`, máy trạng thái, lọc, hiệu chuẩn, hysteresis và điều khiển. Tiếp theo là giao tiếp UART, I2C, SPI, màn hình và bộ nhớ. Phần năm tập trung vào ngắt, tổ chức code, gỡ lỗi, nguồn và độ tin cậy. Cuối cùng, sáu dự án gom mọi thứ thành hệ thống.

Mỗi mạch trong sách giả định nguồn điện áp thấp từ USB hoặc nguồn DC phù hợp với board. Cuốn sách không hướng dẫn đấu điện lưới. Relay không biến người mới thành người đủ năng lực làm việc với 220 V; nếu một dự án cần điện lưới, hãy dùng module/sản phẩm được chứng nhận và nhờ người có chuyên môn thực hiện phần nguy hiểm. Không thử nghiệm trực tiếp với cơ thể, pin lithium trần, thiết bị y tế hoặc hệ thống an toàn.

Bạn nên giữ một *nhật ký bàn mạch*. Mỗi lần lắp, ghi ngày, sơ đồ phiên bản, nguồn, giá trị điện trở, chân dùng, chương trình, triệu chứng và thay đổi duy nhất đã thử. Chụp mạch từ trên xuống trước khi tháo. Khi một project từng chạy nhưng hôm sau không chạy, nhật ký biến ký ức mơ hồ thành bằng chứng.

Một phiên học tốt có nhịp: đọc sơ đồ khi chưa cấp nguồn; dự đoán điện áp ở vài điểm; lắp; kiểm tra lại GND và nguồn; cấp điện; quan sát; thay một biến; ghi kết quả. Nếu có đồng hồ vạn năng, hãy học dùng nó sớm. Serial Monitor cho biết chương trình nghĩ gì; multimeter cho biết mạch đang làm gì. Hai lời kể không khớp chính là nơi ta bắt đầu gỡ lỗi.

Tôi hy vọng sau cuốn sách, bạn không chỉ có một hộp linh kiện đầy project mẫu. Bạn có thể nhìn một nhu cầu, tách thành input–state–output, chọn linh kiện theo giới hạn, viết thuật toán không chặn, đo thay vì đoán và thiết kế một hệ thống có trạng thái an toàn. Một vật biết hành động bắt đầu từ một người biết đặt câu hỏi trước khi cắm dây.

Nguyễn Huỳnh Đăng Nhựt
Tác giả

An toàn và cách học trên bàn mạch

Mười quy tắc trước mọi bài thực hành

1. Ngắt USB/nguồn trước khi thay dây hoặc linh kiện.
2. Xác nhận đúng rail dương và GND; rail breadboard có thể bị ngắt ở giữa.
3. Không nối trực tiếp LED vào nguồn/chân mà thiếu điện trở hạn dòng.
4. Không cấp điện áp cao hơn mức cho phép vào chân I/O; board 3,3 V không mặc nhiên chịu 5 V.
5. Không cấp motor, servo công suất đáng kể từ chân I/O; dùng driver và nguồn phù hợp.
6. Nguồn ngoài và board thường cần chung GND cho tín hiệu, trừ thiết kế cách ly có chủ đích.
7. Kiểm tra cực tính LED, diode, tụ điện phân và nguồn trước khi cấp điện.
8. Nếu linh kiện nóng, có mùi, rung bất thường: ngắt nguồn ngay, không chạm trực tiếp.
9. Không làm việc với điện lưới, pin lithium trần hoặc tải nguy hiểm theo bài tự học này.
10. Chỉ thay một yếu tố mỗi lần gỡ lỗi và ghi lại điều đã thay.

Bộ dụng cụ khởi đầu

Một Arduino Uno hoặc tương đương, cáp USB truyền dữ liệu, breadboard, dây jumper, LED, điện trở 220–330 Ω, 1–10 kΩ, nút nhấn, biến trở 10 kΩ, quang trở, buzzer thụ động/chủ động có ghi loại, servo micro, cảm biến siêu âm và multimeter cơ bản. Chỉ mua module motor/relay có tài liệu rõ; không dùng tải công suất cao trong bài đầu.

Nhịp kiểm tra trước khi cấp nguồn

Đi từ nguồn đến tải: nguồn bao nhiêu volt; GND có chung; dòng dự kiến; linh kiện phân cực; chân tín hiệu đúng; không có dây chạm hàng bên cạnh; code chọn đúng board/port. Sau khi cấp nguồn, dùng LED trạng thái, Serial và phép đo điện áp để kiểm chứng từng đoạn, không thay cả mạch bằng một lần đoán.

Mục lục

Lời nói đầu **iii**

An toàn và cách học trên bàn mạch **v**

I Từ dòng điện đến dòng lệnh **1**

1 Arduino là gì – Một chiếc máy tính không có màn hình **3**

1.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 3

1.1.1 Bước 1: gọi tên đại lượng và đơn vị 4

1.1.2 Bước 2: vẽ chuỗi tín hiệu 4

1.1.3 Bước 3: trạng thái và thời gian 4

1.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 5

1.2.1 1. Tách giả thuyết 5

1.2.2 2. Lập ngân sách điện và thời gian 5

1.2.3 3. Đo đường cơ sở 5

1.2.4 4. Truy vết một chu kỳ 6

1.2.5 5. Thử lỗi có chủ đích 6

1.3 Từ phép đo đến quyết định có độ tin cậy 6

1.4 An toàn là một phần của thuật toán 6

1.5 Sai rồi sửa bằng phép đo 7

1.6 Bài tập ba tầng và hồ sơ bằng chứng 7

1.7 Ba lớp thường bị gọi chung là Arduino 7

1.8 Bản đồ Uno R3 7

1.9 Từ sketch đến tín hiệu điện 8

1.10 Cài IDE và nạp Blink 8

1.11 Compile error và upload error 9

1.12 Một mô hình input–state–output 9

1.13 Bài tập luyện tập 9

1.14 Tóm tắt cuối chương 10

2 Điện tử nền tảng – Điện áp đẩy, dòng điện đi và điện trở giới hạn **11**

2.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 11

2.1.1 Bước 1: gọi tên đại lượng và đơn vị 12

2.1.2 Bước 2: vẽ chuỗi tín hiệu 12

2.1.3 Bước 3: trạng thái và thời gian . . . 12
2.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 13
2.2.1 1. Tách giả thuyết . . . 13
2.2.2 2. Lập ngân sách điện và thời gian . . . 13
2.2.3 3. Đo đường cơ sở . . . 13
2.2.4 4. Truy vết một chu kỳ . . . 14
2.2.5 5. Thử lỗi có chủ đích . . . 14
2.3 Từ phép đo đến quyết định có độ tin cậy . . . 14
2.4 An toàn là một phần của thuật toán . . . 14
2.5 Sai rồi sửa bằng phép đo . . . 15
2.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 15
2.7 Điện áp luôn nằm giữa hai điểm . . . 15
2.8 Định luật Ohm . . . 15
2.9 LED và điện áp thuận . . . 16
2.10 Công suất . . . 16
2.11 Nối tiếp và song song . . . 16
2.12 Ngắn mạch và giới hạn chân . . . 17
2.13 Pull-up, pull-down như một ý niệm . . . 17
2.14 Bài tập luyện tập . . . 17
2.15 Tóm tắt cuối chương . . . 18

3 Breadboard và LED – Mạch đầu tiên từ sơ đồ đến dây thật 19
3.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 19
3.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 20
3.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 20
3.1.3 Bước 3: trạng thái và thời gian . . . 20
3.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 21
3.2.1 1. Tách giả thuyết . . . 21
3.2.2 2. Lập ngân sách điện và thời gian . . . 21
3.2.3 3. Đo đường cơ sở . . . 21
3.2.4 4. Truy vết một chu kỳ . . . 21
3.2.5 5. Thử lỗi có chủ đích . . . 22
3.3 Từ phép đo đến quyết định có độ tin cậy . . . 22
3.4 An toàn là một phần của thuật toán . . . 22
3.5 Sai rồi sửa bằng phép đo . . . 23
3.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 23
3.7 Bên dưới những lỗ cắm . . . 23
3.8 Lắp LED ngoài . . . 23
3.9 Sketch nhấp nháy có tên rõ . . . 24
3.10 Tần số, chu kỳ và duty cycle . . . 24
3.11 Cây chẩn đoán LED không sáng . . . 25
3.12 Một thí nghiệm có kiểm soát . . . 25
3.13 Bài tập luyện tập . . . 25
3.14 Tóm tắt cuối chương . . . 26

4 Cấu trúc sketch và Serial – Cho board một tiếng nói **27**
4.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 27
4.1.1 Bước 1: gọi tên đại lượng và đơn vị 28
4.1.2 Bước 2: vẽ chuỗi tín hiệu 28
4.1.3 Bước 3: trạng thái và thời gian 28
4.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 29
4.2.1 1. Tách giả thuyết 29
4.2.2 2. Lập ngân sách điện và thời gian 29
4.2.3 3. Đo đường cơ sở 29
4.2.4 4. Truy vết một chu kỳ 29
4.2.5 5. Thử lỗi có chủ đích 30
4.3 Từ phép đo đến quyết định có độ tin cậy 30
4.4 An toàn là một phần của thuật toán 30
4.5 Sai rồi sửa bằng phép đo 31
4.6 Bài tập ba tầng và hồ sơ bằng chứng 31
4.7 Một sketch tối thiểu 31
4.8 Biến và kiểu trên Uno AVR 32
4.9 Phạm vi và thời gian sống 32
4.10 Serial.begin và baud 32
4.11 Log có cấu trúc 33
4.12 Nhận lệnh đơn giản 33
4.13 String và mảng char 33
4.14 Bài tập luyện tập 34
4.15 Tóm tắt cuối chương 34

II Đọc thế giới và tạo phản hồi **35**

5 Digital input và nút nhấn – Một tín hiệu chỉ có hai câu trả lời **37**
5.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 37
5.1.1 Bước 1: gọi tên đại lượng và đơn vị 38
5.1.2 Bước 2: vẽ chuỗi tín hiệu 38
5.1.3 Bước 3: trạng thái và thời gian 38
5.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 39
5.2.1 1. Tách giả thuyết 39
5.2.2 2. Lập ngân sách điện và thời gian 39
5.2.3 3. Đo đường cơ sở 39
5.2.4 4. Truy vết một chu kỳ 40
5.2.5 5. Thử lỗi có chủ đích 40
5.3 Từ phép đo đến quyết định có độ tin cậy 40
5.4 An toàn là một phần của thuật toán 40
5.5 Sai rồi sửa bằng phép đo 41
5.6 Bài tập ba tầng và hồ sơ bằng chứng 41
5.7 Floating input 41

5.8 Level khác edge . . . 42
5.9 Debounce không chặn . . . 43
5.10 Nhấn giữ và nhấn ngắn . . . 43
5.11 Nhiều nút và lớp Button . . . 44
5.12 Bài tập luyện tập . . . 44
5.13 Tóm tắt cuối chương . . . 44

6 Analog input và cảm biến – Biến điện áp thành một con số có nghĩa 45
6.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 45
6.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 46
6.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 46
6.1.3 Bước 3: trạng thái và thời gian . . . 46
6.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 47
6.2.1 1. Tách giả thuyết . . . 47
6.2.2 2. Lập ngân sách điện và thời gian . . . 47
6.2.3 3. Đo đường cơ sở . . . 47
6.2.4 4. Truy vết một chu kỳ . . . 48
6.2.5 5. Thử lỗi có chủ đích . . . 48
6.3 Từ phép đo đến quyết định có độ tin cậy . . . 48
6.4 An toàn là một phần của thuật toán . . . 48
6.5 Sai rồi sửa bằng phép đo . . . 49
6.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 49
6.7 Biến trở như cầu phân áp điều chỉnh . . . 49
6.8 ADC và độ phân giải . . . 50
6.9 Lượng tử hóa và nhiễu . . . 50
6.10 Cầu phân áp với quang trở . . . 50
6.11 Ánh xạ và giới hạn . . . 51
6.12 Hiệu chuẩn hai điểm và nhiều điểm . . . 51
6.13 Bài tập luyện tập . . . 52
6.14 Tóm tắt cuối chương . . . 52

7 PWM, LED RGB và buzzer – Tạo mức trung gian bằng thời gian 53
7.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 53
7.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 54
7.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 54
7.1.3 Bước 3: trạng thái và thời gian . . . 54
7.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 55
7.2.1 1. Tách giả thuyết . . . 55
7.2.2 2. Lập ngân sách điện và thời gian . . . 55
7.2.3 3. Đo đường cơ sở . . . 55
7.2.4 4. Truy vết một chu kỳ . . . 56
7.2.5 5. Thử lỗi có chủ đích . . . 56
7.3 Từ phép đo đến quyết định có độ tin cậy . . . 56
7.4 An toàn là một phần của thuật toán . . . 56

7.5 Sai rồi sửa bằng phép đo . . . 57
7.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 57
7.7 Duty cycle và giá trị analogWrite . . . 57
7.8 Điện áp trung bình không kể toàn bộ . . . 58
7.9 LED RGB . . . 58
7.10 Buzzer active và passive . . . 59
7.11 Gamma lookup . . . 59
7.12 Driver cho tải lớn . . . 59
7.13 Bài tập luyện tập . . . 59
7.14 Tóm tắt cuối chương . . . 60

8 Servo, motor và relay – Khi tín hiệu nhỏ điều khiển chuyển động lớn 61
8.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 61
8.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 62
8.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 62
8.1.3 Bước 3: trạng thái và thời gian . . . 62
8.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 63
8.2.1 1. Tách giả thuyết . . . 63
8.2.2 2. Lập ngân sách điện và thời gian . . . 63
8.2.3 3. Đo đường cơ sở . . . 63
8.2.4 4. Truy vết một chu kỳ . . . 64
8.2.5 5. Thử lỗi có chủ đích . . . 64
8.3 Từ phép đo đến quyết định có độ tin cậy . . . 64
8.4 An toàn là một phần của thuật toán . . . 64
8.5 Sai rồi sửa bằng phép đo . . . 65
8.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 65
8.7 Servo hobby . . . 65
8.8 Nguồn servo . . . 66
8.9 Motor DC và transistor . . . 66
8.10 H-bridge và đổi chiều . . . 66
8.11 Relay module . . . 67
8.12 Trạng thái an toàn . . . 67
8.13 Bài tập luyện tập . . . 68
8.14 Tóm tắt cuối chương . . . 68

III Thoát khỏi chiếc bẫy delay 69

9 millis và bộ lập lịch nhỏ – Thoát khỏi chiếc bẫy delay 71
9.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 71
9.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 72
9.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 72
9.1.3 Bước 3: trạng thái và thời gian . . . 72
9.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 73
9.2.1 1. Tách giả thuyết . . . 73

9.2.2 2. Lập ngân sách điện và thời gian . . . 73
9.2.3 3. Đo đường cơ sở . . . 73
9.2.4 4. Truy vết một chu kỳ . . . 74
9.2.5 5. Thử lỗi có chủ đích . . . 74
9.3 Từ phép đo đến quyết định có độ tin cậy . . . 74
9.4 An toàn là một phần của thuật toán . . . 74
9.5 Sai rồi sửa bằng phép đo . . . 75
9.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 75
9.7 Mẫu timer cơ bản . . . 75
9.8 Rollover . . . 76
9.9 Fixed-delay và fixed-rate . . . 76
9.10 Nhiều tác vụ . . . 76
9.11 Một struct Periodic . . . 77
9.12 Micros và giới hạn độ chính xác . . . 77
9.13 Đo thời gian loop . . . 77
9.14 Bài tập luyện tập . . . 78
9.15 Tóm tắt cuối chương . . . 78

10 Máy trạng thái – Viết hành vi theo điều hệ thống đang là 79
10.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 79
10.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 80
10.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 80
10.1.3 Bước 3: trạng thái và thời gian . . . 80
10.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 81
10.2.1 1. Tách giả thuyết . . . 81
10.2.2 2. Lập ngân sách điện và thời gian . . . 81
10.2.3 3. Đo đường cơ sở . . . 81
10.2.4 4. Truy vết một chu kỳ . . . 82
10.2.5 5. Thử lỗi có chủ đích . . . 82
10.3 Từ phép đo đến quyết định có độ tin cậy . . . 82
10.4 An toàn là một phần của thuật toán . . . 82
10.5 Sai rồi sửa bằng phép đo . . . 83
10.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 83
10.7 Từ nhiều boolean đến một enum . . . 83
10.8 Bảng trạng thái . . . 84
10.9 Entry action . . . 84
10.10 Update và chuyển trạng thái . . . 84
10.11 Event queue hay polling . . . 85
10.12 Máy trạng thái phân cấp . . . 85
10.13 Moore và Mealy . . . 85
10.14 Test máy trạng thái mà không chờ thật . . . 85
10.15 Bài tập luyện tập . . . 86
10.16 Tóm tắt cuối chương . . . 86

11 Lấy mẫu, lọc và hiệu chuẩn – Từ số đo rung đến tín hiệu có thể dùng 87
11.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 87
11.1.1 Bước 1: gọi tên đại lượng và đơn vị 88
11.1.2 Bước 2: vẽ chuỗi tín hiệu 88
11.1.3 Bước 3: trạng thái và thời gian 88
11.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 89
11.2.1 1. Tách giả thuyết 89
11.2.2 2. Lập ngân sách điện và thời gian 89
11.2.3 3. Đo đường cơ sở 89
11.2.4 4. Truy vết một chu kỳ 90
11.2.5 5. Thử lỗi có chủ đích 90
11.3 Từ phép đo đến quyết định có độ tin cậy 90
11.4 An toàn là một phần của thuật toán 90
11.5 Sai rồi sửa bằng phép đo 91
11.6 Bài tập ba tầng và hồ sơ bằng chứng 91
11.7 Lấy mẫu là một quyết định 91
11.8 Trung bình trượt 91
11.9 Exponential moving average 92
11.10 Median cho outlier xung 92
11.11 Hiệu chuẩn tuyến tính 93
11.12 Residual và validation 93
11.13 Phát hiện lỗi sensor 93
11.14 Bài tập luyện tập 93
11.15 Tóm tắt cuối chương 94

12 Hysteresis và PID – Từ bật/tắt ổn định đến điều khiển phản hồi 95
12.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 95
12.1.1 Bước 1: gọi tên đại lượng và đơn vị 96
12.1.2 Bước 2: vẽ chuỗi tín hiệu 96
12.1.3 Bước 3: trạng thái và thời gian 96
12.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 97
12.2.1 1. Tách giả thuyết 97
12.2.2 2. Lập ngân sách điện và thời gian 97
12.2.3 3. Đo đường cơ sở 97
12.2.4 4. Truy vết một chu kỳ 98
12.2.5 5. Thử lỗi có chủ đích 98
12.3 Từ phép đo đến quyết định có độ tin cậy 98
12.4 An toàn là một phần của thuật toán 98
12.5 Sai rồi sửa bằng phép đo 99
12.6 Bài tập ba tầng và hồ sơ bằng chứng 99
12.7 Hysteresis 99
12.8 Bài toán điều khiển 100
12.9 P – phản ứng theo sai số 100
12.10 I – tích lũy sai số 100

12.11 D – nhìn tốc độ thay đổi . . . 100
12.12 PID rời rạc cơ bản . . . 100
12.13 Tuning có kiểm soát . . . 101
12.14 Khi không cần PID . . . 102
12.15 Bài tập luyện tập . . . 102
12.16 Tóm tắt cuối chương . . . 102

IV Những thiết bị biết nói chuyện 103

13 UART và giao thức Serial – Từ ký tự rời đến thông điệp đáng tin 105
13.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 105
13.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 106
13.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 106
13.1.3 Bước 3: trạng thái và thời gian . . . 106
13.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 107
13.2.1 1. Tách giả thuyết . . . 107
13.2.2 2. Lập ngân sách điện và thời gian . . . 107
13.2.3 3. Đo đường cơ sở . . . 107
13.2.4 4. Truy vết một chu kỳ . . . 108
13.2.5 5. Thử lỗi có chủ đích . . . 108
13.3 Từ phép đo đến quyết định có độ tin cậy . . . 108
13.4 An toàn là một phần của thuật toán . . . 108
13.5 Sai rồi sửa bằng phép đo . . . 109
13.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 109
13.7 Đường dây và tham chiếu . . . 109
13.8 Vì sao readStringUntil có thể làm đứng loop . . . 110
13.9 Giao thức lệnh dạng text . . . 110
13.10 Framing nhị phân . . . 111
13.11 Sequence, ACK và retry . . . 111
13.12 Flow control và buffer . . . 111
13.13 Cầu nối Python–Arduino . . . 111
13.14 Bài tập luyện tập . . . 112
13.15 Tóm tắt cuối chương . . . 112

14 I2C và SPI – Nhiều thiết bị dùng chung một cuộc trò chuyện 113
14.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 113
14.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 114
14.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 114
14.1.3 Bước 3: trạng thái và thời gian . . . 114
14.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 115
14.2.1 1. Tách giả thuyết . . . 115
14.2.2 2. Lập ngân sách điện và thời gian . . . 115
14.2.3 3. Đo đường cơ sở . . . 115
14.2.4 4. Truy vết một chu kỳ . . . 115

14.2.5 5. Thử lỗi có chủ đích . . . 116
14.3 Từ phép đo đến quyết định có độ tin cậy . . . 116
14.4 An toàn là một phần của thuật toán . . . 116
14.5 Sai rồi sửa bằng phép đo . . . 117
14.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 117
14.7 I2C: SDA và SCL open-drain . . . 117
14.8 Quét địa chỉ . . . 118
14.9 Đọc thanh ghi . . . 118
14.10 Bus bị treo . . . 119
14.11 SPI . . . 119
14.12 I2C hay SPI? . . . 119
14.13 Bài tập luyện tập . . . 120
14.14 Tóm tắt cuối chương . . . 120

15 Màn hình và giao diện nhỏ – Hiển thị trạng thái, không đổ mọi dữ liệu lên người dùng 121
15.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 121
15.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 122
15.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 122
15.1.3 Bước 3: trạng thái và thời gian . . . 122
15.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 123
15.2.1 1. Tách giả thuyết . . . 123
15.2.2 2. Lập ngân sách điện và thời gian . . . 123
15.2.3 3. Đo đường cơ sở . . . 123
15.2.4 4. Truy vết một chu kỳ . . . 124
15.2.5 5. Thử lỗi có chủ đích . . . 124
15.3 Từ phép đo đến quyết định có độ tin cậy . . . 124
15.4 An toàn là một phần của thuật toán . . . 124
15.5 Sai rồi sửa bằng phép đo . . . 125
15.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 125
15.7 Bậc thang hiển thị . . . 125
15.8 7-segment . . . 125
15.9 LCD ký tự . . . 126
15.10 OLED và bộ nhớ . . . 126
15.11 Model–view trong MCU . . . 126
15.12 Menu là máy trạng thái . . . 127
15.13 Thông tin lỗi . . . 127
15.14 Bài tập luyện tập . . . 127
15.15 Tóm tắt cuối chương . . . 128

16 EEPROM và dữ liệu bền – Nhớ qua lần mất nguồn mà không ghi đến mòn 129
16.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 129
16.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 130
16.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 130

16.1.3 Bước 3: trạng thái và thời gian . . . 130
16.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 131
16.2.1 1. Tách giả thuyết . . . 131
16.2.2 2. Lập ngân sách điện và thời gian . . . 131
16.2.3 3. Đo đường cơ sở . . . 131
16.2.4 4. Truy vết một chu kỳ . . . 132
16.2.5 5. Thử lỗi có chủ đích . . . 132
16.3 Từ phép đo đến quyết định có độ tin cậy . . . 132
16.4 An toàn là một phần của thuật toán . . . 132
16.5 Sai rồi sửa bằng phép đo . . . 133
16.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 133
16.7 Ba loại bộ nhớ . . . 133
16.8 Đọc/ghi một cấu hình . . . 133
16.9 Default và validation . . . 134
16.10 update và put . . . 134
16.11 Dirty flag và trì hoãn commit . . . 134
16.12 Hai slot chịu ghi dở . . . 135
16.13 Wear leveling . . . 135
16.14 Factory reset và migration . . . 135
16.15 Bài tập luyện tập . . . 136
16.16 Tóm tắt cuối chương . . . 136

V Viết hệ thống nhúng có thể tin cậy 137

17 Interrupt – Khi một sự kiện không muốn chờ vòng loop 139
17.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 139
17.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 140
17.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 140
17.1.3 Bước 3: trạng thái và thời gian . . . 140
17.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 141
17.2.1 1. Tách giả thuyết . . . 141
17.2.2 2. Lập ngân sách điện và thời gian . . . 141
17.2.3 3. Đo đường cơ sở . . . 141
17.2.4 4. Truy vết một chu kỳ . . . 142
17.2.5 5. Thử lỗi có chủ đích . . . 142
17.3 Từ phép đo đến quyết định có độ tin cậy . . . 142
17.4 An toàn là một phần của thuật toán . . . 142
17.5 Sai rồi sửa bằng phép đo . . . 143
17.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 143
17.7 Một ISR tối thiểu . . . 143
17.8 volatile làm gì và không làm gì . . . 144
17.9 Không làm gì trong ISR . . . 144
17.10 Mode RISING/FALLING/CHANGE/LOW . . . 145

17.11 Đo tần số xung . . . 145
17.12 Ring buffer một producer–một consumer . . . 145
17.13 Latency và jitter . . . 145
17.14 Bài tập luyện tập . . . 146
17.15 Tóm tắt cuối chương . . . 146

18 Tổ chức code nhúng – Tách driver, logic và ứng dụng 147
18.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 147
18.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 148
18.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 148
18.1.3 Bước 3: trạng thái và thời gian . . . 148
18.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 149
18.2.1 1. Tách giả thuyết . . . 149
18.2.2 2. Lập ngân sách điện và thời gian . . . 149
18.2.3 3. Đo đường cơ sở . . . 149
18.2.4 4. Truy vết một chu kỳ . . . 150
18.2.5 5. Thử lỗi có chủ đích . . . 150
18.3 Từ phép đo đến quyết định có độ tin cậy . . . 150
18.4 An toàn là một phần của thuật toán . . . 150
18.5 Sai rồi sửa bằng phép đo . . . 151
18.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 151
18.7 Một kiến trúc vừa đủ . . . 151
18.8 Header và source . . . 152
18.9 Không gọi hardware trong constructor toàn cục . . . 152
18.10 Đơn vị trong tên/kiểu . . . 152
18.11 Logic thuần . . . 153
18.12 Global và single source of truth . . . 153
18.13 Bộ nhớ và cấp phát . . . 153
18.14 Mã lỗi và quality . . . 153
18.15 Cấu hình build và thư viện . . . 154
18.16 Bài tập luyện tập . . . 154
18.17 Tóm tắt cuối chương . . . 154

19 Gỡ lỗi và kiểm thử hệ thống nhúng – Hai thế giới, một chuỗi bằng chứng 155
19.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 155
19.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 156
19.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 156
19.1.3 Bước 3: trạng thái và thời gian . . . 156
19.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 157
19.2.1 1. Tách giả thuyết . . . 157
19.2.2 2. Lập ngân sách điện và thời gian . . . 157
19.2.3 3. Đo đường cơ sở . . . 157
19.2.4 4. Truy vết một chu kỳ . . . 157
19.2.5 5. Thử lỗi có chủ đích . . . 158

19.3 Từ phép đo đến quyết định có độ tin cậy . . . 158
19.4 An toàn là một phần của thuật toán . . . 158
19.5 Sai rồi sửa bằng phép đo . . . 159
19.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 159
19.7 Bắt đầu bằng triệu chứng đo được . . . 159
19.8 Chia đôi chuỗi tín hiệu . . . 159
19.9 Dụng cụ trả lời câu hỏi nào? . . . 160
19.10 Heartbeat và reset cause . . . 160
19.11 Assertions và invariant . . . 160
19.12 Test logic trên host . . . 160
19.13 Hardware-in-the-loop và test jig . . . 161
19.14 Fault injection . . . 161
19.15 Lỗi Heisenbug do logging . . . 161
19.16 Bài tập luyện tập . . . 161
19.17 Tóm tắt cuối chương . . . 162

20 Nguồn, nhiễu và thiết kế an toàn – Hệ thống chỉ đáng tin khi nguồn cũng đáng tin 163

20.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 163
20.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 164
20.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 164
20.1.3 Bước 3: trạng thái và thời gian . . . 164
20.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 165
20.2.1 1. Tách giả thuyết . . . 165
20.2.2 2. Lập ngân sách điện và thời gian . . . 165
20.2.3 3. Đo đường cơ sở . . . 165
20.2.4 4. Truy vết một chu kỳ . . . 166
20.2.5 5. Thử lỗi có chủ đích . . . 166
20.3 Từ phép đo đến quyết định có độ tin cậy . . . 166
20.4 An toàn là một phần của thuật toán . . . 166
20.5 Sai rồi sửa bằng phép đo . . . 167
20.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 167
20.7 Power budget . . . 167
20.8 Decoupling và bulk . . . 168
20.9 Brownout và reset . . . 168
20.10 Back-powering . . . 168
20.11 Watchdog . . . 168
20.12 Các lớp bảo vệ . . . 168
20.13 FMEA nhỏ . . . 169
20.14 Điện lưới, pin và giới hạn sách . . . 169
20.15 Bài tập luyện tập . . . 169
20.16 Tóm tắt cuối chương . . . 170

VI Xưởng dự án **171**

21 Đèn giao thông ở ngã tư tí hon – Từ chuỗi lệnh đến hệ thống có luật **173**
21.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 174
21.1.1 Bước 1: gọi tên đại lượng và đơn vị 174
21.1.2 Bước 2: vẽ chuỗi tín hiệu 174
21.1.3 Bước 3: trạng thái và thời gian 175
21.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 175
21.2.1 1. Tách giả thuyết 175
21.2.2 2. Lập ngân sách điện và thời gian 175
21.2.3 3. Đo đường cơ sở 176
21.2.4 4. Truy vết một chu kỳ 176
21.2.5 5. Thử lỗi có chủ đích 176
21.3 Từ phép đo đến quyết định có độ tin cậy 176
21.4 An toàn là một phần của thuật toán 177
21.5 Sai rồi sửa bằng phép đo 177
21.6 Bài tập ba tầng và hồ sơ bằng chứng 177
21.7 Định nghĩa bài toán trước khi cắm dây 178
21.8 Bất biến – những câu không được phép sai 178
21.9 Vẽ máy trạng thái 178
21.10 Sơ đồ nối và kiểm tra từng output 179
21.11 Cài đặt không chặn 179
21.12 Đọc nút như một sự kiện sạch 181
21.13 Kiểm thử bằng kịch bản 181
21.14 Mở rộng có chủ đích 182
21.15 Bài tập luyện tập 182
21.16 Tóm tắt cuối chương 182

22 Chậu cây biết chờ – Đo độ ẩm, quyết định tưới và phòng lỗi **183**
22.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động 184
22.1.1 Bước 1: gọi tên đại lượng và đơn vị 184
22.1.2 Bước 2: vẽ chuỗi tín hiệu 184
22.1.3 Bước 3: trạng thái và thời gian 185
22.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm 185
22.2.1 1. Tách giả thuyết 185
22.2.2 2. Lập ngân sách điện và thời gian 185
22.2.3 3. Đo đường cơ sở 186
22.2.4 4. Truy vết một chu kỳ 186
22.2.5 5. Thử lỗi có chủ đích 186
22.3 Từ phép đo đến quyết định có độ tin cậy 186
22.4 An toàn là một phần của thuật toán 187
22.5 Sai rồi sửa bằng phép đo 187
22.6 Bài tập ba tầng và hồ sơ bằng chứng 187
22.7 Tách hệ thống thành năm khối 188

22.8 Hiệu chuẩn thay vì tin số phần trăm trên mạng . . . 188
22.9 Cảm biến và tuổi thọ . . . 188
22.10 Chính sách tưới có trí nhớ . . . 189
22.11 Máy trạng thái vận hành . . . 189
22.12 Một phép đo được coi là hợp lệ khi nào? . . . 190
22.13 Driver bơm và bố trí nguồn . . . 191
22.14 Manual không được phá luật . . . 191
22.15 Log để học từ cây . . . 191
22.16 Kế hoạch kiểm thử và FMEA . . . 192
22.17 Bài tập luyện tập . . . 192
22.18 Tóm tắt cuối chương . . . 192

23 Trợ lý đỗ xe để bàn – Đo khoảng cách mà không hoảng vì một tiếng vọng 195
23.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 196
23.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 196
23.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 196
23.1.3 Bước 3: trạng thái và thời gian . . . 197
23.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 197
23.2.1 1. Tách giả thuyết . . . 197
23.2.2 2. Lập ngân sách điện và thời gian . . . 197
23.2.3 3. Đo đường cơ sở . . . 198
23.2.4 4. Truy vết một chu kỳ . . . 198
23.2.5 5. Thử lỗi có chủ đích . . . 198
23.3 Từ phép đo đến quyết định có độ tin cậy . . . 198
23.4 An toàn là một phần của thuật toán . . . 199
23.5 Sai rồi sửa bằng phép đo . . . 199
23.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 199
23.7 Từ thời gian bay đến khoảng cách . . . 200
23.8 Kiến trúc theo nhịp . . . 200
23.9 Đọc một mẫu có timeout . . . 200
23.10 Lọc mà không che mất vật đang đến . . . 201
23.11 Các vùng và hysteresis . . . 201
23.12 Còi là một bộ phát sự kiện, không phải một delay . . . 202
23.13 UNKNOWN phải nhìn khác FAR . . . 203
23.14 Bố trí cơ khí là một phần thuật toán . . . 203
23.15 Kế hoạch thử nghiệm . . . 203
23.16 Bài tập luyện tập . . . 204
23.17 Tóm tắt cuối chương . . . 204

24 Trạm môi trường mini – Khi nhiều cảm biến phải cùng kể một câu chuyện 205
24.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 206
24.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 206
24.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 206
24.1.3 Bước 3: trạng thái và thời gian . . . 207

24.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 207
24.2.1 1. Tách giả thuyết . . . 207
24.2.2 2. Lập ngân sách điện và thời gian . . . 207
24.2.3 3. Đo đường cơ sở . . . 208
24.2.4 4. Truy vết một chu kỳ . . . 208
24.2.5 5. Thử lỗi có chủ đích . . . 208
24.3 Từ phép đo đến quyết định có độ tin cậy . . . 208
24.4 An toàn là một phần của thuật toán . . . 209
24.5 Sai rồi sửa bằng phép đo . . . 209
24.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 209
24.7 Yêu cầu và ngân sách thời gian . . . 210
24.8 Mô hình dữ liệu biết nói thật . . . 210
24.9 Pipeline năm tầng . . . 210
24.9.1 Acquire . . . 211
24.9.2 Validate . . . 211
24.9.3 Derive . . . 211
24.9.4 Present . . . 211
24.9.5 Log . . . 211
24.10 Một scheduler nhỏ, dễ nhìn . . . 211
24.11 I2C khi có hơn một thiết bị . . . 212
24.12 Ánh sáng: từ ADC đến chỉ số ổn định . . . 212
24.13 Thiết kế màn hình nhỏ . . . 212
24.14 CSV có hợp đồng . . . 213
24.15 Min/max, reset và mất điện . . . 213
24.16 Kịch bản lỗi . . . 213
24.17 Thử nghiệm dài hạn . . . 214
24.18 Bài tập luyện tập . . . 214
24.19 Tóm tắt cuối chương . . . 214

25 Robot dò đường – Điều khiển là cuộc đối thoại liên tục 215
25.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 216
25.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 216
25.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 216
25.1.3 Bước 3: trạng thái và thời gian . . . 217
25.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 217
25.2.1 1. Tách giả thuyết . . . 217
25.2.2 2. Lập ngân sách điện và thời gian . . . 217
25.2.3 3. Đo đường cơ sở . . . 218
25.2.4 4. Truy vết một chu kỳ . . . 218
25.2.5 5. Thử lỗi có chủ đích . . . 218
25.3 Từ phép đo đến quyết định có độ tin cậy . . . 218
25.4 An toàn là một phần của thuật toán . . . 219
25.5 Sai rồi sửa bằng phép đo . . . 219
25.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 219

25.7 Kiến trúc năng lượng và cơ khí . . . 220
25.8 Cảm biến phản xạ và hiệu chuẩn . . . 220
25.9 Ước lượng vị trí bằng trọng số . . . 220
25.10 Từ sai lệch đến tốc độ bánh . . . 221
25.11 Chu kỳ vòng điều khiển . . . 222
25.12 Bù lệch motor . . . 222
25.13 Mất vạch và tìm lại . . . 222
25.14 Giao lộ, vạch đích và ngữ cảnh . . . 223
25.15 Quy trình chỉnh P/PD . . . 223
25.16 Ma trận kiểm thử . . . 223
25.17 Bài tập luyện tập . . . 223
25.18 Tóm tắt cuối chương . . . 224

26 Đồ án tốt nghiệp trong chăn – Tự biến một mong muốn thành hệ thống có bằng chứng 225
26.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động . . . 226
26.1.1 Bước 1: gọi tên đại lượng và đơn vị . . . 226
26.1.2 Bước 2: vẽ chuỗi tín hiệu . . . 226
26.1.3 Bước 3: trạng thái và thời gian . . . 227
26.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm . . . 227
26.2.1 1. Tách giả thuyết . . . 227
26.2.2 2. Lập ngân sách điện và thời gian . . . 227
26.2.3 3. Đo đường cơ sở . . . 228
26.2.4 4. Truy vết một chu kỳ . . . 228
26.2.5 5. Thử lỗi có chủ đích . . . 228
26.3 Từ phép đo đến quyết định có độ tin cậy . . . 228
26.4 An toàn là một phần của thuật toán . . . 229
26.5 Sai rồi sửa bằng phép đo . . . 229
26.6 Bài tập ba tầng và hồ sơ bằng chứng . . . 229
26.7 Bước 1: Một trang mô tả đủ rõ . . . 230
26.8 Bước 2: Yêu cầu có mã định danh . . . 230
26.9 Bước 3: Sơ đồ khối và interface . . . 230
26.10 Bước 4: State machine và data flow . . . 231
26.11 Bước 5: Ngân sách . . . 231
26.11.1 Thời gian . . . 231
26.11.2 Bộ nhớ . . . 231
26.11.3 Nguồn . . . 231
26.11.4 Sai số và độ trễ . . . 232
26.12 Bước 6: Xây theo vertical slice . . . 232
26.13 Bước 7: Thiết kế code . . . 232
26.14 Bước 8: Verification plan . . . 233
26.15 Bước 9: Quan sát và chẩn đoán . . . 233
26.16 Bước 10: Review an toàn và FMEA . . . 233
26.17 Bước 11: Báo cáo kỹ thuật . . . 234

26.18 Bước 12: Demo không phụ thuộc may mắn . . . 234
26.19 Rubric tự chấm 100 điểm . . . 234
26.20 Nhật ký 14 mốc . . . 235
26.21 Bộ câu hỏi phản biện cuối . . . 235
26.22 Bài tập – chính là đồ án . . . 236
26.23 Tóm tắt cuối chương . . . 236

A Bàn linh kiện và pinout – Tra cứu trước khi cắm **237**
A.1 Arduino Uno R3 nhìn theo vai trò . . . 237
A.2 Bảng chọn điện trở LED . . . 238
A.3 Mã màu điện trở bốn vạch . . . 238
A.4 Tụ điện theo công việc . . . 239
A.5 Nút, công tắc và input không trôi . . . 239
A.6 BJT, MOSFET và relay theo câu hỏi . . . 239
A.7 Diode cần nhớ . . . 240
A.8 Cảm biến: bảng đọc nhãn trước . . . 240
A.9 Bus và mức logic . . . 241
A.10 Dây, breadboard và connector . . . 241
A.11 Đồng hồ số: bốn phép đo nền tảng . . . 241
A.12 Bộ linh kiện học tập đề xuất . . . 242
A.13 Checklist trước cấp nguồn . . . 242

B Bảng lỗi tại bàn – Tìm nguyên nhân mà không đoán mò **245**
B.1 Quy trình tám bước . . . 245
B.2 Dấu hiệu phải ngắt nguồn ngay . . . 245
B.3 Không upload được . . . 245
B.4 Board không có dấu hiệu sống . . . 246
B.5 LED không sáng hoặc sáng ngược . . . 246
B.6 Nút tự nhấn, nhấn một lần ra nhiều lần . . . 246
B.7 Analog nhảy hoặc luôn 0/1023 . . . 247
B.8 Serial ký tự rác hoặc không có dữ liệu . . . 247
B.9 I2C không thấy thiết bị . . . 247
B.10 SPI đọc ID sai hoặc toàn 0xFF . . . 248
B.11 Servo rung và Arduino reset . . . 248
B.12 Motor yếu, driver nóng hoặc chỉ quay một chiều . . . 248
B.13 Relay click nhưng tải không chạy . . . 248
B.14 Chương trình treo “ngẫu nhiên” . . . 248
B.15 Timing chạy sai sau nhiều ngày . . . 249
B.16 EEPROM đọc cấu hình kỳ lạ . . . 249
B.17 Cảm biến trả số hợp lý nhưng sai . . . 249
B.18 Buzzer/`tone()` làm PWM khác biến mất . . . 249
B.19 Compiler báo lỗi dài . . . 250
B.20 Bảng ghi chẩn đoán . . . 250
B.21 Mười câu hỏi trước khi cầu cứu . . . 250

C Đáp án gợi ý và phòng thí nghiệm thu nhỏ **251**
C.1 Chương 1 – Input, state, output . . . 251
C.2 Chương 2 – Tính điện trở và công suất . . . 252
C.3 Chương 3 – Bring-up LED có bằng chứng . . . 252
C.4 Chương 4 – Serial parser không dựa may mắn . . . 252
C.5 Chương 5 – Debounce và edge . . . 253
C.6 Chương 6 – Hiệu chuẩn analog hai chiều . . . 254
C.7 Chương 7 – Gamma cho LED và nhịp buzzer . . . 254
C.8 Chương 8 – Driver tải và năng lượng . . . 254
C.9 Chương 9 – Scheduler và wraparound . . . 255
C.10 Chương 10 – State table thành test . . . 255
C.11 Chương 11 – Median và EMA bằng dữ liệu . . . 255
C.12 Chương 12 – Hysteresis và PID chống windup . . . 256
C.13 Chương 13 – Frame, checksum và parser phục hồi . . . 256
C.14 Chương 14 – Chẩn đoán I2C/SPI bằng tầng . . . 256
C.15 Chương 15 – UI từ nhu cầu hành động . . . 257
C.16 Chương 16 – Record EEPROM hai slot . . . 257
C.17 Chương 17 – ISR tối thiểu và snapshot nguyên tử . . . 258
C.18 Chương 18 – Ranh giới module . . . 258
C.19 Chương 19 – Từ triệu chứng đến nguyên nhân . . . 259
C.20 Chương 20 – Power budget và FMEA . . . 259
C.21 Chương 21 – Test bất biến đèn giao thông . . . 260
C.22 Chương 22 – Quota tưới dạng cửa sổ . . . 260
C.23 Chương 23 – Bộ lọc vùng khoảng cách . . . 260
C.24 Chương 24 – Health model trạm môi trường . . . 261
C.25 Chương 25 – Dấu correction của robot . . . 261
C.26 Chương 26 – Một mẫu traceability nhỏ . . . 261
C.27 Cách dùng đáp án để học sâu hơn . . . 262

D Từ điển bỏ túi Arduino và hệ thống nhúng **263**
D.1 Năm cặp dễ nhầm . . . 268

E Sổ tay thực hành – Mười bốn phiếu biến "đã chạy" thành bằng chứng **269**
E.1 Phiếu 1 – Kiểm tra trước cấp nguồn . . . 269
E.2 Phiếu 2 – Bring-up một output . . . 270
E.3 Phiếu 3 – Nút nhấn và debounce . . . 271
E.4 Phiếu 4 – Khảo sát một kênh analog . . . 272
E.5 Phiếu 5 – PWM, tải và nhiệt . . . 273
E.6 Phiếu 6 – Scheduler không chặn . . . 274
E.7 Phiếu 7 – Review máy trạng thái . . . 275
E.8 Phiếu 8 – Giao thức Serial . . . 276
E.9 Phiếu 9 – Bring-up I2C hoặc SPI . . . 277
E.10 Phiếu 10 – Dữ liệu bền qua mất nguồn . . . 278
E.11 Phiếu 11 – Power budget và đo rail . . . 279

E.12 Phiếu 12 – Fault injection và recovery 280
E.13 Phiếu 13 – Soak test và phân tích log 281
E.14 Phiếu 14 – Nghiệm thu đồ án 282

F Tài liệu tham khảo và lộ trình đọc tiếp **283**
F.1 Arduino chính thức 283
F.2 Vi điều khiển và điện tử 283
F.3 Hệ thống nhúng và kiến trúc phần mềm 284
F.4 Điều khiển và xử lý tín hiệu 284
F.5 Đo lường và dụng cụ 284
F.6 Cách đọc một datasheet 285
F.7 Lộ trình sau cuốn sách 285

Phần I

Từ dòng điện đến dòng lệnh

Chương 1

Arduino là gì – Một chiếc máy tính không có màn hình

Giới thiệu ngắn

Arduino Uno trông như một bảng mạch nhỏ với nhiều lỗ cắm. Bên trong là vi điều khiển: một máy tính có CPU, bộ nhớ và ngoại vi, được tạo để quan sát tín hiệu rồi phản ứng. Nó không chạy hệ điều hành desktop, không có vô hạn RAM và không tự biết cảm biến nào đang nối. Chính sự giới hạn ấy giúp ta nhìn rõ quan hệ giữa chương trình và phần cứng.

Mục tiêu học tập

- Phân biệt vi điều khiển, board phát triển và máy tính đa dụng.
- Nhận diện khối nguồn, USB, vi điều khiển, chân digital/analog trên Uno R3.
- Hiểu chu trình viết–biên dịch–nạp–reset–chạy.
- Cài/chọn Arduino IDE, board, cổng và ví dụ Blink.
- Đọc pinout thay vì ghi nhớ một sơ đồ cho mọi board.

1.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một đèn cầu thang tự bật khi tối và có người đi qua không cần màn hình hay hệ điều hành desktop. Nó cần đọc hai tín hiệu, giữ một ít trạng thái, quyết định trong thời gian hữu hạn và điều khiển đèn. Arduino là nền tảng giúp ta nhìn rõ vòng quan sát–tính toán–tác động ấy.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **vi điều khiển và board Arduino như máy tính quan sát thế giới vật lý**, ta luôn đi qua bốn

miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

1.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: điện áp nguồn, chân I/O, xung clock, flash, SRAM, chu kỳ loop và trạng thái reset. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$\text{sensor} \rightarrow \text{MCU} \rightarrow \text{decision} \rightarrow \text{actuator}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

1.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

1.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là giá trị biến trong SRAM, thanh ghi ngoại vi và vị trí chương trình đang thực thi. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Board chỉ điều khiển tải trong giới hạn điện; sau reset đầu ra phải về trạng thái an toàn trước khi logic ứng dụng tiếp quản.

1.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Phân rã đèn cầu thang thành cảm biến sáng, cảm biến chuyển động, vi điều khiển và tầng công suất. Trước khi nạp, ta mô tả nguồn, chân, mức logic, chu kỳ phản ứng và cách thử từng khối.

1.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

1.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

1.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

1.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

1.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

1.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

1.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

1.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đồng nhất board Arduino với bản thân vi điều khiển.
- Cho rằng chân I/O có thể cấp mọi tải.
- Nhớ pinout Uno rồi áp cho board khác.
- Sửa code khi lỗi thực sự là cáp, cổng hoặc nguồn.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

1.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ sơ đồ khối của một hệ nhúng trong nhà.
2. Phân biệt flash, SRAM và EEPROM theo vòng đời.
3. Truy vết chu trình compile–upload–reset–run.
4. Lập bảng tài nguyên Uno R3 cần kiểm tra.
5. Thiết kế checklist nạp Blink có bằng chứng.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

1.7 Ba lớp thường bị gọi chung là Arduino

Arduino có thể chỉ board phần cứng, nền tảng phần mềm/IDE hoặc hệ sinh thái thư viện. “Ngôn ngữ Arduino” chủ yếu là C/C++ cùng các hàm và cấu trúc API như `pinMode`, `digitalWrite`, `millis`. Sketch được biên dịch thành mã máy cho vi điều khiển.

Board phát triển đưa chip, mạch nguồn, USB, reset và header ra dạng dễ dùng. Trong sản phẩm cuối, người thiết kế có thể dùng chính module hoặc đưa vi điều khiển vào PCB riêng. Trong lúc học, board giúp bỏ qua phần hàn và nạp phức tạp.

1.8 Bản đồ Uno R3

Uno R3 dựa trên ATmega328P, có 14 chân digital I/O, trong đó sáu chân có thể tạo PWM, sáu ngõ vào analog và xung hệ thống 16 MHz. Các con số là tài nguyên hữu hạn, không phải để thuộc lòng mà để biết phải kiểm tra khi thiết kế.

- **USB**: cấp nguồn và giao tiếp/nạp từ máy tính; cáp phải truyền dữ liệu.
- **Jack/VIN**: đường cấp nguồn ngoài theo giới hạn tài liệu board; không đồng nghĩa đưa mọi điện áp vào tùy ý.
- **5V, 3.3V, GND**: rail nguồn tham chiếu; khả năng cấp dòng có giới hạn.
- **Digital 0–13**: đọc LOW/HIGH hoặc xuất mức logic; 0/1 còn dùng Serial trên Uno.
- **Analog A0–A5**: nối ADC để đo điện áp trong dải tham chiếu; A4/A5 còn là I2C trên Uno R3.
- **Dấu ~**: chân hỗ trợ PWM trong core Uno; PWM không phải điện áp analog thật.
- **Reset**: khởi động lại sketch từ đầu, không xóa chương trình đã nạp.

Số chân không nói hết khả năng

Một chân có thể dùng chung timer, giao tiếp hoặc chức năng đặc biệt. Thư viện Servo có thể ảnh hưởng PWM ở một số chân trên AVR. Luôn đọc pinout và tài liệu core đúng board, không suy từ Uno sang board khác.

1.9 Từ sketch đến tín hiệu điện

Quy trình:

1. IDE tiền xử lý sketch và gọi compiler C++.
2. Compiler kiểm tra cú pháp/kiểu, liên kết core và thư viện, tạo firmware.
3. Công cụ upload giao tiếp với bootloader qua USB–serial.
4. Board reset, bootloader nhận firmware và ghi vào flash.
5. Vi điều khiển khởi động, chạy `setup()` một lần rồi gọi `loop()` liên tục.

Firmware vẫn ở flash khi rút nguồn. Biến trong SRAM mất khi tắt. EEPROM giữ qua mất nguồn nhưng có giới hạn chu kỳ ghi; chương 16 xử lý.

1.10 Cài IDE và nạp Blink

Arduino IDE 2 là môi trường desktop đang được phát triển tích cực. Sau khi cài, nối board bằng cáp dữ liệu, chọn đúng board và port. Mở `File > Examples > 01.Basics > Blink`. Verify biên dịch; Upload biên dịch và nạp. LED tích hợp thường ký hiệu `LED_BUILTIN` bắt đầu nhấp nháy.

```
1 void setup() {
2   pinMode(LED_BUILTIN, OUTPUT);
3 }
4
```

```
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Nếu upload lỗi, tách ba lớp: máy có thấy cổng không; IDE chọn đúng board/cổng không; board có bị tiến trình khác giữ cổng không. Một cáp chỉ sạc có thể cấp nguồn làm LED sáng nhưng không tạo cổng dữ liệu.

1.11 Compile error và upload error

Compile error xảy ra trước khi nói chuyện với board: thiếu dấu chấm phẩy, tên không tồn tại, thư viện thiếu. Upload error xảy ra sau compile: cổng sai, cáp, quyền, bootloader, chân 0/1 bị thiết bị gây nhiễu. Không đổi code ngẫu nhiên khi lỗi nằm ở kết nối.

1.12 Một mô hình input–state–output

Mọi dự án trong sách có thể nhìn bằng ba khối:

- Input: nút, cảm biến, lệnh Serial.
- State: điều hệ thống nhớ, như chế độ đèn hoặc thời điểm lần trước.
- Output: LED, buzzer, servo, thông điệp.

Vòng `loop` liên tục đọc input, cập nhật state, rồi cập nhật output. Khi code rối, vẽ lại ba khối thường giúp tìm trách nhiệm.

1.13 Bài tập luyện tập

1. Chỉ trên pinout: GND, 5V, A0, LED tích hợp, UART, PWM.
2. Thay Blink thành 200 ms sáng, 800 ms tắt; dự đoán chu kỳ/tần số.
3. Gây một compile error và một upload error có kiểm soát; ghi khác biệt thông báo.
4. Dùng `LED_BUILTIN` thay số chân và giải thích tính di động.
5. Vẽ input–state–output cho đèn tự bật khi tối.
6. So sánh Uno R3 với board bạn có bằng tài liệu chính thức: điện áp logic, số chân, ADC, bộ nhớ.

1.14 Tóm tắt cuối chương

Bản đồ kiến thức

Arduino là hệ sinh thái; sketch là C/C++ dùng API của core. Uno R3 là board tham chiếu, không đại diện mọi board. Firmware được biên dịch, nạp vào flash và chạy setup rồi loop. Compile error khác upload error. Mô hình input–state–output là bản đồ chung của hệ thống nhúng.

Góc nhìn cá nhân

Chiếc LED tích hợp nhấp nháy có vẻ là kết quả nhỏ. Nhưng nó chứng minh cả một chuỗi: ý tưởng thành mã, mã thành machine code, byte đi qua cáp, flash giữ chương trình và chân điện đổi trạng thái theo thời gian. Tôi thích bắt đầu từ Blink vì nó làm chiếc bảng mạch thôi là một vật im lặng.

Chương 2

Điện tử nền tảng – Điện áp đẩy, dòng điện đi và điện trở giới hạn

Giới thiệu ngắn

Code có thể đúng nhưng LED vẫn cháy nếu mạch sai. Ba đại lượng đầu tiên là điện áp V, dòng I, điện trở R. Công thức Ohm không phải mẹo bấm máy; nó là cách dự đoán trước khi cấp nguồn. Cùng với công suất, nối tiếp/song song và cực tính, ta có ngôn ngữ tối thiểu để đọc mạch.

Mục tiêu học tập

- Hiểu điện áp là chênh lệch giữa hai điểm và dòng là dòng điện tích.
- Áp dụng $V = IR$ và $P = VI$ với đơn vị đúng.
- Tính điện trở hạn dòng LED theo giả định.
- Phân biệt nối tiếp, song song, GND tham chiếu và ngắn mạch.
- Dùng multimeter đo điện áp/điện trở an toàn ở mức cơ bản.

2.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một LED nối thẳng 5 V có thể sáng rất mạnh rồi hỏng vì dòng không được giới hạn. Điện áp là chênh lệch thế, dòng là tốc độ dịch chuyển điện tích, điện trở liên hệ hai đại lượng và công suất cho biết tốc độ chuyển năng lượng. Công thức phải được dùng cùng giới hạn linh kiện.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **điện áp, dòng, điện trở và công suất như nền toán học của mạch**, ta luôn đi qua bốn miền:

hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

2.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: điện áp hai đầu, dòng nhánh, điện trở, công suất, cực tính, mass tham chiếu và sai số linh kiện. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$V = IR, \qquad P = VI = I^2R = \frac{V^2}{R}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

2.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

2.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là điện tích/tụ, mức nguồn và nhiệt tích lũy ở linh kiện. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Tổng dòng và áp phải tuân định luật mạch; không linh kiện nào vượt điện áp, dòng hoặc công suất cho phép trong mọi trạng thái.

2.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Chọn điện trở LED với nguồn 5 V, sụt áp LED dự kiến 2 V và dòng mục tiêu 10 mA. Ta tính $R = 300\,\Omega$, chọn giá trị chuẩn 330 Ω, rồi tính lại dòng và công suất điện trở trước khi lắp.

2.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

2.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

2.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

2.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

2.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

2.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

2.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

2.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Dùng định luật Ohm mà không xác định điện áp trên đúng phần tử.
- Nhầm dòng pin với dòng tải toàn mạch.
- Bỏ qua công suất điện trở.
- Đo dòng bằng cách mắc đồng hồ song song nguồn.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

2.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Tính điện trở LED cho ba màu.
2. Kiểm tra công suất của điện trở.
3. Vẽ mạch nối tiếp/song song và áp dụng KCL/KVL.
4. Thiết kế phép đo áp và dòng an toàn.
5. Phân tích ảnh hưởng sai số 5 phần trăm.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

2.7 Điện áp luôn nằm giữa hai điểm

Nói "điểm này 5 V" là nói ngắn của "điểm này cao hơn GND 5 V". GND là mốc tham chiếu chung, không phải hố nuốt điện. Khi hai module trao tín hiệu mà không chung tham chiếu, mức HIGH của bên này có thể vô nghĩa với bên kia.

Dòng điện cần đường kín. Nếu LED có một chân nối 5 V nhưng chân kia không có đường về GND, gần như không có dòng liên tục và nó không sáng.

2.8 Định luật Ohm

$$V = IR, \qquad I = \frac{V}{R}, \qquad R = \frac{V}{I}.$$

Một điện trở 1 kΩ đặt trên 5 V lý tưởng có dòng $5/1000 = 0.005$ A $= 5$ mA. Nhớ đổi kΩ thành Ω, mA thành A khi tính.

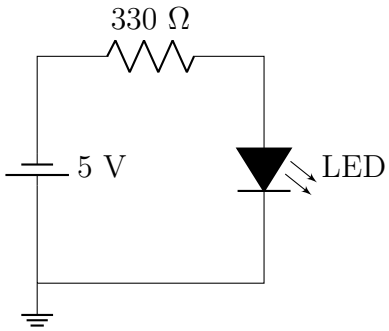
2.9 LED và điện áp thuận

LED là diode phát sáng, dẫn chủ yếu một chiều và có điện áp thuận phụ thuộc màu/dòng/nhiệt độ. Ta không đặt LED trực tiếp giữa chân 5 V và GND vì dòng không được giới hạn tốt.

Giả sử output 5 V, LED đỏ $V_f \approx 2.0$ V, muốn khoảng 10 mA:

$$R = \frac{5 - 2}{0.010} = 300\ \Omega.$$

Chọn giá trị chuẩn 330 Ω cho dòng hơi thấp, an toàn và vẫn sáng. Đây là ước lượng; điện áp output giảm theo tải, V_f phân tán. Không cần đẩy LED đến dòng tối đa để nhìn thấy.



Anode thường chân dài, cathode chân ngắn/cạnh phẳng, nhưng linh kiện đã cắt chân cần datasheet hoặc chế độ diode của đồng hồ.

2.10 Công suất

$$P = VI = I^2R = \frac{V^2}{R}.$$

Điện trở 330 Ω ở 10 mA tiêu tán $0.01^2 \times 330 = 0.033$ W, thấp hơn nhiều điện trở 1/4 W. Nhưng điện trở đặt trực tiếp 12 V, 100 Ω tiêu tán 1.44 W và sẽ quá tải. Kiểm tra công suất không chỉ giá trị ohm.

2.11 Nối tiếp và song song

Điện trở nối tiếp cùng dòng, tổng $R = R_1 + R_2$. Điện trở song song cùng điện áp, tổng thỏa

$$\frac{1}{R} = \frac{1}{R_1} + \frac{1}{R_2} + \cdots.$$

Hai LED song song chỉ dùng một điện trở chung có thể chia dòng không đều vì V_f khác; mỗi nhánh nên có điện trở riêng. Tải song song làm tổng dòng nguồn tăng.

2.12 Ngắn mạch và giới hạn chân

Nối trực tiếp 5 V với GND tạo đường điện trở rất thấp, dòng lớn bị giới hạn bởi nguồn/dây/mạch bảo vệ và có thể làm nóng/hỏng. Cấu hình một chân OUTPUT HIGH rồi nối cứng GND cũng là xung đột.

Thông số dòng của MCU có giới hạn mỗi chân, mỗi nhóm và toàn chip. Giá trị "absolute maximum" không phải điểm vận hành mục tiêu. Dùng LED vài mA; dùng transistor/MOSFET/driver cho tải lớn. Đọc datasheet đúng board/chip.

Không đo dòng như đo điện áp

Đo điện áp: đồng hồ mắc song song giữa hai điểm. Đo dòng: đồng hồ phải nằm nối tiếp và cổng đo đúng; đặt đồng hồ ở thang dòng rồi chạm song song nguồn có thể tạo ngắn mạch qua cầu chì đồng hồ. Khi chưa chắc, ngắt nguồn và nhờ hướng dẫn trực tiếp.

2.13 Pull-up, pull-down như một ý niệm

Một input không nối chắc HIGH hay LOW bị floating và đọc nhiễu. Điện trở kéo đặt trạng thái mặc định yếu; nút nhấn tạo trạng thái mạnh ngược lại. Uno hỗ trợ pull-up nội qua `INPUT_PULLUP`, giúp giảm linh kiện. Khi đó nút thả đọc HIGH, nhấn đọc LOW – logic đảo nhưng ổn định.

2.14 Bài tập luyện tập

1. Tính điện trở LED cho nguồn 3,3 V, $V_f = 2.0$ V, dòng 5 mA; chọn giá trị chuẩn lớn hơn gần nhất.
2. Tính công suất điện trở 220 Ω khi sụt 3 V.
3. Hai điện trở 1 kΩ nối tiếp và song song cho giá trị nào?
4. Vẽ đường dòng cho LED, điện trở và nguồn; đánh dấu anode/cathode.
5. Giải thích vì sao GND chung cần thiết khi servo dùng nguồn ngoài nhưng nhận tín hiệu từ Uno.
6. Đọc datasheet một LED cụ thể: tìm forward voltage tại dòng đo, maximum current và power.

2.15 Tóm tắt cuối chương

Bản đồ kiến thức

Điện áp là chênh lệch; dòng cần vòng kín; điện trở giới hạn dòng. Dùng Ohm và công suất để dự đoán. LED cần đúng cực và điện trở riêng. GND là mốc chung. Ngắn mạch và vượt dòng chân là lỗi phần cứng; absolute maximum không phải mục tiêu thiết kế.

Góc nhìn cá nhân

Công thức Ohm chỉ có ba chữ cái, nhưng nó thay đổi cách tôi lắp mạch: từ “cắm thử xem sao” thành “nếu giả định này đúng, dòng sẽ khoảng bao nhiêu”. Dự đoán trước khi cấp nguồn là một thói quen nhỏ giúp cả linh kiện lẫn người học sống lâu hơn.

Chương 3

Breadboard và LED – Mạch đầu tiên từ sơ đồ đến dây thật

Giới thiệu ngắn

Sơ đồ điện nói các điểm nào nối nhau; breadboard nói chúng nằm ở đâu trên bàn. Người mới thường nhìn dây như hình vẽ và quên rằng hai lỗ cùng hàng đã nối bên trong. Chương này học topology của breadboard, lắp LED ngoài, kiểm tra mạch khi chưa cấp nguồn và tách lỗi phần cứng khỏi code.

Mục tiêu học tập

- Đọc rail nguồn và nhóm lỗ nối trong breadboard.
- Chuyển sơ đồ LED nối tiếp điện trở thành mạch thật.
- Điều khiển LED từ chân digital với trạng thái an toàn lúc khởi động.
- Dùng checklist và multimeter continuity để tìm dây sai.
- Viết thí nghiệm một biến về chu kỳ và độ sáng cảm nhận.

3.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Trên sơ đồ, hai điểm nối nhau bằng một đường. Trên breadboard, các lỗ chỉ nối theo dải kim loại ẩn bên dưới; một dây lệch hàng khiến mạch khác hoàn toàn dù nhìn gần giống. Ta cần ánh xạ từng nút điện trong sơ đồ sang đúng nhóm lỗ.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **breadboard và LED như bài học chuyển sơ đồ thành kết nối vật lý**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp

hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

3.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: nút điện, rail nguồn, hàng nối, cực anode/cathode, điện trở, chân output và dòng LED. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$I_{\mathrm{LED}} \approx \frac{V_{\mathrm{pin}} - V_f}{R}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

3.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

3.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mức logic chân và năng lượng đang cấp cho LED. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Mỗi nút trên sơ đồ tương ứng đúng một nhóm nối; LED luôn có phần tử giới hạn dòng và cực tính đúng.

3.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Lắp LED ngoài nhấp nháy. Ta tô màu các nút 5 V, GND và D8 trên sơ đồ, ánh xạ sang breadboard, kiểm tra continuity khi chưa cấp nguồn, rồi nạp chương trình và đo áp hai đầu LED.

3.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

3.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

3.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

3.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ

liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

3.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

3.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

3.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

3.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Cho hai chân LED vào cùng một dải nối.
- Dùng rail nguồn bị đứt giữa breadboard.
- Đảo LED rồi kết luận code sai.
- Bỏ điện trở vì LED tích hợp trên board vẫn sáng.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

3.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ bản đồ nối trong breadboard.
2. Lắp mạch bằng ba bố cục nhưng cùng netlist.
3. Đo áp chân HIGH/LOW.
4. Tính dòng dự kiến rồi so với số đo.
5. Viết checklist kiểm mạch khi LED không sáng.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

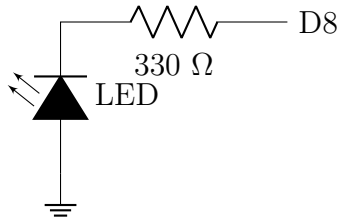
3.7 Bên dưới những lỗ cắm

Ở breadboard phổ biến, mỗi nhóm năm lỗ cùng hàng ở một bên rãnh giữa được nối; hai bên rãnh tách nhau để đặt IC DIP. Rail dọc đỏ/xanh thường nối theo chiều dài nhưng nhiều board bị ngắt giữa. Không tin màu tuyệt đối: kiểm continuity khi chưa cấp nguồn.

Linh kiện nối với nhau khi chân nằm trong cùng node điện, không phải vì dây trông chạm trên hình. Sơ đồ mạch bỏ qua vị trí vật lý; layout breadboard phải hiện thực đúng node.

3.8 Lắp LED ngoài

Mạch: chân D8 → điện trở 330 Ω → anode LED; cathode → GND. Điện trở có thể nằm trước hoặc sau LED trong cùng nhánh nối tiếp vì dòng giống nhau.



Các bước khi rút USB:

1. Nối GND Uno vào rail GND đã chọn.
2. Đặt LED hai chân ở hai nhóm lỗ khác nhau; xác định cathode.
3. Nối cathode về GND.
4. Nối anode qua điện trở đến D8.
5. Dùng mắt đi từ D8 đến GND, đọc tên từng linh kiện.
6. Kiểm tra không có dây 5 V chạm GND, rồi mới cấp nguồn.

3.9 Sketch nhấp nháy có tên rõ

```
const byte LED_PIN = 8;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(250);
  digitalWrite(LED_PIN, LOW);
  delay(750);
}
```

`const byte` ghi một chân không đổi và tiết kiệm bộ nhớ trên AVR. Đặt LOW rõ trong setup khiến trạng thái khởi đầu đọc được, dù sau `pinMode OUTPUT` chân thường mặc định LOW. Chu kỳ là 1000 ms, duty cycle 25%.

3.10 Tần số, chu kỳ và duty cycle

Chu kỳ $T = t_{on} + t_{off}$. Tần số $f = 1/T$ nếu T tính giây. Duty cycle $D = t_{on}/T \times 100\%$. 250 ms sáng, 750 ms tắt: $T = 1$ s, $f = 1$ Hz, $D = 25\%$.

Khi nhấp rất nhanh, mắt tích hợp ánh sáng và thấy độ sáng trung bình thay vì từng xung. Đây là trực giác dẫn đến PWM, nhưng `delay` khóa chương trình nên chưa phải cách tốt cho nhiều nhiệm vụ.

3.11 Cây chẩn đoán LED không sáng

1. LED tích hợp Blink có chạy? Nếu không, kiểm upload/board/cáp.
2. D8 có đổi điện áp so với GND? Đo hoặc chuyển code sang LED tích hợp.
3. GND breadboard có thực sự nối GND Uno?
4. LED đúng cực và hai chân không ở cùng node?
5. Điện trở đúng cỡ, không nhầm 330 kΩ?
6. Dây jumper có continuity?

Đổi cả code, LED và dây cùng lúc làm mất bằng chứng. Bypass từng khối an toàn: thử LED+R trực tiếp 5 V trong thời gian kiểm tra; thử output bằng multimeter; đổi đúng một linh kiện đã biết tốt.

LED cháy hoặc rất nóng

Ngắt nguồn. Kiểm tra điện trở có thật sự nối tiếp, giá trị, cực tính và điện áp. Đừng thay LED mới trước khi tìm đường dòng; linh kiện mới sẽ lặp lại số phận cũ.

3.12 Một thí nghiệm có kiểm soát

Giữ chu kỳ 20 ms, thay duty cycle 10, 25, 50, 75, 90% bằng delayMicroseconds hoặc PWM sau này. Ghi mức sáng cảm nhận, không kết luận mắt tuyến tính. Sau đó giữ duty 50%, thay tần số để tìm vùng còn nhìn thấy nhấp nháy. Mỗi lần chỉ đổi một biến.

3.13 Bài tập luyện tập

1. Lắp hai LED ở D8/D9, mỗi LED có điện trở riêng; nhấp xen kẽ.
2. Viết tín hiệu SOS đơn giản theo chấm/gạch; tách thành hàm.
3. Tính chu kỳ, tần số, duty cho ba cặp thời gian.
4. Đảo cực LED và dự đoán; không để lâu nếu mạch không đúng thông số reverse voltage.
5. Kiểm continuity rail bị ngắt; vẽ sơ đồ nối cầu nếu cần.
6. Tạo checklist ảnh chụp mạch: đánh dấu mỗi node đã xác minh.

3.14 Tóm tắt cuối chương

Bản đồ kiến thức

Breadboard nối các lỗ theo node ẩn; rail có thể ngắt. Chuyển sơ đồ sang layout bằng cách theo đường điện, không theo hình dáng. LED cần đúng cực và điện trở nối tiếp. Chu kỳ/tần số/duty mô tả nhấp nháy. Gỡ lỗi từ nguồn, GND, tín hiệu đến tải và chỉ đổi một yếu tố.

Góc nhìn cá nhân

Mạch đầu tiên dạy một điều code không dạy rõ: hai thứ trông rất gần có thể hoàn toàn không nối, và hai lỗ trông tách lại có thể là cùng một điểm. Học breadboard là học nhìn xuyên qua bề mặt để thấy quan hệ điện đang ẩn phía dưới.

Chương 4

Cấu trúc sketch và Serial – Cho board một tiếng nói

Giới thiệu ngắn

Khi LED không sáng, ta chỉ thấy im lặng. Serial cho vi điều khiển gửi văn bản về máy tính, giống một cửa sổ nhỏ nhìn vào trạng thái. Trước đó, cần hiểu kiểu dữ liệu, phạm vi, setup/loop và khác biệt giữa chương trình biên dịch trên MCU với Python trên máy tính.

Mục tiêu học tập

- Đọc cấu trúc hàm, biến, hằng và kiểu C++ cơ bản trong sketch.
- Hiểu setup chạy một lần, loop lặp nhưng không phải phép màu.
- Khởi tạo và dùng Serial Monitor đúng baud rate.
- In nhãn cùng dữ liệu để log có ý nghĩa.
- Tránh tràn số, chia nguyên và lạm dụng kiểu String trên AVR.

4.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một vi điều khiển sau reset cần cấu hình phần cứng một lần rồi lặp quan sát và phản ứng. `setup()` và `loop()` mô tả vòng đời này. Serial là kênh đo lường bằng thông điệp, nhưng tốc độ truyền hữu hạn nên log quá nhiều có thể thay đổi timing.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **setup, loop và Serial như một hệ thống tuần hoàn có kênh quan sát**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp

hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

4.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: baud rate, số byte, chu kỳ loop, thời điểm reset, cấu hình chân và thông điệp log. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$T_{\text{loop}} = T_{\text{read}} + T_{\text{compute}} + T_{\text{write}} + T_{\text{log}}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

4.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

4.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là cấu hình đã hoàn tất, biến ứng dụng và bộ đệm truyền/nhận. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

setup hoàn tất trước mọi thao tác phụ thuộc cấu hình; lượng log không được làm mất yêu cầu thời gian của hệ thống.

4.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Thiết kế sketch in heartbeat mỗi giây mà không spam mỗi vòng. Ta tính thời gian truyền một dòng ở baud đã chọn, gắn timestamp và dùng điều kiện thời gian để log có kiểm soát.

4.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

4.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

4.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

4.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ

liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

4.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

4.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

4.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

4.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đặt code cần lặp trong setup.
- In Serial mỗi vòng rồi làm loop chậm.
- Không chờ/kiểm tra cổng phù hợp với board.
- Dùng baud Monitor khác baud sketch.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

4.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ lifecycle từ reset.
2. Đo số vòng loop mỗi giây.
3. Tính thời gian truyền 40 byte.
4. Thiết kế định dạng log máy đọc được.
5. Tách log DEBUG khỏi thông điệp giao thức.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

4.7 Một sketch tối thiểu

```
void setup() {
  // Chay mot lan sau khi reset
}

void loop() {
  // Duoc goi lai lien tuc
}
```

`void` nghĩa hàm không trả giá trị. Dấu ngoặc nhọn tạo khối; dấu chấm phẩy kết thúc phần lớn câu lệnh. C++ phân biệt hoa thường. Compiler cần biết kiểu và khai báo trước khi sử dụng.

4.8 Biến và kiểu trên Uno AVR

```
const byte SENSOR_PIN = A0;
int rawValue = 0;
unsigned long lastSampleMs = 0;
float voltage = 0.0;
bool alarm = false;
```

Kích thước kiểu phụ thuộc kiến trúc. Trên Uno AVR điển hình, `int` 16-bit, `long` 32-bit, `float` 32-bit. Trên board khác có thể khác. Dùng `uint8_t`, `int32_t` khi cần chiều rộng rõ; dùng `unsigned long` với `millis()` theo API phổ biến.

Chia nguyên

`5 / 2` cho 2 khi cả hai toán hạng là integer. Viết `5.0 / 2.0` hoặc cast để nhận 2.5. Trong công thức điện áp, một phép chia nguyên sớm có thể biến hầu hết kết quả thành 0.

4.9 Phạm vi và thời gian sống

Biến global tồn tại suốt chương trình và truy cập được nhiều hàm; tiện nhưng tạo phụ thuộc ẩn. Biến local nằm trong khối/hàm và dễ lý luận. `static` local giữ giá trị giữa lần gọi nhưng phạm vi vẫn trong hàm.

```
void countLoops() {
  static unsigned long count = 0;
  count++;
  Serial.println(count);
}
```

Không in mỗi vòng loop không giới hạn; Serial chậm hơn CPU, buffer có thể đầy và làm timing thay đổi.

4.10 Serial.begin và baud

```
void setup() {
  Serial.begin(115200);
  Serial.println(F("Khoi dong he thong"));
}

void loop() {
  int raw = analogRead(A0);
  Serial.print(F("raw="));
  Serial.println(raw);
```

```
  delay(500);
}
```

Serial Monitor phải chọn cùng baud. Ký tự rác thường do lệch baud hoặc reset giữa đường. Macro `F("...")` trên AVR giữ literal trong flash thay vì chép vào SRAM, hữu ích với nhiều chuỗi; khả năng/ý nghĩa có thể khác core.

4.11 Log có cấu trúc

In `123` không cho biết đó là nhiệt độ hay ADC. Dùng nhãn và dấu phân cách ổn định:

```
Serial.print(F("t_ms="));
Serial.print(millis());
Serial.print(F(",raw="));
Serial.print(raw);
Serial.print(F(",voltage="));
Serial.println(voltage, 3);
```

Định dạng key=value dễ đọc và phân tích. Không in dữ liệu quá nhanh; chọn chu kỳ log khác chu kỳ điều khiển. Log có thể làm thay đổi timing, nên bản release có mức log hoặc cờ compile.

4.12 Nhận lệnh đơn giản

```
void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == '1') {
      digitalWrite(LED_BUILTIN, HIGH);
    } else if (command == '0') {
      digitalWrite(LED_BUILTIN, LOW);
    }
  }
}
```

`Serial.read()` trả byte tiếp theo. Monitor có thể gửi thêm `\n`/`\r` tùy line ending; code một ký tự bỏ qua chúng. Giao thức nhiều trường cần buffer, delimiter, timeout và giới hạn chiều dài; chương 13 xây cẩn thận.

4.13 String và mảng char

Lớp `String` tiện nhưng nối/xóa lặp lại trên MCU SRAM nhỏ có thể phân mảnh heap. Với sketch nhỏ, dùng có kiểm soát không phải tội; với hệ thống chạy lâu, buffer `char[]` cố định

và parser giới hạn thường dự đoán hơn. Không tối ưu mù: đo free memory, thời gian và mô phỏng dữ liệu dài.

4.14 Bài tập luyện tập

1. In bảng thời gian `millis`, ADC A0 và trạng thái LED mỗi giây.
2. Gây chia nguyên trong công thức điện áp, quan sát rồi sửa.
3. Viết hàm in một bản ghi CSV; kiểm tra dấu xuống dòng.
4. Nhận lệnh `on/off/status` theo dòng với buffer tối đa 32 ký tự.
5. So sánh lượng log 10 Hz và 1000 Hz; quan sát loop bị ảnh hưởng.
6. Tìm kích thước các kiểu trên board bằng `sizeof` và so với board khác.

4.15 Tóm tắt cuối chương

Bản đồ kiến thức

Sketch C++ được biên dịch và có kiểu/phạm vi rõ. Setup chạy sau reset; loop được core gọi lặp. Serial là kênh quan sát và lệnh, cần cùng baud và định dạng có nhãn. Chú ý chia nguyên, tràn số và bộ nhớ SRAM. Log cũng tiêu tốn thời gian, nên phải có nhịp.

Góc nhìn cá nhân

Serial Monitor giống việc chiếc board học cách kể lại suy nghĩ của mình. Nhưng một lời kể tốt không phải tuôn mọi thứ thật nhanh; nó chọn đúng dấu mốc và đặt tên. Khi log có thời gian, nhãn và nhịp ổn định, gỡ lỗi bớt là đoán và bắt đầu giống một cuộc điều tra.

Phần II

Đọc thế giới và tạo phản hồi

Chương 5

Digital input và nút nhấn – Một tín hiệu chỉ có hai câu trả lời

Giới thiệu ngắn

Nút nhấn tưởng là cảm biến đơn giản nhất: nhấn hoặc không. Nhưng một chân input để hở không có trạng thái chắc, tiếp điểm cơ khí nảy nhiều lần và logic pull-up bị đảo. Chính vì vậy, nút nhấn là bài học hoàn hảo về tín hiệu vật lý, trạng thái ổn định và khác biệt giữa "đang nhấn" với "vừa được nhấn".

Mục tiêu học tập

- Đọc input digital với pull-up/pull-down.
- Lắp nút bằng `INPUT_PULLUP` và hiểu active-low.
- Phân biệt level, edge và event.
- Khử dội bằng thời gian không chặn.
- Thiết kế nhấn ngắn, nhấn giữ và double-click ở mức máy trạng thái.

5.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Tiếp điểm cơ khí không chuyển từ 0 sang 1 hoàn hảo; nó có thể rung nhiều lần trong vài mili giây. Một đầu vào thả nổi lại thu nhiễu và đổi mức không dự đoán được. Mạch kéo và thuật toán debounce cùng cần thiết để biến hiện tượng cơ khí thành một sự kiện số đáng tin.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **nút nhấn số như bài toán mức logic, pull-up và dội tiếp điểm**, ta luôn đi qua bốn miền: hiện

tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

5.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: mức logic, điện trở kéo, trạng thái thô, trạng thái ổn định, thời gian đổi và sự kiện nhấn/thả. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$x(t) \in \{\text{LOW}, \text{HIGH}\}, \qquad e_k = [x_k \neq x_{k-1}] \wedge (\Delta t > T_d)$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

5.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

5.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mẫu trước, mức ứng viên, thời điểm thay đổi cuối và trạng thái đã xác nhận. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Một lần nhấn vật lý tạo tối đa một sự kiện nhấn logic; đầu vào luôn có mức xác định khi công tắc hở.

5.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Đếm số lần nhấn nút. Ta dùng `INPUT_PULLUP` nên logic bị đảo, quan sát waveform dội, chỉ công nhận mức mới sau khoảng ổn định và tăng bộ đếm ở cạnh nhấn thay vì mỗi vòng nút đang giữ.

5.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

5.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

5.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

5.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

5.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

5.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

5.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

5.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đọc nút không có pull-up/pull-down.
- Tăng biến đếm trong toàn bộ thời gian nút giữ.
- Debounce bằng delay làm chặn hệ thống.
- Quên logic đảo của `INPUT_PULLUP`.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

5.6 Bài tập ba tầng và hồ sơ bằng chứng

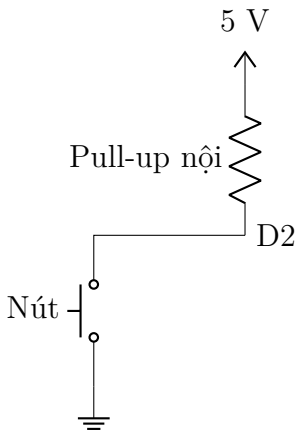
1. Vẽ hai cách mắc pull-up/pull-down.
2. Truy vết thuật toán debounce.
3. Phân biệt level và edge.
4. Thiết kế nhấn ngắn/nhấn giữ.
5. Đo thời gian dội thực tế bằng log.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

5.7 Floating input

Một chân INPUT có trở kháng cao. Nếu không nối về nguồn hoặc GND qua đường xác định, nhiễu điện nhỏ có thể làm kết quả HIGH/LOW thay đổi. Nút mở không được phép để input "trôi".

Với pull-up nội, nối nút giữa chân và GND:



Nút thả: điện trở kéo chân HIGH. Nút nhấn: đường GND mạnh kéo LOW. Không có ngắn mạch 5 V–GND vì pull-up có điện trở lớn bên trong.

```
const byte BUTTON_PIN = 2;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(115200);
}

void loop() {
  bool pressed = digitalRead(BUTTON_PIN) == LOW;
  Serial.println(pressed ? F("PRESSED") : F("RELEASED"));
  delay(50);
}
```

Đặt biến `pressed` chuyển active-low thành ý nghĩa người dùng, để phần còn lại không phải nhớ LOW là nhấn.

5.8 Level khác edge

Nếu viết “khi pressed thì tăng”, loop có thể tăng hàng nghìn lần trong một lần giữ. Sự kiện “vừa nhấn” là chuyển từ released sang pressed.

```
bool previousPressed = false;
unsigned long count = 0;

void loop() {
  bool pressed = digitalRead(BUTTON_PIN) == LOW;
  if (pressed && !previousPressed) {
    count++;
    Serial.println(count);
  }
  previousPressed = pressed;
}
```

Code vẫn bị dội: tiếp điểm rung tạo nhiều edge trong vài millisecond.

5.9 Debounce không chặn

Ta giữ raw reading, thời điểm nó đổi và stable state. Chỉ công nhận nếu raw giữ nguyên đủ lâu.

```
const unsigned long DEBOUNCE_MS = 30;
bool rawLast = false;
bool stablePressed = false;
unsigned long rawChangedAt = 0;

bool updateButton(unsigned long now) {
  bool raw = digitalRead(BUTTON_PIN) == LOW;
  if (raw != rawLast) {
    rawLast = raw;
    rawChangedAt = now;
  }

  if (now - rawChangedAt >= DEBOUNCE_MS && raw != stablePressed) {
    stablePressed = raw;
    return true;  // stable state changed
  }
  return false;
}

void loop() {
  unsigned long now = millis();
  if (updateButton(now) && stablePressed) {
    Serial.println(F("PRESS EVENT"));
  }
}
```

Phép trừ unsigned chịu được `millis` rollover; không so `now >= changed + interval`. Hàm trả event state-changed, còn biến nói state mới.

5.10 Nhấn giữ và nhấn ngắn

Khi nhận press edge, lưu `pressedAt`. Khi release edge, duration = now - pressedAt. Nếu vượt 800 ms là long press, nếu ngắn hơn là click. Nếu muốn long-press event phát ngay khi đang giữ, cần cờ `longEmitted` để chỉ phát một lần.

Double-click cần chờ xem click thứ hai có đến trong cửa sổ, nên single-click bị trì hoãn. Đây là đánh đổi trải nghiệm, không phải chỉ code. Với thiết bị đơn giản, click/long-press dễ dự đoán hơn nhiều tổ hợp bí mật.

5.11 Nhiều nút và lớp Button

Khi có nhiều nút, sao chép biến debounce dễ nhầm. Một struct/class giữ pin và state riêng, với `update(now)`. Hoặc dùng mảng struct và vòng lặp. Tránh cấp phát động; số nút thường biết trước.

Không nối nút giữa 5 V và OUTPUT LOW

Nếu một chân bị cấu hình OUTPUT LOW trong khi nút nối 5 V, nhấn có thể tạo xung đột. Sơ đồ pull-up nối input–GND có trạng thái mặc định an toàn hơn cho bài học.

5.12 Bài tập luyện tập

1. Nút bật/tắt LED mỗi press event, không đổi liên tục khi giữ.
2. Đo raw bounce bằng Serial với timestamp; không kết luận từ một nút duy nhất.
3. Viết lớp DebouncedButton cho hai nút.
4. Phân biệt click dưới 500 ms và long press trên 800 ms; xử lý vùng giữa.
5. Thiết kế double-click, vẽ timeline và nêu độ trễ single-click.
6. Tạo nút emergency ở mức học tập: khi nhấn, output về trạng thái an toàn; giải thích vì sao hệ thống an toàn thật không chỉ dựa software polling.

5.13 Tóm tắt cuối chương

Bản đồ kiến thức

Input phải có trạng thái mặc định; pull-up nội làm nút active-low. Level là trạng thái, edge là chuyển đổi, event là điều đã khử dội và gắn ý nghĩa. Debounce không chặn giữ raw, stable và thời điểm đổi. Tương tác nhấn giữ/double-click là máy trạng thái có đánh đổi trải nghiệm.

Góc nhìn cá nhân

Một nút nhấn dạy rằng thế giới vật lý không đưa cho chương trình những giá trị sạch sẽ. Giữa "không" và "có" là vài millisecond rung, nhiễu và dây. Thuật toán bắt đầu ở việc tôn trọng khoảng xám ấy, không ép hiện thực phải giống biến boolean trong đầu mình.

Chương 6

Analog input và cảm biến – Biến điện áp thành một con số có nghĩa

Giới thiệu ngắn

Digital nói HIGH/LOW; thế giới thường thay đổi liên tục. Biến trở, quang trở và nhiều cảm biến tạo điện áp phụ thuộc đại lượng. Bộ ADC lấy mẫu điện áp và lượng tử hóa thành số nguyên. Con số đó chưa phải lux, độ ẩm hay nhiệt độ; cần mạch, tham chiếu, đường cong và hiệu chuẩn.

Mục tiêu học tập

- Hiểu ADC, độ phân giải, tham chiếu và lượng tử hóa.
- Đọc biến trở an toàn và đổi raw sang điện áp ước lượng.
- Dùng cầu phân áp cho cảm biến điện trở.
- Hiệu chuẩn min/max và ánh xạ có constrain.
- Nhận biết nhiễu, trở nguồn và giới hạn của `map`.

6.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Biến trở tạo điện áp liên tục nhưng ADC trả về một trong các mã rời rạc. Mã 512 chỉ có nghĩa khi biết độ phân giải và điện áp tham chiếu. Cảm biến thêm đặc tuyến, offset, gain và nhiễu; chuyển đổi thẳng bằng map không thay thế hiệu chuẩn.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **ADC và cảm biến như phép lấy mẫu, lượng tử hóa và hiệu chuẩn**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp

hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

6.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: điện áp vào, Vref, số bit ADC, mã thô, đặc tuyến cảm biến, offset, gain và nhiễu. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$q = \mathrm{round}\left(\frac{V_{\mathrm{in}}}{V_{\mathrm{ref}}}(2^N - 1)\right), \qquad V_{\mathrm{in}} \approx \frac{qV_{\mathrm{ref}}}{2^N - 1}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

6.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

6.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mẫu thô, mẫu đã lọc, hệ số hiệu chuẩn và giá trị vật lý. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k, x_k, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Điện áp vào luôn trong miền ADC; kết quả công bố kèm đơn vị, miền hiệu chuẩn và độ phân giải thực.

6.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Hiệu chuẩn cảm biến ánh sáng tại hai mức tham chiếu. Ta thu nhiều mẫu mỗi mức, lấy trung bình, suy ra đường tuyến tính, kiểm tra điểm thứ ba và chỉ dùng nội suy trong miền đã đo.

6.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

6.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

6.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

6.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

6.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

6.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

6.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

6.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đưa điện áp vượt Vref vào chân analog.
- Xem độ phân giải danh nghĩa là độ chính xác.
- Map hai điểm rồi ngoại suy vô hạn.
- Đặt ngưỡng từ một mẫu duy nhất.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

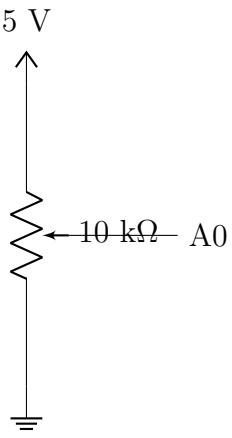
6.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Tính kích thước một LSB.
2. Thu histogram ADC khi tín hiệu ổn định.
3. Hiệu chuẩn hai điểm và kiểm tra residual.
4. So sánh Vref nguồn với tham chiếu nội.
5. Thiết kế báo lỗi khi ngoài miền hiệu chuẩn.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

6.7 Biến trở như cầu phân áp điều chỉnh

Nối hai chân ngoài 5 V/GND, chân giữa (wiper) vào A0. Xoay thay tỉ lệ điện trở, điện áp wiper từ gần 0 đến gần 5 V.



Trước khi cấp nguồn, xác nhận chân giữa theo datasheet/đo; biến trở khác package có pinout khác.

6.8 ADC và độ phân giải

Trên Uno R3, `analogRead` thường trả 0–1023 (10 bit), biểu diễn 1024 mức từ 0 đến điện áp tham chiếu. Điện áp ước lượng:

$$V_{in} \approx \frac{raw}{1023} V_{ref}.$$

```
const byte POT_PIN = A0;

void loop() {
  int raw = analogRead(POT_PIN);
  float voltage = raw * (5.0 / 1023.0);
  Serial.print(F("raw="));
  Serial.print(raw);
  Serial.print(F(",V="));
  Serial.println(voltage, 3);
  delay(100);
}
```

5.0 V chỉ là giả định. USB có thể không đúng 5.000 V và tham chiếu/ADC có sai số. Nếu cần chính xác, đo Vref, dùng tham chiếu phù hợp, hiệu chuẩn toàn hệ thống và đọc datasheet.

6.9 Lượng tử hóa và nhiễu

ADC trả mức rời rạc; một LSB xấp xỉ $V_{ref}/1024$. Giá trị đứng gần ranh giới có thể nhảy giữa hai mã. Dây dài, nguồn, breadboard và tải số tạo nhiễu. Đừng mong raw đứng im tuyệt đối.

Biện pháp: wiring ngắn/gọn, tụ decoupling theo datasheet, trở nguồn phù hợp, lấy mẫu/lọc, tham chiếu ổn định. Không dùng phần mềm để che một mạch sai.

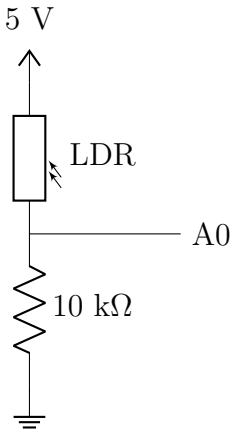
6.10 Cầu phân áp với quang trở

Hai điện trở nối tiếp giữa Vcc/GND, điểm giữa vào ADC:

$$V_{out} = V_{cc} \frac{R_2}{R_1 + R_2}.$$

Nếu quang trở ở phía Vcc và điện trở cố định phía GND, sáng làm điện trở LDR giảm, V_{out} tăng. Đảo vị trí thì chiều thay đổi đảo. Sách/project cần nói rõ sơ đồ trước khi nói “giá

trị sáng lớn".



6.11 Ánh xạ và giới hạn

```
int percent = map(raw, rawDark, rawBright, 0, 100);
percent = constrain(percent, 0, 100);
```

`map` dùng số nguyên và không tự constrain; nếu raw ngoài khoảng hiệu chuẩn, kết quả vượt 0–100. Nếu `rawDark == rawBright`, phép chia bên trong không hợp lệ. Hiệu chuẩn phải kiểm tra span tối thiểu.

Một hàm float rõ hơn:

```
float mapFloat(float x, float inMin, float inMax,
               float outMin, float outMax) {
  if (inMax == inMin) return outMin;
  float ratio = (x - inMin) / (inMax - inMin);
  ratio = constrain(ratio, 0.0, 1.0);
  return outMin + ratio * (outMax - outMin);
}
```

6.12 Hiệu chuẩn hai điểm và nhiều điểm

Hai điểm giả định quan hệ tuyến tính. Nhiều cảm biến như LDR không tuyến tính, và "độ sáng phần trăm" chỉ là thang tương đối. Với cảm biến nhiệt analog có datasheet tuyến tính, dùng công thức đặc trưng rồi hiệu chuẩn offset/gain bằng chuẩn.

Quy trình:

1. Làm ấm thiết bị nếu cần, giữ điều kiện ổn định.
2. Ghi nhiều mẫu tại mỗi điểm, dùng trung bình/trung vị.
3. Ghi giá trị chuẩn và raw, không chỉ hệ số cuối.
4. Fit mô hình phù hợp, kiểm tra trên điểm chưa dùng fit.

5. Lưu ngày, board, Vref và điều kiện môi trường.

Không đưa điện áp vượt rail vào ADC

ADC không phải voltmeter đa năng. Chân analog vẫn có giới hạn điện áp theo board. Muốn đo nguồn cao hơn cần cầu chia áp/bảo vệ được thiết kế và xác minh; không áp dụng công thức mà bỏ qua transient, sai số và mức tuyệt đối.

6.13 Bài tập luyện tập

1. Đọc biến trở, in raw/voltage/phần trăm; đo Vcc thật và so.
2. Ghi 200 mẫu cố định, tính min/max/mean và độ dao động.
3. Lắp LDR hai chiều cầu phân áp, giải thích dấu của đáp ứng.
4. Hiệu chuẩn cảm biến tương đối hai điểm và từ chối span quá nhỏ.
5. So sánh `map` integer với mapFloat trên các giá trị lẻ.
6. Thiết kế thí nghiệm kiểm tra tuyến tính: ít nhất năm mức chuẩn và đồ thị residual.

6.14 Tóm tắt cuối chương

Bản đồ kiến thức

ADC lượng tử hóa điện áp so với tham chiếu; raw chưa phải đơn vị vật lý. Cầu phân áp biến điện trở thành điện áp, chiều đáp ứng phụ thuộc vị trí linh kiện. Map cần constrain và kiểm span. Hiệu chuẩn là mô hình cộng dữ liệu chuẩn, phải ghi điều kiện và kiểm tra ngoài điểm fit.

Góc nhìn cá nhân

Cảm biến không “đưa nhiệt độ” cho Arduino; nó đưa một tín hiệu, rồi chúng ta kể một câu chuyện toán học để biến tín hiệu thành đại lượng. Khoảng cách giữa hai thứ ấy là nơi tồn tại sai số, giả định và trách nhiệm phải ghi lại.

Chương 7

PWM, LED RGB và buzzer – Tạo mức trung gian bằng thời gian

Giới thiệu ngắn

Chân digital chỉ HIGH hoặc LOW, nhưng LED có thể trông sáng ở nhiều mức. PWM bật/tắt nhanh với duty cycle khác nhau để tải chậm hoặc giác quan cảm nhận giá trị trung bình. Cùng nguyên lý điều khiển độ sáng, tốc độ qua driver và âm lượng giới hạn; `tone` dùng tần số để tạo nốt.

Mục tiêu học tập

- Hiểu PWM là chuỗi xung, không phải điện áp DC thật.
- Dùng `analogWrite` đúng chân và dải của Uno.
- Trộn màu RGB, hiểu common anode/cathode và gamma cảm nhận.
- Phân biệt buzzer active/passive và dùng `tone`.
- Nhận biết xung đột timer, tải và giới hạn dòng.

7.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một chân số chỉ HIGH hoặc LOW, nhưng nếu chuyển đủ nhanh, LED nhận công suất trung bình thay đổi và mắt thấy độ sáng trung gian. PWM điều khiển duty cycle chứ không tạo điện áp DC lý tưởng. Tần số, tải và bộ lọc quyết định đầu ra được cảm nhận thế nào.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **PWM như điều chế năng lượng theo tỉ lệ thời gian**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín

hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

7.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: duty cycle, tần số, chu kỳ, điện áp đỉnh, đáp ứng tải, kênh màu và tần số âm. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$D = \frac{t_{\text{on}}}{T}, \qquad \bar{V} \approx D\,V_{\text{cc}}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

7.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

$$\text{Hiện tượng} \longrightarrow \text{cảm biến/mạch vào} \longrightarrow \text{ADC hoặc chân số} \longrightarrow \text{thuật toán} \longrightarrow \text{mạch công suất} \longrightarrow \text{tác động}.$$

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

7.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là giá trị duty của từng kênh, timer dùng chung và pha/tần số hiện tại. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Duty nằm trong miền cho phép; tài nguyên timer không xung đột và dòng mỗi kênh được giới hạn.

7.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Pha màu RGB bằng ba duty cycle. Ta chuẩn hóa giá trị, xét LED common anode hay cathode, đo dòng từng kênh, rồi thử gamma đơn giản để bước sáng cảm nhận đều hơn.

7.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

7.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

7.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

7.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

7.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

7.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

7.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

7.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Gọi PWM là điện áp analog thật.
- Không xét loại common anode của LED RGB.
- Dùng timer chung rồi làm Servo/tone xung đột.
- Điều khiển buzzer thụ động như buzzer chủ động.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

7.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Đo waveform PWM ở ba duty.
2. Tính điện áp trung bình dự kiến.
3. Tạo dải màu có giới hạn tổng dòng.
4. So sánh buzzer active/passive.
5. Thiết kế fade không dùng delay.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

7.7 Duty cycle và giá trị analogWrite

Trên core Uno truyền thống, `analogWrite(pin, value)` dùng giá trị 0–255. Duty xấp xỉ $value/255$. 128 gần 50%. Tần số phụ thuộc chân/timer; không giả định giống mọi board.

```
const byte LED_PIN = 9;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  for (int level = 0; level <= 255; level++) {
    analogWrite(LED_PIN, level);
    delay(5);
  }
```

```
  for (int level = 255; level >= 0; level--) {
    analogWrite(LED_PIN, level);
    delay(5);
  }
}
```

Ví dụ dùng delay để tập trung PWM; chương 9 sẽ thay bằng fade không chặn. Trên chân không PWM, hành vi core có thể chỉ LOW/HIGH theo ngưỡng, không tạo fade.

7.8 Điện áp trung bình không kể toàn bộ

Multimeter có thể đọc giá trị trung bình, nhưng oscilloscope thấy xung đủ biên. LED/motor phản ứng theo động lực riêng. Không dùng PWM trực tiếp như nguồn analog sạch cho cảm biến; muốn lọc thành điện áp cần RC và xem tải/tần số/ripple, hoặc dùng DAC trên board hỗ trợ.

7.9 LED RGB

RGB chứa ba LED, mỗi kênh cần điện trở riêng vì V_f khác. Common cathode nối chân chung GND, giá trị PWM lớn sáng hơn. Common anode nối chung Vcc, logic đảo: 0 sáng tối đa, 255 tắt (theo cấu hình thông thường).

```
const byte R_PIN = 9;
const byte G_PIN = 10;
const byte B_PIN = 11;
const bool COMMON_ANODE = false;

void writeChannel(byte pin, byte value) {
  analogWrite(pin, COMMON_ANODE ? 255 - value : value);
}

void setColor(byte red, byte green, byte blue) {
  writeChannel(R_PIN, red);
  writeChannel(G_PIN, green);
  writeChannel(B_PIN, blue);
}
```

Các kênh 255 không tạo “trắng chuẩn” do hiệu suất LED và mắt khác. Hiệu chỉnh hệ số từng kênh. Độ sáng cảm nhận phi tuyến; bước số bằng nhau không trông bằng nhau. Có thể áp gamma lookup table để fade mượt thị giác.

7.10 Buzzer active và passive

Buzzer active tự dao động khi cấp DC, chủ yếu bật/tắt. Buzzer passive/piezo cần tín hiệu tần số, dùng `tone(pin, frequency, duration)` và `noTone(pin)`. Kiểm module và dòng; không giả định mọi buzzer lái trực tiếp từ chân.

```
const byte BUZZER_PIN = 6;

void setup() {
  tone(BUZZER_PIN, 440, 300);  // A4 trong 300 ms
}

void loop() {}
```

Tạo giai điệu cần lịch nốt/khoảng nghỉ không chặn nếu hệ thống còn việc. `tone` dùng timer, có thể xung đột PWM/thư viện tùy board. Đọc tài liệu core.

7.11 Gamma lookup

Thay vì tính `pow` mỗi vòng trên MCU nhỏ, tạo bảng 256 byte trong flash hoặc bảng ít mức. Công thức chuẩn hóa gần $out = 255(in/255)^{\gamma}$, γ khoảng 2–2.8 tùy mục tiêu. Đây là chỉnh cảm nhận, không làm LED tuyến tính vật lý.

7.12 Driver cho tải lớn

PWM chân MCU chỉ là tín hiệu điều khiển. Motor, dải LED và buzzer công suất cần transistor/MOSFET/driver, điện trở gate/base theo thiết kế, diode flyback với tải cảm và nguồn đủ. Không cấp dòng tải qua chân I/O. Common ground và decoupling đúng.

Tải cảm tạo xung áp

Cuộn motor, relay, solenoid chống lại thay đổi dòng và có thể tạo điện áp cao khi tắt. Dùng driver/module có diode flyback hoặc mạch dập phù hợp. Không thử nối cuộn trực tiếp vào chân rồi “xem có quay không”.

7.13 Bài tập luyện tập

1. Đo duty 25/50/75% bằng oscilloscope nếu có, hoặc đo trung bình và nói giới hạn phép đo.
2. Fade LED không dùng float; tính thời gian chu kỳ.
3. Lắp RGB mỗi kênh điện trở riêng, xác định common bằng datasheet/đo.
4. Tạo bảng sáu màu và chuyển màu theo nút nhấn.

5. Tạo ba nốt cảnh báo có ý nghĩa khác nhau nhưng không gây khó chịu.
6. Nghiên cứu timer mapping của board: PWM pin nào chung timer, tone/Servo ảnh hưởng gì.

7.14 Tóm tắt cuối chương

Bản đồ kiến thức

PWM là HIGH/LOW theo duty, không phải DAC. AnalogWrite trên Uno thường 0–255 và chỉ đúng PWM ở chân hỗ trợ. RGB cần ba điện trở, logic phụ thuộc common anode/cathode; gamma cải thiện cảm nhận. Buzzer passive dùng tần số. Tải lớn/cảm cần driver và bảo vệ.

Góc nhìn cá nhân

PWM là một ví dụ đẹp về việc thời gian tạo ra cảm giác của một đại lượng mới. Chân không bao giờ ở “nửa HIGH”, nhưng khi bật tắt đủ nhanh, LED trông như sáng một nửa. Hiểu sự khác biệt giữa tín hiệu thật và điều hệ thống cảm nhận giúp ta không dùng đúng kết quả cho sai mục đích.

Chương 8

Servo, motor và relay – Khi tín hiệu nhỏ điều khiển chuyển động lớn

Giới thiệu ngắn

LED tiêu thụ ít và không chuyển động. Servo có dòng khởi động/stall, motor DC tạo nhiễu và relay là cuộn cảm. Chân Arduino chỉ phát tín hiệu; tầng công suất cung cấp năng lượng và bảo vệ. Chương này tập trung kiến trúc điều khiển an toàn hơn, không hướng dẫn đấu điện lưới.

Mục tiêu học tập

- Điều khiển servo hobby bằng thư viện và nguồn phù hợp.
- Hiểu transistor/MOSFET/driver tách tín hiệu khỏi công suất.
- Điều khiển motor một chiều qua driver, PWM và H-bridge ở mức khái niệm.
- Dùng relay module chỉ với tải điện áp thấp trong thực hành.
- Thiết kế trạng thái khởi động/tắt an toàn và common ground.

8.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Servo, motor DC và relay đều biến tín hiệu điều khiển nhỏ thành hành động lớn nhưng nhu cầu điện khác nhau. Motor có dòng khởi động và sức điện động ngược; relay có cuộn dây cảm; servo tạo xung dòng theo tải. Chân I/O chỉ cung cấp tín hiệu điều khiển, không phải nguồn công suất.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **cơ cấu chấp hành như tải năng lượng cần tầng công suất và phản hồi**, ta luôn đi qua bốn miền:

hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

8.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: điện áp tải, dòng thường/đỉnh, tín hiệu điều khiển, tầng driver, diode bảo vệ, góc/tốc độ và phản hồi. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$P_{\text{load}} = VI, \qquad I_{\text{stall}} \gg I_{\text{no-load}}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

8.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

8.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là lệnh mong muốn, trạng thái enable, timeout và trạng thái phản hồi nếu có. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Tải luôn có nguồn và driver phù hợp; khi reset hoặc mất tín hiệu, lệnh đầu ra trở về trạng thái an toàn.

8.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Điều khiển servo quét góc và relay mô phỏng bơm. Ta tách nguồn tải, nối chung GND khi cần, giới hạn thời gian relay bật, kiểm tra sụt áp khi servo chuyển động và không suy trạng thái cơ khí chỉ từ lệnh.

8.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

8.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

8.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

8.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

8.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

8.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

8.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

8.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Cấp motor/servo trực tiếp từ chân I/O.
- Quên diode flyback với tải cảm DC.
- Dùng chung nguồn yếu làm board reset.
- Cho rằng lệnh góc chứng minh servo đã tới góc.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

8.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Lập ngân sách dòng cho ba tải.
2. Vẽ driver transistor và diode.
3. Thiết kế timeout đầu ra.
4. Đo sụt áp khi tải khởi động.
5. Thêm cảm biến phản hồi trạng thái.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

8.7 Servo hobby

Servo ba dây thường gồm nguồn, GND, signal; màu không phải tiêu chuẩn tuyệt đối. Signal nhận xung vị trí khoảng chu kỳ 20 ms, độ rộng theo góc tùy servo. Thư viện Servo che timing.

```
#include <Servo.h>

Servo arm;
const byte SERVO_PIN = 9;

void setup() {
  arm.attach(SERVO_PIN);
  arm.write(90);
}

```

```
void loop() {}
```

Không giả định `write(0)` đến `write(180)` là an toàn cơ khí. Cơ cấu có giới hạn, servo có sai số; hiệu chuẩn min/max dần từ giữa, nghe rung/kẹt và dừng khi bất thường.

8.8 Nguồn servo

Servo có thể hút dòng cao khi khởi động hoặc bị kẹt, làm sụt 5 V và reset Uno. Dùng nguồn 5–6 V phù hợp servo với đủ dòng, nối GND nguồn servo và GND Arduino để signal có tham chiếu. Không nối hai nguồn 5 V với nhau tùy tiện; sơ đồ phân phối nguồn cần rõ.

Tụ bulk gần servo có thể giảm sụt ngắn nhưng không thay nguồn thiếu công suất. Không dùng servo như cơ cấu an toàn giữ tải có thể gây thương tích.

8.9 Motor DC và transistor

Chân I/O không cấp motor. Một MOSFET logic-level low-side có thể bật/tắt một chiều: motor nối nguồn ngoài, đầu kia drain, source GND, gate nhận PWM qua điện trở phù hợp, có pull-down và diode flyback song song motor theo cực đúng. Chọn MOSFET theo $R_{DS(on)}$ ở điện áp gate thật, dòng/thermal và transient, không chỉ dòng headline.

Với người mới, module driver motor có datasheet rõ an toàn hơn lắp công suất rời. Vẫn phải xem điện áp motor, logic, dòng liên tục/đỉnh và tản nhiệt.

8.10 H-bridge và đổi chiều

H-bridge đổi cực điện áp trên motor bằng bốn khóa. Driver cung cấp hai input chiều và enable/PWM. Không bật hai khóa cùng nhánh gây shoot-through; module/core thường có logic bảo vệ nhưng code phải tuân bảng chân lý.

```
const byte IN1 = 7;
const byte IN2 = 8;
const byte EN  = 5;  // PWM

void setMotor(int speedValue) {
  speedValue = constrain(speedValue, -255, 255);
  if (speedValue > 0) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    analogWrite(EN, speedValue);
  } else if (speedValue < 0) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    analogWrite(EN, -speedValue);
  } else {
```

```
    analogWrite(EN, 0);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
  }
}
```

Đảo chiều tức thì khi motor đang nhanh gây dòng/cơ khí lớn. Ramp về 0, chờ phù hợp, rồi đổi hướng. Brake/coast phụ thuộc driver và bảng chân lý.

8.11 Relay module

Relay điện cơ dùng cuộn để đổi tiếp điểm, tạo cách ly cơ khí giữa coil và contact, nhưng module rẻ có thiết kế cách ly/clearance khác nhau. Trong sách chỉ đóng cắt tải DC điện áp thấp như đèn 5–12 V có nguồn giới hạn. Không đấu 220 V.

Module có thể active-low. Trạng thái lúc reset chân input floating có thể làm relay chớp. Dùng pull-up/down phần cứng theo module, chọn pin và trình tự setup để output an toàn trước khi điều khiển.

```
const byte RELAY_PIN = 4;
const byte RELAY_OFF = HIGH;  // module active-low, phai xac minh
const byte RELAY_ON  = LOW;

void setup() {
  digitalWrite(RELAY_PIN, RELAY_OFF);
  pinMode(RELAY_PIN, OUTPUT);
}
```

Viết latch output trước khi chuyển pin OUTPUT giúp giảm glitch trên AVR phổ biến; vẫn cần xác minh board/module và phần cứng fail-safe.

8.12 Trạng thái an toàn

Hãy định nghĩa output khi: vừa cấp nguồn, code chưa setup, mất tín hiệu, sensor lỗi, watchdog reset, người dùng nhấn stop. Với motor, thường là disable/coast hoặc brake tùy nguy cơ. Với heater/pump, thường OFF, nhưng có quy trình cần làm mát. An toàn là phân tích hệ thống, không mặc định LOW.

Không dùng relay hobby cho điện lưới trong tự học

Điện lưới có nguy cơ chết người/cháy, cần enclosure, cầu chì, khoảng cách cách điện, tiếp địa, dây/đầu nối đúng chuẩn và người đủ năng lực. Xếp hạng in trên relay không chứng nhận toàn bộ module hay cách lắp. Thực hành chỉ với nguồn DC thấp, giới hạn dòng.

8.13 Bài tập luyện tập

1. Điều khiển servo ba vị trí, giới hạn góc đã hiệu chuẩn và chuyển mượt không chặn.
2. Đo sụt nguồn khi servo chuyển tải nhẹ; không cố tình stall lâu.
3. Viết hàm motor có ramp tốc độ và interlock đổi chiều.
4. Đọc bảng chân lý driver bạn có, viết test từng trạng thái khi motor tháo tải.
5. Điều khiển relay với tải LED DC riêng; xác minh active-low và startup.
6. Lập bảng fail-safe cho mất nguồn logic, mất nguồn motor, dây sensor đứt, code treo.

8.14 Tóm tắt cuối chương

Bản đồ kiến thức

Servo/motor/relay cần năng lượng lớn hơn chân I/O; Arduino chỉ gửi tín hiệu đến tầng driver. Servo cần nguồn đủ và common ground. Motor cảm cần diode/driver; H-bridge đổi chiều với interlock. Relay trong sách chỉ dùng tải DC thấp. Trạng thái an toàn phải được định nghĩa cho khởi động và lỗi, có hỗ trợ phần cứng.

Góc nhìn cá nhân

Khi output bắt đầu chuyển động, lỗi không còn chỉ là một pixel sai. Một cánh tay servo có thể kẹt, motor có thể kéo dòng lớn, relay có thể bật lúc reset. Từ đây, tôi luôn tách câu hỏi "tín hiệu nào" khỏi "năng lượng đến từ đâu" và "nếu phần mềm im lặng thì hệ thống làm gì".

Phần III

Thoát khỏi chiếc bẫy delay

Chương 9

millis và bộ lập lịch nhỏ – Thoát khỏi chiếc bẫy delay

Giới thiệu ngắn

`delay(1000)` khiến LED dễ nhấp nháy nhưng trong một giây ấy, chương trình không kiểm nút, không cập nhật servo và không gửi cảnh báo. Vi điều khiển không cần thread để làm nhiều việc phản hồi; nó cần các tác vụ ngắn, nhớ lần cuối và chỉ làm khi đến hạn. `millis()` là chiếc đồng hồ cho kiến trúc không chặn.

Mục tiêu học tập

- Hiểu `millis` là bộ đếm unsigned và sẽ rollover.
- Viết timer theo phép trừ an toàn qua rollover.
- Chọn lịch fixed-delay hoặc fixed-rate.
- Điều phối nhiều tác vụ với chu kỳ khác nhau.
- Đo loop latency, jitter và tránh công việc chặn dài.

9.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một trạm nhỏ phải chớp LED mỗi 500 ms, đọc cảm biến mỗi giây và cập nhật màn hình mỗi 200 ms. Ba chuỗi delay nối tiếp làm nhiệm vụ này chặn nhiệm vụ kia. Lập lịch bằng thời điểm đến hạn cho phép loop quay nhanh và mỗi tác vụ tự quyết định khi nào chạy.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **millis và lập lịch hợp tác như mô hình nhiều nhiệm vụ theo thời gian**, ta luôn đi qua bốn miền:

hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

9.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: thời gian hiện tại, chu kỳ từng tác vụ, thời điểm chạy cuối, thời gian thực thi và jitter. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$\text{due} \iff (t_{\text{now}} - t_{\text{last}}) \geq T$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

9.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

9.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là deadline kế tiếp, nhiệm vụ đang chạy và số lần trễ. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Không tác vụ nào chặn loop lâu hơn ngân sách; so sánh thời gian phải an toàn qua overflow của bộ đếm.

9.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Lập lịch ba nhiệm vụ với chu kỳ khác nhau. Ta đo thời gian thực thi, dùng phép trừ unsigned, quyết định chính sách bù trễ và ghi log jitter thay vì chỉ nhìn LED.

9.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

9.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

9.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

9.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

9.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

9.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

9.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

9.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- So sánh `millis() > last + period` dễ lỗi overflow.
- Cập nhật last bằng now mà không hiểu drift.
- Một task chạy quá lâu làm mọi task trễ.
- Dùng scheduler nhưng vẫn chứa delay bên trong task.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

9.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ timeline ba tác vụ.
2. Tính utilization gần đúng.
3. Mô phỏng overflow.
4. So sánh hai chính sách cập nhật deadline.
5. Đo jitter từ log timestamp.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

9.7 Mẫu timer cơ bản

```
const unsigned long BLINK_MS = 500;
unsigned long lastBlink = 0;
bool ledOn = false;

void loop() {
  unsigned long now = millis();
  if (now - lastBlink >= BLINK_MS) {
    lastBlink = now;
    ledOn = !ledOn;
    digitalWrite(LED_BUILTIN, ledOn);
  }
}
```

Không chờ. Mỗi vòng hỏi “đã đủ thời gian chưa?”. Trong lúc chưa đủ, loop làm việc khác.

9.8 Rollover

Trên Uno với bộ đếm 32-bit millisecond, `millis()` quay về 0 sau khoảng 49,7 ngày. Phép trừ unsigned modulo vẫn cho elapsed đúng miễn khoảng cần đo nhỏ hơn nửa miền phù hợp.

```
if (now - last >= interval) { ... }       // mau dung
if (now >= last + interval) { ... }       // co the sai qua rollover
```

Không "reset millis"; viết code đúng modulo. Dùng cùng kiểu unsigned long cho timestamp/interval.

9.9 Fixed-delay và fixed-rate

```
last = now;          // fixed-delay: chu ky tinh tu luc thuc thi
last += interval;    // fixed-rate: giu luoi thoi gian ly tuong
```

Fixed-delay tránh chạy dồn nhưng trôi theo thời gian thực thi. Fixed-rate giảm drift; nếu loop trễ nhiều chu kỳ, điều kiện vẫn đúng và có thể chạy dồn ở các vòng sau. Một scheduler cần chính sách bỏ lỡ: catch up mọi tick, chạy một lần, hay bỏ đến tick tương lai.

```
if (now - lastSample >= SAMPLE_MS) {
  do {
    lastSample += SAMPLE_MS;
  } while (now - lastSample >= SAMPLE_MS);
  sampleSensor();  // mot lan, bo cac tick cu
}
```

Vòng do chỉ điều chỉnh timestamp; với interval hợp lý. Không dùng cho interval 0.

9.10 Nhiều tác vụ

```
void loop() {
  unsigned long now = millis();
  updateButton(now);       // moi loop, ham ngan
  updateLed(now);          // 20 ms
  sampleSensor(now);       // 100 ms
  updateControl(now);      // 100 ms
  writeLog(now);           // 1000 ms
}
```

Mỗi hàm tự giữ timestamp/state hoặc nhận một struct timer. Thứ tự có ý nghĩa: đọc input trước control, cập nhật output sau. Dùng cùng `now` tạo ảnh chụp thời gian nhất quán và tránh gọi millis nhiều lần.

9.11 Một struct Periodic

```
struct Periodic {
  unsigned long interval;
  unsigned long last;

  bool due(unsigned long now) {
    if (now - last < interval) return false;
    last += interval;
    return true;
  }
};

Periodic sensorTimer{100, 0};
Periodic logTimer{1000, 0};
```

Nếu loop trễ hơn nhiều interval, phiên bản này sẽ trả true nhiều vòng để catch up. Hãy ghi chính sách trong tên/tài liệu và thêm xử lý missed periods nếu cần.

9.12 Micros và giới hạn độ chính xác

`micros()` cho độ phân giải nhỏ hơn nhưng cũng rollover nhanh hơn và độ phân giải thật phụ thuộc board. Không dùng loop software cho giao thức yêu cầu timing nghiêm nếu phần cứng timer/peripheral hỗ trợ. Interrupt, PWM hardware và thư viện giao thức tồn tại vì loop có jitter.

9.13 Đo thời gian loop

```
unsigned long previousLoopUs = 0;
unsigned long maxLoopUs = 0;

void loop() {
  unsigned long start = micros();
  // update tasks...
  unsigned long duration = micros() - start;
  if (duration > maxLoopUs) maxLoopUs = duration;
}
```

In max theo chu kỳ chậm, không in mỗi loop. Response time nút bị giới hạn bởi loop dài nhất cộng debounce. Một lần `pulseIn` timeout hoặc Serial write blocking có thể phá giả định.

Không chặn dưới tên khác

Rời `delay` nhưng dùng vòng `while` chờ sensor, `readStringUntil` timeout dài hoặc animation for-loop có delay vẫn chặn. Mỗi update nên làm một bước nhỏ và quay lại nhanh.

9.14 Bài tập luyện tập

1. Hai LED nhấp chu kỳ 300/700 ms, nút vẫn phản hồi.
2. Fade PWM không chặn: cập nhật mỗi 10 ms và đảo chiều ở biên.
3. So sánh drift fixed-delay/fixed-rate trong 10 phút bằng log timestamp.
4. Mô phỏng rollover bằng kiểu `uint8_t` hoặc hàm nhận clock giả.
5. Đo max loop time khi bật/tắt log; tìm thao tác chậm nhất.
6. Thiết kế scheduler có chính sách skip missed periods và đếm số tick bỏ.

9.15 Tóm tắt cuối chương

Bản đồ kiến thức

Millis timer dùng elapsed = now - last để an toàn rollover. Fixed-delay trôi nhưng không dồn; fixed-rate giữ lưới nhưng cần chính sách tick bị lỡ. Loop hợp tác gọi nhiều update ngắn. Đo latency/jitter và tìm mọi thao tác chặn, không chỉ delay.

Góc nhìn cá nhân

Thoát khỏi delay là lúc chương trình ngừng “làm xong một việc rồi mới nhìn thế giới”. Nó bắt đầu quay lại thường xuyên, giữ lời hẹn nhỏ với từng nhiệm vụ. Sự phản hồi không đến từ chạy thật nhanh ở một chỗ, mà từ việc không giữ quyền điều khiển quá lâu.

Chương 10

Máy trạng thái – Viết hành vi theo điều hệ thống đang là

Giới thiệu ngắn

Đèn giao thông không chỉ "bật LED". Nó ở trạng thái Xanh, Vàng hoặc Đỏ; mỗi trạng thái có output, thời lượng và điều kiện chuyển. Khi hành vi phụ thuộc lịch sử, một rừng if với nhiều boolean dễ tạo tổ hợp vô nghĩa. Máy trạng thái hữu hạn đặt tên cho các chế độ và chuyển đổi hợp lệ.

Mục tiêu học tập

- Xác định state, event, transition, action và guard.
- Biểu diễn trạng thái bằng enum thay nhiều boolean.
- Viết entry action và update không chặn.
- Lập bảng chuyển đổi trước khi code.
- Thiết kế trạng thái lỗi/an toàn và xử lý event ưu tiên.

10.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Đèn giao thông không chỉ chạy một danh sách lệnh; nó đang ở một trạng thái, nhận sự kiện thời gian hoặc nút bấm, rồi chuyển sang trạng thái khác. Máy trạng thái làm lộ các trạng thái, sự kiện, guard và hành động, giúp tránh rừng if phụ thuộc lịch sử ngầm.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **máy trạng thái như mô hình hành vi có luật chuyển rõ ràng**, ta luôn đi qua bốn miền: hiện

tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

10.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: tập trạng thái, sự kiện, guard, hành động vào/ra, timeout và trạng thái an toàn. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$S_{k+1} = \delta(S_k, e_k), \qquad y_k = \lambda(S_k)$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

10.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

10.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là trạng thái hiện tại, thời điểm vào trạng thái và sự kiện đang chờ. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Tại mọi thời điểm hệ thống ở đúng một trạng thái hợp lệ; mỗi cặp trạng thái–sự kiện có chuyển tiếp xác định hoặc bị từ chối rõ.

10.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Mô hình cửa tự động với CLOSED, OPENING, OPEN, CLOSING và FAULT. Ta vẽ bảng chuyển, xử lý cảm biến vật cản, timeout motor và reset về trạng thái an toàn.

10.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

10.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

10.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

10.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

10.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

10.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

10.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

10.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Dùng nhiều biến boolean tạo tổ hợp trạng thái vô nghĩa.
- Thực hiện hành động lặp lại mỗi loop thay vì lúc vào trạng thái.
- Không có chuyển tiếp cho lỗi/timeout.
- Trộn guard với hành động làm khó kiểm thử.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

10.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ state diagram đèn ba màu.
2. Lập bảng transition đầy đủ.
3. Tìm trạng thái không tới được.
4. Thiết kế timeout cho motor.
5. Viết trace từ chuỗi sự kiện.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

10.7 Từ nhiều boolean đến một enum

Ba cờ `isRed`, `isYellow`, `isGreen` tạo tám tổ hợp, chỉ ba hợp lệ. Enum chỉ cho một trạng thái.

```
enum class LightState : byte { Red, Green, Yellow, Fault };
LightState state = LightState::Red;
unsigned long enteredAt = 0;
```

10.8 Bảng trạng thái

State	Output	Event/guard	Next
Red	red on	elapsed 5 s	Green
Green	green on	elapsed 4 s	Yellow
Yellow	yellow on	elapsed 1 s	Red
Any	safe output	sensor/system fault	Fault
Fault	red blink	reset + self-test pass	Red

Bảng buộc ta trả lời event từ mọi state. “Any” có ưu tiên cao và nên kiểm trước chuyển bình thường.

10.9 Entry action

```
void enterState(LightState next, unsigned long now) {
  state = next;
  enteredAt = now;

  digitalWrite(RED_PIN, next == LightState::Red ? HIGH : LOW);
  digitalWrite(YELLOW_PIN, next == LightState::Yellow ? HIGH : LOW);
  digitalWrite(GREEN_PIN, next == LightState::Green ? HIGH : LOW);
}
```

Output được đặt một lần khi vào state, tránh rải digitalWrite. Với Fault nhấp nháy, entry đặt nền rồi update fault quản timing.

10.10 Update và chuyển trạng thái

```
void updateLight(unsigned long now) {
  if (hasFault()) {
    if (state != LightState::Fault) enterState(LightState::Fault, now);
    updateFaultBlink(now);
    return;
  }

  unsigned long elapsed = now - enteredAt;
  switch (state) {
    case LightState::Red:
      if (elapsed >= 5000) enterState(LightState::Green, now);
      break;
    case LightState::Green:
      if (elapsed >= 4000) enterState(LightState::Yellow, now);
      break;
    case LightState::Yellow:
```

```
      if (elapsed >= 1000) enterState(LightState::Red, now);
      break;
    case LightState::Fault:
      updateFaultBlink(now);
      break;
  }
}
```

Không dùng `default` im lặng khi enum mới; compiler có thể cảnh báo case thiếu. Trong hệ thống an toàn, giá trị state hỏng cần đưa output safe.

10.11 Event queue hay polling

Hệ nhỏ có thể kiểm button event và timer trong update. Khi nhiều event, queue giúp tách producer/consumer. Nhưng queue hữu hạn có thể đầy; cần chính sách ưu tiên/coalescing. Emergency event không nên đứng sau 100 log event. Một số tín hiệu an toàn cần đường phần cứng độc lập.

10.12 Máy trạng thái phân cấp

Một thiết bị có state Off/On; trong On có Idle/Running/Error. Nhồi mọi tổ hợp thành enum phẳng tăng nhanh. Có thể tách hai máy hoặc dùng state phân cấp. Với Arduino nhỏ, hai enum và quy tắc sở hữu rõ thường đủ, tránh framework nặng.

10.13 Moore và Mealy

Máy Moore output phụ thuộc state; Mealy phụ thuộc state và input hiện tại. Moore dễ lý luận; Mealy phản hồi ngay nhưng nhạy input/nhiễu. Ví dụ output LED traffic theo state là Moore; beep ngay khi nút nhấn trong state Armed là Mealy. Thực tế kết hợp cả hai, nhưng phân loại giúp tìm glitch.

10.14 Test máy trạng thái mà không chờ thật

Tách hàm update nhận `now` và input/event, trả output/next state. Test gọi ở timestamp giả. Không dùng delay trong test.

```
// Y tuong test
enterState(LightState::Red, 1000);
updateLight(5999);  // van Red
updateLight(6000);  // Green
```

Muốn test host C++, tách logic không phụ thuộc digitalWrite; adapter phần cứng áp output sau.

State là điều phải nhớ để hiểu phản ứng

Nếu cùng input nhưng hệ thống phản ứng khác vì chuyện đã xảy ra trước đó, bạn có state. Hãy đặt tên cho nó thay vì giấu trong nhiều biến rời.

10.15 Bài tập luyện tập

1. Vẽ bảng máy đèn giao thông có nút người đi bộ và thời gian tối thiểu xanh.
2. Tạo khóa số: Locked, Entering, Unlocked, Lockout; xử lý timeout và sai ba lần.
3. Tạo máy bơm: Idle, Watering, Cooldown, Fault; sensor lỗi vào Fault.
4. Thay ba boolean của một project cũ bằng enum và đếm tổ hợp vô nghĩa đã loại.
5. Tách logic state khỏi hardware để test thời gian giả.
6. Thiết kế ưu tiên event khi timeout, nút stop và sensor fault cùng xảy ra.

10.16 Tóm tắt cuối chương

Bản đồ kiến thức

Máy trạng thái gồm state, event, guard, transition và action. Enum loại tổ hợp cờ vô nghĩa. Entry action đặt output; update kiểm event/timing ngắn. Bảng chuyển đổi lộ case thiếu và ưu tiên. Logic nhận thời gian/input cho phép test không chờ thật.

Góc nhìn cá nhân

Đặt tên một trạng thái giống bật đèn trong căn phòng rối. Thay vì hỏi “tại sao ba cờ này như vậy”, ta nói “thiết bị đang Cooldown”. Một cái tên tốt gom lịch sử cần thiết thành điều có thể nhìn, thảo luận và kiểm thử.

Chương 11

Lấy mẫu, lọc và hiệu chuẩn – Từ số đo rung đến tín hiệu có thể dùng

Giới thiệu ngắn

Cảm biến không đưa cùng một số dù đại lượng gần như đứng yên. Nhiễu, lượng tử hóa, nguồn và môi trường làm số đo rung. Lọc giảm thành phần không mong muốn nhưng cũng thêm trễ và có thể xóa sự kiện thật. Hiệu chuẩn sửa quan hệ hệ thống với chuẩn; lọc không biến cảm biến sai thành đúng.

Mục tiêu học tập

- Chọn tần số lấy mẫu theo động lực tín hiệu.
- Cài moving average, exponential moving average và median.
- Hiểu đánh đổi nhiễu–trễ–bộ nhớ.
- Thực hiện hiệu chuẩn offset/gain và đánh giá residual.
- Phát hiện sensor out-of-range, stuck và missing.

11.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Đặt cảm biến đứng yên vẫn cho nhiều giá trị khác nhau vì nhiễu điện, lượng tử hóa và môi trường. Lấy trung bình làm êm nhưng tạo trễ; median chống spike; hiệu chuẩn sửa offset/gain. Phải tách lọc khỏi hiệu chuẩn vì chúng giải hai loại sai số khác nhau.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **lấy mẫu, lọc và hiệu chuẩn như quá trình suy ra đại lượng từ số đo nhiễu**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ

cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

11.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: chu kỳ mẫu, cửa sổ, trung bình, median, offset, gain, residual và miền hiệu chuẩn. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$\bar{x}_k = \frac{1}{N}\sum_{i=0}^{N-1} x_{k-i}, \qquad y = ax + b$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

11.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

11.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là bộ đệm mẫu, tổng trượt, hệ số hiệu chuẩn và thống kê chất lượng. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Không công bố độ chính xác cao hơn dữ liệu chuẩn; bộ lọc không được tạo độ trễ vượt yêu cầu điều khiển.

11.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Hiệu chuẩn cảm biến nhiệt ở hai điểm chuẩn và kiểm tra ở điểm thứ ba. Ta thu nhiều mẫu, lọc spike, ước lượng hệ số, tính residual và lưu hệ số kèm phiên bản.

11.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

11.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

11.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

11.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

11.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

11.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

11.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

11.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Dùng lọc để che sai lệch hệ thống.
- Hiệu chuẩn bằng đúng một mẫu.
- Tăng cửa sổ vô hạn để số đẹp.
- Trộn đơn vị thô và đơn vị vật lý.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

11.6 Bài tập ba tầng và hồ sơ bằng chứng

1. So sánh mean và median trên dữ liệu có spike.
2. Tính độ trễ cửa sổ.
3. Hiệu chuẩn tuyến tính hai điểm.
4. Vẽ residual theo giá trị.
5. Thiết kế metadata cho hệ số.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

11.7 Lấy mẫu là một quyết định

Đọc càng nhanh không mặc nhiên tốt. Một cảm biến nhiệt thay đổi chậm không cần 10 kHz; đọc quá nhanh tăng log/nhiễu và chiếm CPU. Tín hiệu chuyển động nhanh cần tần số đủ cao để tránh aliasing. Định lý Nyquist nói lấy mẫu trên hai lần tần số cao nhất lý tưởng, nhưng hệ thật cần lọc chống alias và dư địa.

Chọn theo: tốc độ hiện tượng, thời gian đáp ứng sensor, băng thông ADC, latency điều khiển và ngân sách CPU. Ghi timestamp thật; đừng giả định loop đều nếu chưa đo.

11.8 Trung bình trượt

```
const byte WINDOW = 8;
int samples[WINDOW] = {0};
long sum = 0;
```

```
byte index = 0;
byte count = 0;

float updateMovingAverage(int value) {
  sum -= samples[index];
  samples[index] = value;
  sum += value;
  index = (index + 1) % WINDOW;
  if (count < WINDOW) count++;
  return static_cast<float>(sum) / count;
}
```

Running sum đưa mỗi update $O(1)$. Dùng `long` để tránh tràn tổng: 1023*8 an toàn int16, nhưng window lớn hơn cần tính. Giai đoạn khởi động chia theo count chứ không các số 0 chưa thật.

Window lớn giảm nhiễu nhanh nhưng tăng trễ và làm mờ cạnh. Moving average nhạy outlier.

11.9 Exponential moving average

$$y_k = \alpha x_k + (1 - \alpha) y_{k-1}.$$

```
float filtered = 0;
bool initialized = false;
const float ALPHA = 0.15;

float updateEma(float sample) {
  if (!initialized) {
    filtered = sample;
    initialized = true;
  } else {
    filtered += ALPHA * (sample - filtered);
  }
  return filtered;
}
```

EMA dùng ít bộ nhớ. Alpha lớn phản ứng nhanh/nhiễu hơn; nhỏ mượt/trễ hơn. Ý nghĩa alpha phụ thuộc chu kỳ lấy mẫu; nếu sampling jitter lớn, dùng alpha theo thời gian hoặc giữ chu kỳ ổn định.

11.10 Median cho outlier xung

Median window 3/5 loại spike đơn lẻ tốt mà giữ cạnh hơn mean. Trên MCU nhỏ, sort mảng nhỏ đủ. Không dùng median nếu outlier kéo dài quá nửa window. Một pipeline có thể median

3 rồi EMA.

11.11 Hiệu chuẩn tuyến tính

Giả sử raw (x_1, x_2) tương ứng chuẩn (y_1, y_2). Mô hình $y = ax + b$:

$$a = \frac{y_2 - y_1}{x_2 - x_1}, \qquad b = y_1 - ax_1.$$

Đo nhiều mẫu tại mỗi điểm, dùng trung bình/trung vị và ước lượng độ phân tán. Hai điểm chỉ xác định đường; kiểm tại các điểm ở giữa để phát hiện phi tuyến.

11.12 Residual và validation

Residual = giá trị dự đoán - giá trị chuẩn. Vẽ residual theo mức: hình cong gợi ý phi tuyến, xu hướng theo nhiệt độ gợi ý drift, độ phân tán tăng gợi ý heteroscedasticity. Báo MAE/max error trên điểm không dùng fit. "R^2 cao" không bảo đảm sai số nhỏ đúng vùng quan trọng.

11.13 Phát hiện lỗi sensor

Các kiểm tra:

- Range vật lý: raw ngoài dải hợp lý.
- Rate-of-change: nhảy nhanh hơn hệ thống có thể.
- Stuck: không đổi trong thời gian dài khi kỳ vọng biến đổi.
- Timeout/missing: không có mẫu mới.
- Cross-check: hai cảm biến hoặc quan hệ vật lý không khớp.

Không tự động thay mọi lỗi bằng giá trị 0; 0 có thể là số đo thật và kích hoạt actuator. Giữ quality flag cùng giá trị: VALID, STALE, OUT_OF_RANGE, MISSING. Control quyết định state safe.

Lọc có thể che cảnh báo

Một spike có thể là nhiễu hoặc sự kiện nguy hiểm thật. Đường an toàn cần threshold/raw/rate độc lập, sensor redundancy hoặc phần cứng bảo vệ. Đừng cho một bộ lọc "mượt đẹp" quyền trì hoãn shutdown mà không phân tích.

11.14 Bài tập luyện tập

1. Thu 500 mẫu biến trở đứng yên, báo mean, median, min, max, standard deviation trên máy tính.

2. So sánh moving average window 4/16/64 khi xoay nhanh.
3. Cài median 5 không cấp phát động; test với spike.
4. Hiệu chuẩn hai điểm rồi kiểm ít nhất ba điểm trung gian.
5. Ghi residual CSV và vẽ bằng Python.
6. Thiết kế quality flag cho sensor: range, stale, stuck; xác định phản ứng output.

11.15 Tóm tắt cuối chương

Bản đồ kiến thức

Sampling phải phù hợp băng thông hiện tượng/sensor. Moving average dễ hiểu nhưng dùng window và trễ; EMA ít bộ nhớ; median chống spike. Lọc giảm nhiễu, không sửa bias. Hiệu chuẩn cần chuẩn, nhiều mẫu, kiểm điểm ngoài fit và residual. Giá trị sensor nên đi cùng chất lượng.

Góc nhìn cá nhân

Một đường tín hiệu mượt nhìn rất thuyết phục, nhưng vẻ đẹp không phải độ đúng. Tôi học cách luôn giữ hai câu hỏi tách biệt: bộ lọc đã làm số ổn định hơn chưa, và phép đo đã gần sự thật hơn chưa. Câu thứ hai chỉ có chuẩn và kiểm chứng mới trả lời được.

Chương 12

Hysteresis và PID – Từ bật/tắt ổn định đến điều khiển phản hồi

Giới thiệu ngắn

Nếu đèn bật khi raw dưới 500 và tắt khi trên 500, nhiễu quanh 500 làm nó chớp liên tục. Hysteresis dùng hai ngưỡng để tạo vùng nhớ. Khi cần giữ nhiệt độ/tốc độ gần setpoint thay vì chỉ bật/tắt, PID kết hợp sai số hiện tại, tích lũy và tốc độ thay đổi. Chương này đặt trực giác, không thay môn lý thuyết điều khiển hay phân tích an toàn.

Mục tiêu học tập

- Cài điều khiển on/off có hysteresis.
- Hiểu setpoint, process variable, error và actuator saturation.
- Hiểu vai trò P, I, D cùng chu kỳ lấy mẫu.
- Cài PID rời rạc cơ bản có anti-windup và derivative on measurement.
- Nhận biết khi PID không phải giải pháp và cần fail-safe độc lập.

12.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một quạt bật khi nhiệt độ vượt 30 độ và tắt dưới 30 độ sẽ đóng cắt liên tục quanh ngưỡng. Hysteresis tạo hai ngưỡng. Khi cần giữ nhiệt độ gần setpoint liên tục, PID kết hợp sai lệch hiện tại, lịch sử và tốc độ thay đổi, nhưng chỉ hoạt động tốt khi chu kỳ mẫu và giới hạn đầu ra được hiểu.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **hysteresis và PID như hai cấp độ của điều khiển phản hồi**, ta luôn đi qua bốn miền: hiện tượng

vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

12.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: setpoint, measurement, error, hai ngưỡng, K_p, K_i, K_d, chu kỳ mẫu, saturation và windup. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de}{dt}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

12.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

12.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là đầu ra hiện tại, tích phân sai lệch, sai lệch trước và mode điều khiển. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Đầu ra luôn trong giới hạn; tích phân không tiếp tục tăng vô ích khi actuator bão hòa; hệ thống có trạng thái an toàn.

12.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Điều khiển nhiệt độ mô phỏng. Ta bắt đầu bằng on/off có hysteresis, đo dao động, rồi thêm P, PI và PID từng bước; mỗi lần chỉ đổi một tham số và ghi response theo thời gian.

12.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

12.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

12.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

12.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

12.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

12.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

12.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

12.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Tuning cả ba hệ số cùng lúc.
- Dùng đạo hàm trên tín hiệu nhiễu.
- Bỏ qua chu kỳ mẫu trong tích phân/đạo hàm.
- Không xử lý windup và saturation.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

12.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ vòng phản hồi.
2. Truy vết hysteresis quanh ngưỡng.
3. Mô phỏng P và PI.
4. Thiết kế anti-windup.
5. Đánh giá rise time, overshoot và steady-state error.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

12.7 Hysteresis

Đèn hiện đang tắt chỉ bật khi ánh sáng dưới 450. Đang bật chỉ tắt khi trên 550. Trong 450–550, giữ trạng thái.

```
const int ON_BELOW = 450;
const int OFF_ABOVE = 550;
bool lampOn = false;

void updateLamp(int light) {
  if (!lampOn && light < ON_BELOW) {
    lampOn = true;
  } else if (lampOn && light > OFF_ABOVE) {
    lampOn = false;
  }
  digitalWrite(LAMP_PIN, lampOn ? HIGH : LOW);
```

```
12 }
```

Hai ngưỡng phải đúng thứ tự và khoảng đủ lớn hơn nhiễu nhưng không quá lớn so với yêu cầu. Hysteresis là state: cùng raw 500, output phụ thuộc trước đó.

12.8 Bài toán điều khiển

Process (đối tượng) có input điều khiển u, output đo y, setpoint r, error $e = r - y$. Controller đọc error và chọn u. Plant có quán tính, delay, nhiễu và giới hạn. Nếu sensor/actuator đặt sai chiều, phản hồi trở thành dương và chạy khỏi setpoint.

Trước PID, xác minh:

1. Tăng output có làm measurement đi đúng chiều dự kiến?
2. Output 0 và max cho dải nào? Có giới hạn nhiệt/cơ?
3. Chu kỳ sensor/control ổn định và nhanh đủ?
4. Khi sensor lỗi hoặc MCU treo, phần cứng đưa plant về an toàn?

12.9 P – phản ứng theo sai số

$$u = K_p e.$$

Kp lớn phản ứng mạnh, có thể dao động; nhỏ chậm và còn steady-state error. Saturate output 0–255. P không nhớ sai số cũ.

12.10 I – tích lũy sai số

$$I_k = I_{k-1} + K_i e_k \Delta t.$$

I loại offset bằng tích lũy, nhưng khi actuator saturated mà error còn, integral windup tăng lớn, gây overshoot lâu. Anti-windup giới hạn integral hoặc chỉ tích khi output chưa bão hòa/đang đi ra bão hòa.

12.11 D – nhìn tốc độ thay đổi

$$D = -K_d \frac{y_k - y_{k-1}}{\Delta t}.$$

Derivative on measurement tránh cú "kick" khi setpoint đổi. D nhạy nhiễu, thường cần lọc measurement và không phải hệ nào cũng cần. Không tính khi $\Delta t = 0$.

12.12 PID rời rạc cơ bản

```
struct PID {
  float kp, ki, kd;
  float integral = 0;
  float previousMeasurement = 0;
  bool initialized = false;

  float update(float setpoint, float measurement, float dt) {
    if (dt <= 0) return 0;
    float error = setpoint - measurement;
    float derivative = 0;
    if (initialized) {
      derivative = -(measurement - previousMeasurement) / dt;
    }

    float candidateI = integral + error * dt;
    float raw = kp * error + ki * candidateI + kd * derivative;
    float output = constrain(raw, 0.0, 255.0);

    // Conditional integration: khong day sau hon vao saturation
    bool allowI = (raw == output) ||
                  (raw > 255 && error < 0) ||
                  (raw < 0 && error > 0);
    if (allowI) integral = candidateI;

    previousMeasurement = measurement;
    initialized = true;
    return output;
  }
};
```

So sánh float `raw == output` ở đây hoạt động vì output là raw khi trong range qua `constrain`; vẫn có thể viết điều kiện range rõ hơn. Đây là khung học tập; thư viện PID đã xử lý nhiều chi tiết hơn.

12.13 Tuning có kiểm soát

Không tune trên hệ nguy hiểm. Với mô hình motor nhỏ/đèn giả lập:

1. Đặt Ki=Kd=0, tăng Kp từ nhỏ đến phản ứng đủ nhưng chưa dao động mạnh.
2. Thêm Ki nhỏ để giảm offset, quan sát overshoot/windup.
3. Chỉ thêm D nếu cần damping và measurement đủ sạch.
4. Ghi step response: rise time, overshoot, settling, steady error.
5. Kiểm ở nhiều tải/setpoint, không chỉ một điểm.

Thông số phụ thuộc đơn vị và sample time. Đổi từ Celsius sang raw hoặc đổi chu kỳ mà giữ gains có thể thay hành vi.

12.14 Khi không cần PID

Hysteresis đủ cho thermostat đơn giản. Feedforward + P có thể tốt khi mô hình biết. Bang-bang tối ưu cho vài hệ giới hạn. PID không sửa actuator thiếu công suất, sensor đặt sai, delay quá lớn hoặc plant không ổn định. Bắt đầu từ yêu cầu và mô hình.

Điều khiển nhiệt/cơ cần lớp bảo vệ độc lập

Không dùng project tự học PID để điều khiển heater công suất, pin, cơ cấu gây thương tích. Hệ thật cần thermal fuse/limit switch/E-stop/watchdog, thiết kế công suất, kiểm định và phân tích failure độc lập với cùng MCU/controller.

12.15 Bài tập luyện tập

1. Đo nhiễu sensor và chọn khoảng hysteresis có lý do.
2. Mô phỏng nhiệt độ bằng phương trình bậc nhất trong Python/Arduino không tải; thử on/off.
3. Cài P controller cho độ sáng LED từ LDR nhưng phân tích coupling sensor–LED.
4. Thêm anti-windup, tạo bão hòa dài và so sánh recovery.
5. Log step response CSV, vẽ setpoint/measurement/output.
6. Lập bảng failure sensor disconnected/stuck/reversed và phản ứng safe ngoài PID.

12.16 Tóm tắt cuối chương

Bản đồ kiến thức

Hysteresis dùng hai ngưỡng và state để tránh chattering. Điều khiển phản hồi so measurement với setpoint. P phản ứng hiện tại, I xóa offset nhưng windup, D damping nhưng nhạy nhiễu. PID cần dt ổn định, saturation, anti-windup và tuning bằng dữ liệu. Fail-safe không được phụ thuộc duy nhất PID/phần mềm.

Góc nhìn cá nhân

Điều khiển là lúc chương trình không chỉ quan sát mà can thiệp rồi lại quan sát hậu quả của chính mình. Vòng phản hồi ấy buộc ta khiêm tốn: một lệnh nhỏ có thể làm thế giới đổi, và phép đo tiếp theo không còn độc lập với điều ta vừa làm.

Phần IV

Những thiết bị biết nói chuyện

Chương 13

UART và giao thức Serial – Từ ký tự rời đến thông điệp đáng tin

Giới thiệu ngắn

Serial Monitor cho ta gửi vài ký tự, nhưng một hệ thống cần biết thông điệp bắt đầu/kết thúc ở đâu, trường nào là lệnh, dữ liệu hỏng xử lý ra sao và buffer đầy thì thế nào. UART truyền byte nối tiếp bằng tốc độ thống nhất. Giao thức biến chuỗi byte thành câu có cấu trúc.

Mục tiêu học tập

- Hiểu TX/RX/GND, baud và mức điện áp logic.
- Phân biệt UART TTL với RS-232/RS-485/USB.
- Viết parser dòng không chặn với buffer hữu hạn.
- Thiết kế framing, checksum, sequence và timeout ở mức cơ bản.
- Tránh blocking read, buffer overflow và nhầm binary/text.

13.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Serial Monitor hiển thị chữ, nhưng UART thực sự truyền bit theo tốc độ thống nhất mà không có clock chung. Ở tầng ứng dụng, người nhận cần biết đâu là đầu khung, độ dài, kiểu dữ liệu và cách phát hiện lỗi. In vài giá trị cách nhau bằng dấu phẩy chưa tự tạo giao thức đáng tin.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **UART và giao thức Serial như truyền khung qua dòng byte**, ta luôn đi qua bốn miền: hiện

tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

13.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: baud, bit dữ liệu, parity/stop, buffer, delimiter, length, checksum, timeout và version. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$T_{\text{byte}} \approx \frac{10}{\text{baud}}, \qquad \text{frame} = \text{header} + \text{payload} + \text{check}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

13.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

$$\text{Hiện tượng} \longrightarrow \text{cảm biến/mạch vào} \longrightarrow \text{ADC hoặc chân số} \longrightarrow \text{thuật toán} \longrightarrow \text{mạch công suất} \longrightarrow \text{tác động}.$$

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

13.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là vị trí parser trong khung, byte đã nhận và thời điểm byte cuối. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Parser luôn có thể trở lại điểm đồng bộ; khung chỉ được chấp nhận khi cấu trúc, độ dài và kiểm tra lỗi đều hợp lệ.

13.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Thiết kế giao thức gửi nhiệt độ và độ ẩm. Ta chọn frame text có newline cho giai đoạn debug, quy định số trường, timeout, CRC đơn giản và parser không dùng String nối vô hạn.

13.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

13.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

13.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

13.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

13.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

13.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

13.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

13.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đọc đến khi available bằng 0 rồi coi là hết khung.
- Không có delimiter hoặc length.
- Baud hai phía khác nhau.
- Dùng buffer không giới hạn.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

13.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Tính throughput thực.
2. Thiết kế frame text và binary.
3. Viết state machine parser.
4. Thử mất byte và byte rác.
5. Thêm version giao thức.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

13.7 Đường dây và tham chiếu

UART cơ bản có TX của A nối RX của B, RX của A nối TX của B và GND chung. Mức logic phải tương thích. UART TTL 5 V của Uno không phải cổng RS-232 điện áp dương/âm và không cắm trực tiếp; cần transceiver. USB trên Uno đi qua mạch USB–serial.

Baud là số symbol/giây gần bit/giây cho UART thông thường. Hai bên thống nhất baud, số bit dữ liệu, parity và stop bit; Arduino Serial mặc định thường 8N1. Clock sai hoặc dây nhiễu gây byte lỗi.

Chân 0/1 dùng chung hardware Serial trên Uno

Thiết bị nối RX/TX có thể cản upload hoặc tranh tín hiệu với USB–serial. Tháo khi nạp hoặc dùng cổng serial khác/SoftwareSerial có giới hạn. Không nối hai output TX với nhau.

13.8 Vì sao readStringUntil có thể làm đứng loop

Các hàm đọc chuỗi có timeout chờ delimiter. Trong lúc chờ, nút/điều khiển có thể chậm. Parser không chặn đọc mọi byte đang có, lưu đến newline, rồi xử lý.

```
const size_t RX_SIZE = 64;
char rxBuffer[RX_SIZE];
size_t rxLength = 0;

void pollSerial() {
  while (Serial.available() > 0) {
    char c = static_cast<char>(Serial.read());
    if (c == '\r') continue;
    if (c == '\n') {
      rxBuffer[rxLength] = '\0';
      handleLine(rxBuffer);
      rxLength = 0;
    } else if (rxLength < RX_SIZE - 1) {
      rxBuffer[rxLength++] = c;
    } else {
      rxLength = 0;
      Serial.println(F("ERR line_too_long"));
    }
  }
}
```

Buffer chừa một byte `'\0'`. Khi overflow, bỏ cả dòng thay vì xử lý phần đuôi như lệnh mới; tốt hơn có cờ discard-until-newline.

13.9 Giao thức lệnh dạng text

Ví dụ:

```
SET LED 128
GET SENSOR
PING 42
```

Response:

```
OK LED 128
DATA SENSOR 734
PONG 42
ERR bad_value
```

Parser tách token có giới hạn, kiểm số và range. Không dùng `atoi` mà không biết lỗi vì `atoi("abc")` trả 0 giống số 0; `strtol` cung cấp end pointer/range.

```
bool parseByte(const char* text, byte& result) {
```

```
  char* end = nullptr;
  long value = strtol(text, &end, 10);
  if (end == text || *end != '\0' || value < 0 || value > 255) {
    return false;
  }
  result = static_cast<byte>(value);
  return true;
}
```

13.10 Framing nhị phân

Text dễ debug nhưng tốn byte và parse. Binary frame có thể gồm sync, version, length, type, sequence, payload, CRC. Không chỉ tìm byte sync nếu payload có thể chứa nó; length/escaping hoặc COBS/SLIP giải framing. Parser là máy trạng thái và có timeout giữa byte để thoát frame dở.

Checksum tổng đơn giản phát hiện một số lỗi nhưng yếu; CRC có tính chất tốt hơn. CRC phát hiện lỗi truyền, không chống kẻ sửa dữ liệu; bảo mật cần xác thực/mã hóa phù hợp.

13.11 Sequence, ACK và retry

Nếu sender retry vì ACK mất, receiver có thể thực hiện lệnh hai lần. Sequence ID cho receiver nhận ra duplicate và trả lại kết quả cũ mà không lặp tác dụng. Đây là idempotency ở giao thức.

Đặt timeout dài hơn thời gian truyền + xử lý + jitter. Retry hữu hạn và backoff; trạng thái lỗi khi hết. Không để hai bên retry đồng bộ vô hạn tạo bão byte.

13.12 Flow control và buffer

UART không biết người nhận xử lý kịp. Hardware flow control RTS/CTS cần chân/hỗ trợ; giao thức có thể dùng credit/ACK. Nếu sender liên tục nhanh hơn, buffer chắc chắn tràn. Giải pháp thật là giảm tốc, xử lý nhanh, gói batch hoặc flow control, không chỉ tăng buffer đến hết RAM.

Serial TX cũng có buffer; in nhiều có thể block chờ. Log cạnh timing cần rate limit hoặc buffer riêng với chính sách drop.

13.13 Cầu nối Python–Arduino

Máy tính có thể dùng pyserial gửi dòng và kiểm response. Giao thức nên có version, newline rõ, UTF-8/ASCII quy ước. MCU thường chỉ cần ASCII cho lệnh; không giả định một `char` bằng một ký tự Unicode.

Giao thức đo cảm biến phiên bản 1

Thiết kế ba lệnh `HELLO`, `READ A0`, `STREAM A0 period`. Có giới hạn period, sequence, lệnh STOP, response lỗi và heartbeat. Viết bảng request/response trước code, fuzz dòng dài/ký tự lạ và đo loop latency khi stream.

13.14 Bài tập luyện tập

1. Hoàn thiện parser dòng có discard mode sau overflow.
2. Viết lệnh SET LED, GET LED, HELP; mọi lỗi có mã ổn định.
3. Thêm timeout cho dòng dở không newline.
4. Viết frame binary length+type+payload+checksum và parser state machine.
5. Mô phỏng mất ACK, chứng minh sequence ngăn hành động lặp.
6. Dùng Python gửi 1.000 lệnh ngẫu nhiên hợp lệ/không hợp lệ; board không treo.

13.15 Tóm tắt cuối chương

Bản đồ kiến thức

UART nối chéo TX/RX và chung GND với mức logic đúng; không nhầm TTL với RS-232. Giao thức cần framing, validation, giới hạn và lỗi rõ. Parser không chặn đọc vào buffer hữu hạn. Sequence/ACK/retry phải xử lý duplicate. Tốc độ nguồn vượt đích cần flow control, không thể chữa vô hạn bằng RAM.

Góc nhìn cá nhân

Khi hai thiết bị nói chuyện, điều khó không phải gửi một byte mà là đồng ý byte ấy có nghĩa gì, biết lúc nào câu kết thúc và xử lý khi chỉ nghe được nửa câu. Giao thức tốt là một bản cam kết về cả thành công lẫn hiểu lầm.

Chương 14

I2C và SPI – Nhiều thiết bị dùng chung một cuộc trò chuyện

Giới thiệu ngắn

UART thường là cuộc nói chuyện hai phía. I2C cho nhiều thiết bị chia sẻ hai dây theo địa chỉ; SPI dùng clock và đường dữ liệu tốc độ cao cùng chip select riêng. Thư viện giúp gọi dễ, nhưng wiring, mức logic, pull-up, chiều dài dây và xung đột bus vẫn là vật lý.

Mục tiêu học tập

- Hiểu vai trò controller/target, address, ACK của I2C.
- Lắp I2C với pull-up và mức điện áp đúng.
- Quét địa chỉ có trách nhiệm và đọc thanh ghi.
- Hiểu SCK/MOSI/MISO/CS, mode và transaction của SPI.
- Chọn giao thức theo topology, tốc độ, khoảng cách và thiết bị.

14.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Nhiều cảm biến cần giao tiếp với cùng vi điều khiển. I2C dùng địa chỉ và hai dây open-drain cần pull-up; SPI dùng clock và đường chọn riêng, nhanh hơn nhưng nhiều dây. Chọn bus dựa trên tốc độ, khoảng cách, số thiết bị, điện áp và khả năng debug.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **I2C và SPI như hai cách chia sẻ bus với ràng buộc điện và thời gian**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp

hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

14.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: địa chỉ, pull-up, điện dung bus, clock, mode SPI, chip select, điện áp logic và chiều dữ liệu. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$t_{\text{rise}} \approx 0.8473 R_p C_b, \qquad f_{\text{SPI}} \le f_{\text{device,max}}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

14.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

14.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là thiết bị đang được chọn, byte transaction và trạng thái bus. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Chỉ một thiết bị điều khiển đường dữ liệu tại thời điểm cho phép; mức logic và timing đều nằm trong datasheet của mọi phía.

14.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Nối hai cảm biến I2C cùng địa chỉ mặc định. Ta quét bus, đọc datasheet, đổi địa chỉ nếu hỗ trợ hoặc dùng multiplexer; sau đó đo rise time và kiểm tra pull-up.

14.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

14.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

14.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

14.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ

liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

14.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

14.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

14.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

14.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Cho rằng module luôn đã có pull-up phù hợp.
- Nối thiết bị khác điện áp không có chuyển mức.
- Quên nhả chip select SPI.
- Dùng dây breadboard dài ở clock cao.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

14.6 Bài tập ba tầng và hồ sơ bằng chứng

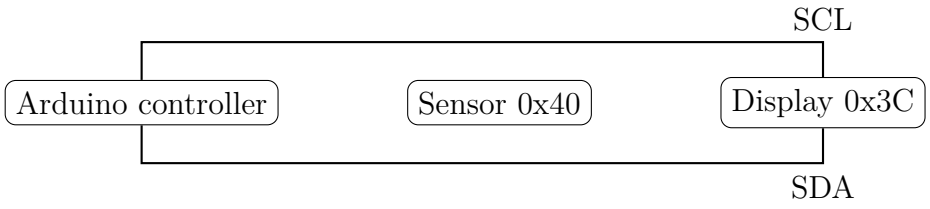
1. Vẽ transaction I2C.
2. Tính ảnh hưởng $R_p C_b$.
3. So sánh bốn mode SPI.
4. Thiết kế bus nhiều thiết bị.
5. Viết checklist khi scanner không thấy địa chỉ.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

14.7 I2C: SDA và SCL open-drain

Thiết bị kéo đường xuống LOW, điện trở pull-up đưa lên HIGH. Bus cần pull-up về điện áp logic tương thích; nhiều module đã có pull-up, mắc quá nhiều song song làm điện trở tổng quá thấp. Controller phát clock và địa chỉ, target ACK/NACK.

Trên Uno R3, SDA/SCL tương ứng A4/A5 (và header riêng nối cùng tín hiệu). Board khác khác. I2C thiết kế cho khoảng cách ngắn trên PCB/bàn mạch; dây dài/nhiễu cần giải pháp khác hoặc giảm tốc, buffer, bố trí đúng.



14.8 Quét địa chỉ

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin();
  for (byte address = 1; address < 127; address++) {
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission();
    if (error == 0) {
      Serial.print(F("Found 0x"));
      if (address < 16) Serial.print('0');
      Serial.println(address, HEX);
    }
  }
}
void loop() {}
```

Scanner cho manh mối, không chứng minh thiết bị đúng loại. Một số thiết bị không thích probe tùy ý hoặc giữ bus; đọc datasheet. Hai module cùng địa chỉ cần chân cấu hình, multiplexer hoặc bus khác.

14.9 Đọc thanh ghi

Mẫu phổ biến: ghi địa chỉ register rồi repeated start và đọc N byte. Endianness/signed/scale từ datasheet.

```
bool readRegister16(byte device, byte reg, int16_t& value) {
  Wire.beginTransmission(device);
  Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return false;

  byte count = Wire.requestFrom(device, static_cast<byte>(2));
  if (count != 2 || Wire.available() < 2) return false;
  byte highByteValue = Wire.read();
  byte lowByteValue = Wire.read();
  value = static_cast<int16_t>((highByteValue << 8) | lowByteValue);
  return true;
}
```

Không giả định byte cao trước. Shift cần kiểu phù hợp. Hàm trả quality thay vì dùng 0 cho lỗi.

14.10 Bus bị treo

Target reset giữa transaction có thể giữ SDA LOW. Watchdog reset controller không chắc giải bus. Thiết kế robust có timeout Wire nếu core hỗ trợ, recovery clock pulses theo chuẩn/datasheet, power-cycle target qua phần cứng phù hợp, và trạng thái safe. Không dùng recovery chung mà không hiểu thiết bị.

14.11 SPI

SPI thường có SCK, MOSI (controller out), MISO (controller in), CS/SS mỗi target. Full-duplex, không có địa chỉ chung; controller kéo CS của thiết bị cần nói. Mode (CPOL/CPHA), bit order và tốc độ phải đúng datasheet.

```
#include <SPI.h>

const byte CS_PIN = 10;
SPISettings settings(1000000, MSBFIRST, SPI_MODE0);

byte transferRegister(byte reg) {
  SPI.beginTransaction(settings);
  digitalWrite(CS_PIN, LOW);
  SPI.transfer(reg);
  byte value = SPI.transfer(0x00);
  digitalWrite(CS_PIN, HIGH);
  SPI.endTransaction();
  return value;
}
```

`beginTransaction` cấu hình và phối hợp thư viện. Đặt CS output HIGH trước khi chọn. MISO của target không được tri-state đúng có thể tranh bus. Level shifting cần mạch phù hợp tốc độ/chiều, không chỉ chia áp tùy tiện.

14.12 I2C hay SPI?

Tiêu chí	I2C	SPI
Dây cơ bản	2 + GND	3 bus + CS mỗi target + GND
Chọn thiết bị	địa chỉ	chân CS
Tốc độ	vừa, tùy mode/bus	thường cao hơn
Duplex	half-duplex logic	full-duplex
Pull-up	cần trên SDA/SCL	push-pull thường
Độ phức tạp thiết bị	có address/ACK	protocol chip-specific

Khoảng cách dài không phải sở trường của cả hai. UART qua RS-485/CAN hoặc giao thức công nghiệp có thể phù hợp, kèm transceiver/termination.

Module 5 V không đồng nghĩa tín hiệu 5 V-safe

Một breakout có regulator nguồn nhưng SDA/SCL vẫn kéo lên 3,3 V hoặc ngược lại. Đọc schematic module, đo rail và xem datasheet chip. Cấp đúng VCC chưa đủ bảo đảm logic tương thích.

14.13 Bài tập luyện tập

1. Quét I2C, đối chiếu địa chỉ với datasheet và chân address.
2. Tính điện trở pull-up tổng khi ba module đều có 4,7 kΩ; bàn hệ quả.
3. Viết hàm đọc register có timeout/quality flag.
4. Đọc logic analyzer I2C nếu có: start, address, R/W, ACK, stop.
5. Kết nối hai SPI target, mỗi CS riêng và settings khác; bảo đảm transaction.
6. Viết bảng lựa chọn bus cho display, SD card, sensor cách 10 m, nhiều node công nghiệp.

14.14 Tóm tắt cuối chương

Bản đồ kiến thức

I2C dùng đường open-drain có pull-up, địa chỉ và ACK; SPI dùng clock/dữ liệu/CS, mode và transaction. Cả hai cần chung GND, mức logic và wiring đúng. Scanner chỉ tìm ACK; datasheet quyết định register/scale. Bus treo và tranh đường cần timeout/recovery/fail-safe.

Góc nhìn cá nhân

Một bus chung giống bàn họp: ít dây hơn nhưng cần địa chỉ, lượt nói và cách xử lý người không trả lời. Khi hệ thống thêm thiết bị, dây có thể ít đi trong khi quy tắc giao tiếp nhiều lên. Sự gọn gàng vật lý luôn được đổi bằng kỷ luật giao thức.

Chương 15

Màn hình và giao diện nhỏ – Hiển thị trạng thái, không đổ mọi dữ liệu lên người dùng

Giới thiệu ngắn

Serial dành cho người phát triển; một thiết bị độc lập cần LED trạng thái, 7-segment, LCD hoặc OLED. Màn hình không chỉ là output đẹp. Nó có tốc độ cập nhật, bộ nhớ, giao thức và thiết kế thông tin. Giao diện tốt trả lời nhanh "thiết bị đang làm gì, số nào quan trọng, tôi có thể làm gì tiếp".

Mục tiêu học tập

- Chọn LED/7-segment/LCD/OLED theo lượng thông tin và môi trường.
- Điều khiển 7-segment với điện trở/driver phù hợp.
- Dùng LCD/OLED qua thư viện, chỉ cập nhật khi cần.
- Xây menu bằng máy trạng thái và nút đã debounce.
- Tách model trạng thái khỏi render giao diện.

15.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Màn hình có ít điểm ảnh không thể hiển thị mọi số đo cùng lúc. Giao diện phải chọn thông tin giúp người dùng quyết định, tạo phân cấp và tránh nhấp nháy do redraw toàn bộ. Đồng thời thư viện màn hình có thể chiếm đáng kể RAM và thời gian bus.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **màn hình**

nhỏ như bài toán thiết kế thông tin và ngân sách cập nhật, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

15.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: kích thước pixel, font, buffer, tốc độ bus, chu kỳ refresh, trạng thái UI và mức ưu tiên thông tin. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$\text{frame time} = T_{\text{render}} + T_{\text{transfer}}, \qquad \text{information density} = \frac{\text{useful cues}}{\text{area}}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

15.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

15.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là màn hình hiện tại, vùng bẩn cần cập nhật và dữ liệu đã trình bày. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Thông tin quan trọng luôn đọc được; refresh không chặn tác vụ thời gian và giá trị hiển thị có timestamp/trạng thái rõ.

15.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Thiết kế màn hình trạm môi trường gồm giá trị chính, xu hướng, trạng thái cảm biến và cảnh báo. Ta phác trên giấy, đo độ dài chuỗi, chỉ redraw vùng đổi và kiểm tra ngoài trời.

15.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

15.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

15.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

15.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

15.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

15.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

15.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

15.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đổ mọi dữ liệu lên một màn hình.
- Refresh mỗi loop gây nhấp nháy.
- Dùng buffer vượt SRAM.
- Hiển thị số không đơn vị hoặc không trạng thái lỗi.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

15.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ wireframe 128x64.
2. Tính ngân sách framebuffer.
3. Thiết kế dirty-region.
4. Phân cấp cảnh báo.
5. Đo thời gian refresh.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

15.7 Bậc thang hiển thị

Một LED có thể nói power/error bằng màu/nhịp. 7-segment hiển thị số nhìn xa. LCD 16x2 hiển thị chữ đơn giản, ít RAM. OLED đồ họa linh hoạt nhưng framebuffer có thể chiếm đáng kể SRAM trên Uno. Chọn thứ nhỏ nhất đủ thông tin; màn hình lớn không chữa hierarchy kém.

15.8 7-segment

Bảy LED a–g và dấu chấm; common cathode/anode quyết định logic. Mỗi segment cần hạn dòng, và khi multiplex nhiều digit cần transistor/driver vì dòng tổng. Không lái số nhiều digit vượt giới hạn chân.

```
1 const byte segmentPins[7] = {2, 3, 4, 5, 6, 7, 8};
2 const byte digits[10] = {
```

```
  0b1111110, 0b0110000, 0b1101101, 0b1111001, 0b0110011,
  0b1011011, 0b1011111, 0b1110000, 0b1111111, 0b1111011
};

void showDigit(byte digit) {
  if (digit > 9) return;
  for (byte i = 0; i < 7; i++) {
    bool on = bitRead(digits[digit], 6 - i);
    digitalWrite(segmentPins[i], on ? HIGH : LOW);
  }
}
```

Mapping pin/bit phải khớp loại display; test từng segment trước số. Dùng driver như MAX7219/TM1637 với thư viện đúng module cho nhiều digit.

15.9 LCD ký tự

LCD HD44780 có thể nối song song nhiều dây hoặc backpack I2C. Địa chỉ backpack khác nhau. Thư viện LiquidCrystal chính thức minh họa parallel; thư viện I2C bên thứ ba có API/pin mapping không thống nhất.

```
#include <LiquidCrystal.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  lcd.begin(16, 2);
  lcd.print("Arduino san sang");
}
```

Không gọi `lcd.clear()` mỗi loop; clear chậm và gây nhấp. Giữ nội dung cũ, chỉ render khi model thay đổi hoặc ghi đè đủ khoảng trắng.

15.10 OLED và bộ nhớ

OLED 128x64 monochrome framebuffer đầy đủ cần 1024 byte, khoảng nửa 2 KB SRAM của Uno, chưa kể thư viện/biến. Một số thư viện page buffer giảm RAM đổi lượt vẽ. Kiểm compile report và free SRAM; font/bitmap nên ở flash.

Không cập nhật OLED I2C 100 Hz khi dữ liệu người nhìn chỉ cần 5 Hz. Bus bị chiếm, control jitter và không thêm giá trị.

15.11 Model–view trong MCU

```
struct UiModel {
  float temperature;
  bool sensorOk;
  byte menuIndex;
  bool editing;
};

void render(const UiModel& model) {
  // Chi doc model, khong tu doc sensor hay doi actuator.
}
```

Sensor/control cập nhật model; render đọc snapshot. Tách giúp test format trên host và thay LCD bằng OLED.

15.12 Menu là máy trạng thái

State: Home, Menu, EditValue, Confirm. Event: Up, Down, Select, Back, Timeout. Mỗi state định nghĩa event hợp lệ và view. Không viết vòng while chờ nút trong menu; update mỗi loop để control vẫn chạy.

Khi edit, giữ `draftValue`; chỉ commit khi Select, Back hủy. Hiển thị dấu hiệu đang sửa. Giá trị quan trọng cần range và bước.

15.13 Thông tin lỗi

Không chỉ hiện "ERROR". Hiện mã ngắn và hành động: `SENSOR E2 / CHECK CABLE`. Giữ lỗi đến khi người dùng acknowledge nếu cần; không thay bằng số cuối như vẫn hợp lệ. LED/buzzer dự phòng nếu display chính lỗi.

Tốc độ máy không phải tốc độ người

Control có thể chạy 100 Hz; display 5–10 Hz; log 1 Hz. Tách nhịp theo người nhận thông tin giảm bus, nhấp và tải CPU.

15.14 Bài tập luyện tập

1. Test từng segment bằng walking-one trước khi hiển thị số.
2. Hiển thị counter nút nhấn trên 7-segment, không block multiplex.
3. LCD hiển thị sensor và quality flag, chỉ update 4 Hz.
4. Đo SRAM trước/sau thư viện OLED và bitmap.
5. Vẽ bảng state/event cho menu set threshold, có cancel/timeout.

6. Thiết kế ba mức cảnh báo bằng text, màu/nhịp LED mà vẫn phân biệt với người mù màu.

15.15 Tóm tắt cuối chương

Bản đồ kiến thức

Chọn hiển thị nhỏ nhất đủ nhiệm vụ. 7-segment cần điện trở/driver; LCD tránh clear liên tục; OLED cần ngân sách framebuffer. Tách model khỏi render và tách nhịp display khỏi control. Menu là máy trạng thái không chặn; lỗi cần mã, ý nghĩa và hành động.

Góc nhìn cá nhân

Màn hình dễ làm project trông "hoàn thiện", nhưng giao diện thật sự hoàn thiện khi người dùng không cần nhìn lâu. Một con số có đơn vị, một biểu tượng chất lượng và một hành động rõ thường giá trị hơn tám dòng dữ liệu kỹ thuật đang chạy thật nhanh.

Chương 16

EEPROM và dữ liệu bền – Nhớ qua lần mất nguồn mà không ghi đến mòn

Giới thiệu ngắn

Biến mất khi tắt nguồn; EEPROM giữ dữ liệu. Nhưng EEPROM có dung lượng nhỏ và số chu kỳ ghi hữu hạn. Ghi mỗi loop sẽ làm mòn nhanh. Dữ liệu cũng cần magic, version, checksum và cơ chế chịu mất nguồn giữa bản ghi. Bộ nhớ bền là một định dạng lâu dài, không chỉ địa chỉ byte.

Mục tiêu học tập

- Hiểu EEPROM khác SRAM/flash và giới hạn endurance.
- Dùng `EEPROM.get/put/update` cho cấu hình nhỏ.
- Thiết kế record có magic, version, CRC và default.
- Giảm ghi bằng dirty flag, debounce cấu hình và wear leveling.
- Xử lý power loss bằng hai slot/sequence ở mức cơ bản.

16.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Hệ số hiệu chuẩn cần tồn tại sau mất nguồn nhưng EEPROM không chịu được ghi liên tục mỗi vòng. Mất nguồn giữa bản ghi có thể tạo dữ liệu nửa chừng. Ta cần schema nhỏ, version, checksum và chính sách chỉ ghi khi thay đổi có ý nghĩa.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **EEPROM như bộ nhớ bền có giới hạn chu kỳ ghi và nhu cầu toàn vẹn**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu

chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

16.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: địa chỉ, kích thước, chu kỳ ghi, version, checksum, giá trị mặc định và wear leveling. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$L = \frac{N_{\text{endurance}}}{f_{\text{write}}}, \qquad \text{record} = \text{version} + \text{data} + \text{CRC}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

16.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

16.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là slot đang dùng, sequence number và bản ghi hợp lệ cuối. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Khởi động luôn tìm được bản ghi hợp lệ hoặc giá trị mặc định; không ghi nếu byte không đổi và không vượt ngân sách tuổi thọ.

16.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Lưu hệ số hiệu chuẩn theo hai slot. Ta ghi payload trước, marker hợp lệ sau, kiểm checksum khi boot và chọn sequence mới nhất; mô phỏng mất nguồn ở từng bước.

16.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

16.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

16.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

16.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

16.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

16.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

16.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

16.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Ghi mỗi loop.
- Đọc struct mà không có version.
- Không kiểm byte chưa khởi tạo.
- Coi EEPROM là cơ sở dữ liệu không giới hạn.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

16.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Tính tuổi thọ theo tần suất ghi.
2. Thiết kế record có CRC.
3. Mô phỏng power loss.
4. Thêm version migration.
5. So sánh update với write.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

16.7 Ba loại bộ nhớ

Flash giữ firmware và literal; SRAM giữ biến lúc chạy; EEPROM giữ dữ liệu qua mất nguồn. Dung lượng và endurance tùy chip/board. Uno R3 ATmega328P có EEPROM nhỏ ở mức kilobyte, phù hợp cấu hình/counter thưa, không phải log liên tục.

16.8 Đọc/ghi một cấu hình

```
#include <EEPROM.h>

struct Config {
  uint16_t magic;
  uint8_t version;
  int16_t threshold;
  uint8_t brightness;
```

```
  uint16_t crc;
};

Config config;
EEPROM.get(0, config);
```

Ghi raw struct phụ thuộc padding, endian và thay đổi compiler/architecture. Trong cùng firmware/board với version quản lý có thể chấp nhận; định dạng di động nên serialize từng trường rõ.

16.9 Default và validation

EEPROM mới không chứa object hợp lệ. Magic xác suất nhỏ nhận nhầm, version chọn parser, CRC phát hiện bit lỗi/ghi dở; sau đó vẫn kiểm range.

```
bool validConfig(const Config& c) {
  if (c.magic != 0xB17E || c.version != 1) return false;
  if (c.threshold < 0 || c.threshold > 1023) return false;
  if (c.crc != calculateCrc(c)) return false;
  return true;
}

void loadConfig() {
  EEPROM.get(0, config);
  if (!validConfig(config)) {
    config = defaultConfig();
    saveConfig();
  }
}
```

CRC calculation phải bỏ trường crc hoặc đặt 0. CRC không bảo mật.

16.10 update và put

`EEPROM.update(address, value)` chỉ ghi byte khi khác, giảm wear. `put` trên AVR core dùng update semantics cho object theo tài liệu triển khai phổ biến, nhưng kiểm core. Dù vậy, thay đổi counter mỗi giây vẫn ghi liên tục; cần thiết kế cấp cao.

16.11 Dirty flag và trì hoãn commit

Khi người dùng chỉnh menu, chỉ sửa RAM và đặt dirty. Nếu không đổi thêm trong 2 giây, hoặc nhấn Save, ghi một lần.

```
bool configDirty = false;
unsigned long configChangedAt = 0;
```

```
3
4  void markConfigChanged(unsigned long now) {
5    configDirty = true;
6    configChangedAt = now;
7  }
8
9  void updatePersistence(unsigned long now) {
10   if (configDirty && now - configChangedAt >= 2000) {
11     saveConfig();
12     configDirty = false;
13   }
14 }
```

Mất nguồn trong 2 giây làm mất chỉnh mới; đó là trade-off. Có thể dùng nút save, capacitor/power-fail signal hoặc loại memory khác cho yêu cầu cao.

16.12 Hai slot chịu ghi dở

Giữ hai record A/B, mỗi record magic/version/sequence/payload/CRC. Khi lưu, ghi vào slot không hiện hành với sequence+1, ghi CRC/valid marker cuối. Khởi động đọc cả hai, chọn record hợp lệ sequence mới nhất. Nếu mất điện giữa ghi, slot cũ vẫn còn.

Sequence unsigned rollover cần so modulo hoặc dùng phạm vi đủ và reset có kiểm soát. Ghi marker cuối không bảo đảm atomic từng byte cho mọi memory; đọc datasheet.

16.13 Wear leveling

Counter cập nhật nhiều có thể quay vòng qua nhiều slot, mỗi record có sequence/CRC. Boot scan tìm mới nhất. Số lần ghi mỗi cell giảm theo số slot. Nhưng boot scan và mất nguồn phức tạp; dùng FRAM hoặc external storage nếu workload thực sự ghi nhiều.

16.14 Factory reset và migration

Không xóa từng byte vô cớ; chỉ làm invalid magic hoặc ghi default. Factory reset cần thao tác khó kích hoạt nhầm (giữ nút lúc boot 5 s + xác nhận) và không cắt nguồn giữa. Khi version tăng, đọc v1, validate, chuyển sang v2 trong RAM rồi ghi record mới; giữ default nếu không thể migrate.

Đừng lưu con trỏ hoặc object phức tạp

Địa chỉ RAM vô nghĩa sau reset. `String`, virtual table và object chứa pointer không thể put/get như dữ liệu bền. Serialize giá trị đơn giản với độ dài giới hạn.

16.15 Bài tập luyện tập

1. Lưu threshold/brightness, validate range và factory default.
2. Đếm số lần save thật, chứng minh dirty debounce giảm ghi.
3. Thêm magic/version/CRC16 và cố tình sửa byte để phát hiện.
4. Cài hai slot, mô phỏng mất nguồn bằng cách bỏ dở từng bước ghi.
5. Thiết kế wear-level counter 16 slot và ước lượng lifetime theo tần suất.
6. Viết migration v1 sang v2 thêm trường hysteresis, giữ dữ liệu cũ.

16.16 Tóm tắt cuối chương

Bản đồ kiến thức

EEPROM giữ cấu hình qua mất nguồn nhưng nhỏ và có endurance hữu hạn. Dữ liệu cần magic, version, checksum và range validation. Trì hoãn save/dirty flag giảm write. Hai slot giữ bản cũ khi ghi dở; wear leveling phân tán mòn. Không lưu pointer/String raw; migration là phần của định dạng.

Góc nhìn cá nhân

Khi một thiết bị biết nhớ, mỗi byte trở thành lời hứa với lần khởi động sau. Nếu chỉ nhớ giá trị mà không nhớ phiên bản, tính hợp lệ và cách khôi phục, ký ức ấy có thể nguy hiểm hơn quên. Bộ nhớ bền cần cả cách tin và cách không tin dữ liệu của chính mình.

Phần V

Viết hệ thống nhúng có thể tin cậy

Chương 17

Interrupt – Khi một sự kiện không muốn chờ vòng loop

Giới thiệu ngắn

Polling hỏi đi hỏi lại; interrupt báo CPU tạm dừng luồng chính để chạy một hàm ngắn. Nó hữu ích cho xung nhanh, timer và sự kiện cần latency nhỏ, nhưng không làm mọi input "tốt hơn". ISR chạy trong bối cảnh hạn chế; chia sẻ dữ liệu sai tạo race condition khó thấy.

Mục tiêu học tập

- Hiểu interrupt, ISR, latency và ưu tiên ở mức khái niệm.
- Dùng `attachInterrupt` với chân hỗ trợ.
- Viết ISR ngắn, không delay/Serial và chỉ ghi dữ liệu tối thiểu.
- Chia sẻ biến bằng `volatile` cùng vùng nguyên tử phù hợp.
- Chọn polling, hardware counter hoặc interrupt theo yêu cầu.

17.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một cạnh xung ngắn có thể biến mất trước lần poll tiếp theo. Interrupt cho phép phần cứng tạm ngắt luồng chính để ghi nhận sự kiện. ISR phải cực ngắn, không chặn và chia sẻ dữ liệu cẩn thận vì nó có thể chạy giữa hai lệnh của loop.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **interrupt như cơ chế phản ứng sự kiện với hợp đồng thời gian nghiêm ngặt**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ

cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

17.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: nguồn ngắt, cạnh/kích mức, latency, thời gian ISR, biến shared, atomicity và tần suất sự kiện. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$T_{\text{response}} = T_{\text{latency}} + T_{\text{ISR}}, \qquad T_{\text{ISR}} \ll T_{\text{event period}}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

17.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

17.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là cờ sự kiện, bộ đếm volatile và dữ liệu snapshot giữa ISR/loop. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

ISR không chặn và không làm mất sự kiện trong miền thiết kế; truy cập dữ liệu nhiều byte phải nhất quán.

17.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Đếm xung encoder. ISR chỉ tăng bộ đếm và ghi timestamp tối thiểu; loop sao chép atomic, tính tốc độ theo cửa sổ và xử lý debounce/nhiễu bên ngoài ISR.

17.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

17.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

17.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

17.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

17.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

17.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

17.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

17.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Gọi delay hoặc Serial trong ISR.
- Làm quá nhiều tính toán trong ISR.
- Chỉ thêm volatile nhưng bỏ qua atomicity.
- Tần suất ngắt vượt khả năng xử lý.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

17.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ timeline interrupt.
2. Đo thời gian ISR.
3. Thiết kế handoff bằng cờ.
4. Thử biến nhiều byte.
5. So sánh polling và interrupt.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

17.7 Một ISR tối thiểu

```
volatile bool pulseSeen = false;

void onPulse() {
  pulseSeen = true;
}

void setup() {
  pinMode(2, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(2), onPulse, FALLING);
}

void loop() {
  if (pulseSeen) {
    noInterrupts();
```

```
    pulseSeen = false;
    interrupts();
    handlePulse();
  }
}
```

`digitalPinToInterrupt` tránh giả định số interrupt. Chân hỗ trợ khác board. ISR chỉ đặt cờ; xử lý ở loop.

17.8 volatile làm gì và không làm gì

`volatile` bảo compiler mỗi lần đọc/ghi thật vào memory vì giá trị có thể đổi ngoài luồng. Nó không làm thao tác nhiều byte atomic, không tạo hàng đợi và không giải mọi race.

Trên MCU 8-bit, đọc `volatile uint32_t` có thể bị interrupt chen giữa các byte. Copy trong critical section:

```
volatile uint32_t pulseCount = 0;

void onPulse() {
  pulseCount++;
}

uint32_t snapshotPulseCount() {
  noInterrupts();
  uint32_t copy = pulseCount;
  interrupts();
  return copy;
}
```

Nếu trước đó interrupt đã bị tắt bởi nơi khác, gọi `interrupts()` vô điều kiện có thể sai; thư viện atomic block hoặc lưu trạng thái thanh ghi phù hợp tốt hơn cho code chung. Critical section phải rất ngắn.

17.9 Không làm gì trong ISR

Tránh `delay`, Serial, cấp phát động, I2C và xử lý dài. Một số hàm dựa interrupt khác sẽ không tiến triển khi ISR chạy. `millis` thường không cập nhật trong ISR. ISR dài làm mất interrupt khác, tăng jitter và phá giao tiếp.

Nếu event nhanh hơn loop xử lý và chỉ dùng cờ boolean, nhiều event bị gộp. Dùng counter, ring buffer hoặc peripheral hardware tùy cần. Counter cũng có thể overflow; tính rate bằng delta unsigned.

17.10 Mode RISING/FALLING/CHANGE/LOW

Chọn cạnh theo tín hiệu và pull. `CHANGE` gấp đôi event và cần đọc state. Level-triggered LOW có thể tái kích hoạt khi còn giữ, nguy cơ loop interrupt; dùng đúng mục đích. Nút nhấn có bounce nên interrupt không tự khử dội. Polling debounce thường đủ và dễ hơn.

17.11 Đo tần số xung

ISR tăng counter. Mỗi 1000 ms loop snapshot delta, tính Hz dựa elapsed thật.

```
volatile uint32_t edges = 0;
uint32_t previousEdges = 0;
unsigned long lastReport = 0;

void onEdge() { edges++; }

void updateFrequency(unsigned long now) {
  if (now - lastReport < 1000) return;
  unsigned long elapsed = now - lastReport;
  lastReport = now;

  noInterrupts();
  uint32_t current = edges;
  interrupts();

  uint32_t delta = current - previousEdges;
  previousEdges = current;
  float hz = delta * 1000.0 / elapsed;
  Serial.println(hz);
}
```

Tần số cao vượt khả năng ISR; dùng hardware timer/counter input capture. Xung nhiễu cần conditioning, Schmitt trigger/filter và layout.

17.12 Ring buffer một producer–một consumer

ISR ghi timestamp vào buffer cố định, loop đọc. Head/tail và atomic cần thiết kế đúng kiến trúc; khi đầy, drop newest/oldest và tăng overflow counter. Không bao giờ ghi vượt. Đây là cấu trúc nâng cao; dùng thư viện/peripheral đã kiểm chứng khi dữ liệu quan trọng.

17.13 Latency và jitter

Interrupt latency gồm hoàn thành lệnh hiện tại, ưu tiên, ISR khác/critical section và overhead lưu context. Không hứa real-time cứng chỉ vì có interrupt. Đo bằng toggle pin ở đầu/cuối

ISR trên oscilloscope/logic analyzer, nhưng toggle cũng thêm thời gian.

Interrupt không thay phần cứng an toàn

E-stop, overcurrent hoặc overtemperature nguy hiểm không nên chỉ dựa ISR và cùng MCU. Dùng mạch ngắt năng lượng/driver protection độc lập phù hợp tiêu chuẩn, rồi phần mềm ghi nhận trạng thái.

17.14 Bài tập luyện tập

1. So sánh polling và interrupt cho nút; nêu vì sao polling đủ.
2. Đếm xung chậm bằng counter, snapshot atomic và báo overflow.
3. Tạo ISR cố ý dài, đo ảnh hưởng Serial/timer rồi khôi phục code an toàn.
4. Thiết kế ring buffer 16 phần tử có overflow counter.
5. Dùng logic analyzer đo pulse width của ISR toggle pin.
6. Chọn giải pháp cho encoder nhanh: CHANGE interrupt, pin-change, hardware counter; dựa datasheet/rate.

17.15 Tóm tắt cuối chương

Bản đồ kiến thức

Interrupt giảm latency cho sự kiện phù hợp; ISR phải ngắn và dời xử lý sang loop. Volatile ngăn tối ưu sai nhưng không tạo atomic. Dữ liệu nhiều byte cần snapshot critical section; boolean gộp event, counter/buffer giữ nhiều hơn. Tần số cao nên dùng peripheral. Interrupt phần mềm không phải fail-safe.

Góc nhìn cá nhân

Interrupt dễ tạo cảm giác quyền lực vì sự kiện có thể chen ngang. Nhưng quyền chen ngang càng lớn, quy tắc càng chặt: nói thật ngắn, không chờ, chỉ để lại một dấu hiệu rồi rời đi. Đó là bài học về ưu tiên mà mọi hệ thống đồng thời đều phải trả.

Chương 18

Tổ chức code nhúng – Tách driver, logic và ứng dụng

Giới thiệu ngắn

Sketch đầu có thể nằm một file. Dự án có sensor, menu, motor và lưu cấu hình sẽ biến loop thành nơi mọi thứ biết nhau. Tách code không phải tạo thật nhiều tab; ta chia theo trách nhiệm và hướng phụ thuộc: driver biết phần cứng, logic biết quy tắc, application lắp chúng lại.

Mục tiêu học tập

- Tách hardware abstraction, driver, service/state machine và app.
- Dùng header/source, include guard và interface nhỏ.
- Tránh global mutable, cấp phát động và constructor phần cứng sớm.
- Tiêm clock/I/O để test logic trên máy tính.
- Quản lý cấu hình chân, đơn vị và lỗi có cấu trúc.

18.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một prototype có thể để mọi thứ trong loop, nhưng khi thêm cảm biến và chế độ, phần đọc chân, quy tắc nghiệp vụ và giao diện dính vào nhau. Tách driver, service và application cho phép kiểm thử logic không cần phần cứng và đổi thiết bị mà không viết lại toàn bộ.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **kiến trúc code nhúng như tách driver, logic và ứng dụng**, ta luôn đi qua bốn miền: hiện tượng

vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

18.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: module, interface, pin mapping, state, phụ thuộc phần cứng, cấu hình và điểm khởi tạo. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$\text{application} \rightarrow \text{interface} \leftarrow \text{driver}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

18.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

$$\text{Hiện tượng} \longrightarrow \text{cảm biến/mạch vào} \longrightarrow \text{ADC hoặc chân số} \longrightarrow \text{thuật toán} \longrightarrow \text{mạch công suất} \longrightarrow \text{tác động}.$$

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

18.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là trạng thái từng module và dependency đã được ghép. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Logic miền không phụ thuộc trực tiếp API chân; mỗi module có ownership rõ và không có hai nơi cùng điều khiển một tài nguyên.

18.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Tách bộ tưới cây thành sensor driver, controller, pump driver và app scheduler. Controller nhận số đo qua interface nên được test trên máy với dữ liệu mô phỏng.

18.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

18.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

18.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

18.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

18.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

18.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

18.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

18.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Tách file nhưng vẫn dùng biến global khắp nơi.
- Driver chứa quyết định nghiệp vụ.
- Hai module cùng ghi một chân.
- Ẩn dependency bằng singleton.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

18.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ dependency graph.
2. Định nghĩa interface sensor.
3. Mock driver trong test.
4. Thiết kế config pin một chỗ.
5. Refactor loop lớn thành task nhỏ.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

18.7 Một kiến trúc vừa đủ

```
src/
  app.ino
  board_config.h
  button.h / button.cpp
  light_sensor.h / light_sensor.cpp
  controller.h / controller.cpp
  ui.h / ui.cpp
test/
  controller_test.cpp
```

`app.ino` setup và loop, không chứa thuật toán dài. `board_config` ánh xạ chân. Driver chuyển API phần cứng thành giá trị/quality. Controller nhận input thuần, cập nhật state/output mong muốn. UI render model.

18.8 Header và source

```
// button.h
#pragma once
#include <Arduino.h>

class Button {
 public:
  explicit Button(uint8_t pin, unsigned long debounceMs = 30);
  void begin();
  void update(unsigned long now);
  bool pressed() const;
  bool fell();

 private:
  uint8_t pin_;
  unsigned long debounceMs_;
  bool raw_ = false;
  bool stable_ = false;
  bool fell_ = false;
  unsigned long changedAt_ = 0;
};
```

Header công bố interface, source triển khai. `const` sau phương thức nói không thay state quan sát. Dấu gạch dưới là quy ước member. `explicit` ngăn chuyển đổi ngầm từ số pin thành Button.

18.9 Không gọi hardware trong constructor toàn cục

Object global được dựng trước runtime Arduino khởi tạo đầy đủ; gọi pinMode/Serial trong constructor có thể sai. Constructor chỉ lưu cấu hình; `begin()` khởi tạo trong setup.

```
Button startButton(2);

void setup() {
  startButton.begin();
}
```

18.10 Đơn vị trong tên/kiểu

`timeout` là ms hay us? `temperature` C hay raw? Đặt `timeoutMs`, `temperatureC`, `adcRaw`. Với dự án lớn, wrapper type/strong units ngăn trộn; trên Arduino nhỏ, tên và comment hợp đồng đã giúp nhiều.

18.11 Logic thuần

```
struct Inputs {
  bool buttonPressed;
  bool sensorValid;
  float lightPercent;
};

struct Outputs {
  bool lampOn;
  bool faultLed;
};

Outputs updateController(ControllerState& state,
                         const Inputs& input,
                         unsigned long nowMs);
```

Hàm không gọi digitalRead/Write. App đọc driver thành Inputs, gọi logic, áp Outputs. Test host truyền giá trị giả và thời gian giả.

18.12 Global và single source of truth

Không thể tránh mọi global trên MCU, nhưng giới hạn phạm vi file và sở hữu. Một state chỉ có một nguồn sự thật. Nếu UI, controller và driver mỗi nơi giữ bản `lampOn`, chúng có thể lệch. Controller sở hữu state; UI nhận bản đọc; driver áp lệnh.

18.13 Bộ nhớ và cấp phát

MCU nhỏ có SRAM hạn. Tránh tạo/xóa object động liên tục, String nối dài, recursion sâu và container không giới hạn. Dùng array/ring buffer capacity cố định, kiểm compile memory report, đặt upper bound. Không tối ưu mọi byte trước khi đo, nhưng thiết kế phải biết giới hạn.

18.14 Mã lỗi và quality

Không trả `-1` cho mọi lỗi khi -1 có thể là giá trị. Dùng enum và struct:

```
enum class SensorStatus : uint8_t {
  Ok, Timeout, OutOfRange, BusError
};

struct Reading {
  float value;
  SensorStatus status;
```

```
  unsigned long timestampMs;
};
```

Controller kiểm status/age trước dùng. UI hiển thị status. Log có mã ổn định.

18.15 Cấu hình build và thư viện

Khóa phiên bản library/core trong tài liệu hoặc PlatformIO config nếu cần tái lập. Đọc license. Không copy library vào project mà mất nguồn/cập nhật. Khi nâng version, chạy hardware-in-loop test. Board selection là phần cấu hình phát hành.

Hướng phụ thuộc

Application được phép biết driver và controller để lắp. Controller không biết Arduino pin hay display. Driver không quyết định quy tắc nghiệp vụ. Khi mũi tên đi một chiều, logic có thể sống ngoài board và được test nhanh.

18.16 Bài tập luyện tập

1. Tách project đèn LDR thành driver, controller, app.
2. Viết Button header/source và hai instance độc lập.
3. Tìm mọi global mutable, ghi owner và ai đọc/ghi.
4. Đổi tên biến để mang đơn vị; thêm static assertion khi phù hợp.
5. Tạo FakeClock/FakeSensor và test controller trên host.
6. Lập budget SRAM: buffer, framebuffer, object; đặt capacity có lý do.

18.17 Tóm tắt cuối chương

Bản đồ kiến thức

Tổ chức code theo trách nhiệm và hướng phụ thuộc, không số lượng file. Driver che phần cứng; controller giữ logic; app lắp ghép; UI render. Constructor global không gọi hardware, begin ở setup. Đơn vị/tình trạng cần nằm trong dữ liệu. Logic thuần và dependency injection giúp test ngoài board.

Góc nhìn cá nhân

Một sketch lớn thường rối không phải vì thiếu comment mà vì không ai biết biến nào thuộc về ai. Khi mỗi module có một lời hứa nhỏ và mũi tên phụ thuộc rõ, loop trở lại đúng vai trò: người điều phối, không phải căn phòng nhét mọi đồ.

Chương 19

Gỡ lỗi và kiểm thử hệ thống nhúng – Hai thế giới, một chuỗi bằng chứng

Giới thiệu ngắn

Trong Arduino, lỗi có thể nằm ở code, mạch, nguồn, timing, cơ khí hoặc tương tác. Serial nói phần mềm nghĩ gì; multimeter/oscilloscope nói điện đang làm gì. Quy trình gỡ lỗi phải cắt hệ thống thành khối, dùng tín hiệu đã biết và giữ điều kiện thử ổn định.

Mục tiêu học tập

- Phân loại lỗi nguồn, kết nối, logic, timing và giao tiếp.
- Dùng Serial, LED, multimeter, logic analyzer theo câu hỏi.
- Xây test jig, self-test và fault injection.
- Test logic trên host và contract driver trên hardware.
- Viết báo cáo tái hiện có sơ đồ/firmware/điều kiện.

19.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Khi motor không quay, nguyên nhân có thể là nguồn, dây, driver, tín hiệu PWM, trạng thái chương trình hoặc tải cơ khí. Debug bằng cách sửa code ngẫu nhiên làm số biến tăng. Ta cần cây chẩn đoán đi từ nguồn đến tín hiệu rồi đến logic, mỗi bước có phép đo pass/fail.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **gỡ lỗi hệ nhúng như nối bằng chứng phần mềm với phép đo phần cứng**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu

chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

19.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: điện áp, dòng, continuity, tín hiệu logic, timestamp, state, log và giả thuyết lỗi. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$P(H_i \mid E) \propto P(E \mid H_i)P(H_i)$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

19.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

19.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là giả thuyết còn lại, phép đo đã làm và cấu hình có thể tái tạo. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k, x_k, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Mỗi phép thử chỉ thay một biến; kết luận phải dựa trên số đo hoặc trace, không dựa vào việc linh kiện nhìn có vẻ hoạt động.

19.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Điều tra board reset khi servo chạy. Ta tái tạo, log lý do reset nếu có, đo rail 5 V bằng oscilloscope/đồng hồ phù hợp, cấp servo từ nguồn riêng và chứng minh nguyên nhân là sụt áp.

19.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

19.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

19.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

19.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ

liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

19.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

19.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

19.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

19.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Sửa nhiều thứ giữa hai lần thử.
- Tin LED nguồn chứng minh rail ổn định.
- Log quá nhiều làm timing đổi.
- Không lưu phiên bản sketch và sơ đồ dây.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

19.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Lập cây chẩn đoán LED không sáng.
2. Viết phiếu tái tạo lỗi.
3. Thiết kế test jig cho nút/LED.
4. Phân biệt unit, integration và system test.
5. Thu bằng chứng cho lỗi ngắt quãng.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

19.7 Bắt đầu bằng triệu chứng đo được

"Không chạy" đổi thành "sau reset, LED power sáng, không có dòng boot ở 115200, D8 đo 0 V". Ghi board/core/library version, nguồn, sơ đồ, code commit, bước tái hiện và tần suất. Chụp wiring rõ node.

19.8 Chia đôi chuỗi tín hiệu

Sensor → ADC → filter → controller → PWM → driver → motor. Tiêm giá trị giả ở giữa: thay sensor bằng biến trở; in raw; bypass filter; ép output thấp; đo PWM; tháo motor dùng LED/test load phù hợp. Tìm ranh giới cuối đúng/đầu sai.

19.9 Dụng cụ trả lời câu hỏi nào?

- Multimeter: rail DC, continuity khi tắt nguồn, điện trở, dòng có kỹ thuật đúng.
- Oscilloscope: dạng sóng analog, ripple, ringing, PWM, thời gian cạnh.
- Logic analyzer: digital protocol/timing, decode UART/I2C/SPI.
- Serial/log: state, event, error code, timestamp.
- LED test: heartbeat/fault tối thiểu khi Serial không có.

Không dùng logic analyzer 3,3 V không 5 V tolerant lên Uno mà chưa xác minh. Ground clip oscilloscope nối earth có thể gây ngắn nếu đặt sai trong mạch không cách ly; thực hành nguồn USB/low-voltage và hiểu thiết bị.

19.10 Heartbeat và reset cause

Heartbeat đổi theo chu kỳ chứng minh loop còn tiến. Nhưng heartbeat ISR/timer vẫn có thể chạy khi main loop treo, nên đặt ở loop. Lưu/đọc reset cause sớm để phân biệt power-on, watchdog, brown-out; thanh ghi tùy MCU. Log boot count thưa có wear consideration.

19.11 Assertions và invariant

Trong debug build, kiểm state/range; khi vi phạm, đưa output safe, log file/line và reset hoặc dừng tùy hệ.

```
void fail(const __FlashStringHelper* message) {
  setOutputsSafe();
  Serial.println(message);
  while (true) {
    blinkFaultCode();
  }
}

#define REQUIRE(cond, msg) do { if (!(cond)) fail(F(msg)); } while (0)
```

Vòng dừng phù hợp bench, không phải mọi sản phẩm; watchdog/reset loop có thể tệ. Production cần fault policy.

19.12 Test logic trên host

Controller thuần C++ có thể compile bằng g++ và framework test, không cần board. Test thời gian rollover bằng số gần max, state transition, hysteresis, parser. Driver cần integration test trên board/mock HAL.

```
void test_hysteresis_keeps_state_in_band() {
  Controller state;
  state.lampOn = false;
  update(state, 500);
  assert(!state.lampOn);
  state.lampOn = true;
  update(state, 500);
  assert(state.lampOn);
}
```

19.13 Hardware-in-the-loop và test jig

Một jig nối output về input qua bảo vệ, tạo mức sensor bằng DAC/pot số, đọc rail/current và điều khiển reset. Test: boot safe, pin mapping, range ADC, actuator enable, communication, watchdog. Không loopback output công suất trực tiếp vào input.

19.14 Fault injection

Rút sensor, giữ I2C line low qua mạch an toàn, gửi frame hỏng, làm buffer đầy, mô phỏng millis rollover, giới hạn nguồn để quan sát brownout trong điều kiện bảo vệ. Mục tiêu xác minh fault state, không phá thiết bị.

19.15 Lỗi Heisenbug do logging

Thêm Serial làm timing đổi và lỗi biến mất. Dùng GPIO marker, ring buffer trong RAM xuất sau, giảm baud load, logic analyzer, hoặc conditional logging. Ghi timestamp bằng micros nhưng hiểu overhead.

Đừng bypass bảo vệ để “test nhanh”

Cầu chì, diode, current limit, interlock và watchdog là phần hệ thống. Tháo chúng có thể biến lỗi chẩn đoán thành hỏng phần cứng. Test bảo vệ bằng phương pháp có giới hạn và tải an toàn.

19.16 Bài tập luyện tập

1. Viết bug report đầy đủ cho một lỗi wiring thật.
2. Xây heartbeat loop và đo khoảng cách tối đa bằng logic analyzer.
3. Test parser với dòng dài, byte ngẫu nhiên, thiếu newline.
4. Tách hysteresis/state machine ra host test, gồm rollover.

5. Lập ma trận fault: sensor missing/stuck, bus timeout, low voltage, storage CRC.
6. Thiết kế self-test boot không kích actuator nguy hiểm.

19.17 Tóm tắt cuối chương

Bản đồ kiến thức

Gỡ lỗi nhúng cần bằng chứng cả phần mềm lẫn điện. Mô tả triệu chứng đo được, chia chuỗi và tiêm tín hiệu giả. Dụng cụ có câu hỏi riêng. Test logic thuần trên host, driver trên board, toàn hệ bằng jig. Fault injection xác minh trạng thái an toàn; logging có thể làm đổi timing.

Góc nhìn cá nhân

Một mạch lỗi thường kể hai câu chuyện: code nói đã bật, đồng hồ nói chân không đổi; hoặc chân đổi, motor không quay. Gỡ lỗi là đặt hai lời kể cạnh nhau cho đến khi tìm được điểm chúng bắt đầu bất đồng.

Chương 20

Nguồn, nhiễu và thiết kế an toàn – Hệ thống chỉ đáng tin khi nguồn cũng đáng tin

Giới thiệu ngắn

Nhiều project chạy trên USB nhưng reset khi motor khởi động. Nguồn không chỉ là "5 V": có dòng đỉnh, sụt áp, ripple, đường hồi dòng, regulator nhiệt và transient. Thiết kế an toàn còn cần trạng thái khi sensor hỏng, watchdog, bảo vệ phần cứng và phân tích điều gì có thể sai.

Mục tiêu học tập

- Lập power budget theo dòng trung bình/đỉnh và regulator.
- Hiểu decoupling, bulk capacitor, ground return và tách tải nhiễu.
- Nhận biết brownout, reset loop và back-powering.
- Dùng watchdog, fuse/current limit, diode và interlock theo vai trò.
- Thực hiện FMEA nhỏ và định nghĩa safe state.

20.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một mạch chạy tốt bằng USB nhưng reset khi dùng motor không chứng minh code sai. Nguồn có điện trở trong, dây có sụt áp, tải cảm tạo xung, tụ và layout ảnh hưởng đường hồi dòng. Thiết kế nguồn phải xét dòng đỉnh, margin, decoupling và lỗi nối nhầm.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **nguồn, nhiễu và an toàn như điều kiện tồn tại của hệ thống**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

20.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: điện áp vào/ra, dòng trung bình/đỉnh, sụt áp, nhiệt, ripple, ground return và bảo vệ. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$V_{\text{load}} = V_s - I R_{\text{path}}, \qquad P_{\text{reg}} = (V_{\text{in}} - V_{\text{out}})I$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

20.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

20.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mức rail, năng lượng tụ, nhiệt regulator và trạng thái brown-out. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Mọi rail nằm trong giới hạn ở tải đỉnh; lỗi nguồn đưa hệ thống về trạng thái không gây nguy hiểm.

20.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Thiết kế nguồn cho Uno, cảm biến và servo. Ta lập bảng dòng, nhân hệ số dự phòng, tách đường tải, thêm tụ gần tải, nối ground có chủ đích và đo sụt áp tại đúng điểm.

20.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

20.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

20.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

20.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

20.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

20.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

20.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

20.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Cộng dòng danh nghĩa mà bỏ dòng khởi động.
- Dùng regulator tuyến tính không tính nhiệt.
- Nối ground vòng dài chung motor/sensor.
- Đưa điện lưới lên breadboard.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

20.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Lập power budget.
2. Tính nhiệt regulator.
3. Vẽ đường hồi dòng.
4. Đo ripple ở ba trạng thái.
5. Thiết kế fuse/reverse polarity ở mức khái niệm.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

20.7 Power budget

Liệt kê mỗi rail, điện áp, tải, dòng idle/typical/peak, duty và margin. Servo 200 mA trung bình có thể peak/stall trên 1 A tùy loại. Nguồn 500 mA không đủ chỉ vì tổng trung bình 300 mA.

Công suất regulator tuyến tính:

$$P_{loss} = (V_{in} - V_{out})I.$$

12 V xuống 5 V ở 0,5 A mất 3,5 W, quá nóng cho regulator nhỏ. Dùng buck phù hợp, tản nhiệt và hiệu suất. Không dựa jack Uno cấp tải lớn.

20.8 Decoupling và bulk

Tụ gốm nhỏ gần chân Vcc cung cấp dòng chuyển mạch cao tần với vòng ngắn; tụ bulk xử lý biến động chậm/đỉnh lớn hơn. Giá trị/ESR/layout theo datasheet. Một tụ lớn ở xa không thay tụ decoupling gần chip.

Motor/relay tạo nhiễu; tách rail công suất/logic có lọc, nối ground theo đường hồi dòng hợp lý, diode/snubber và dây xoắn có thể cần. Breadboard dài có điện trở/điện cảm, không phù hợp dòng lớn.

20.9 Brownout và reset

Sụt Vcc làm MCU reset hoặc chạy ngoài thông số. Brown-out detector đặt ngưỡng reset khi nguồn thấp; fuse setting tùy MCU/core. Nếu actuator bật khi boot và gây sụt lặp, hệ vào reset loop. Thiết kế output disabled bằng phần cứng cho đến khi MCU chủ động enable sau self-test.

Log boot cause, đo Vcc bằng oscilloscope tại MCU khi tải chuyển, không chỉ multimeter trung bình.

20.10 Back-powering

Thiết bị tắt nguồn nhưng chân tín hiệu nhận HIGH có thể được cấp ngược qua diode bảo vệ, tạo hành vi lạ/hỏng. Khi hai nguồn bật/tắt độc lập, xem power sequencing, level shifter có power-off protection, series resistor/buffer và trạng thái pin. Không nối rail 5 V USB và nguồn ngoài 5 V trực tiếp nếu chưa thiết kế current sharing/ORing.

20.11 Watchdog

Watchdog reset nếu phần mềm không “cho ăn” đúng hạn. Chỉ feed sau khi vòng quan trọng hoàn tất, không trong timer ISR độc lập vì main có thể treo mà watchdog vẫn được feed. Sau reset, output phải safe và log cause. Watchdog không sửa deadlock nguồn, sensor nguy hiểm hoặc bug tái diễn; có thể tạo reset loop.

20.12 Các lớp bảo vệ

- Fuse/polyfuse/current limit: giới hạn hậu quả overcurrent.
- Reverse polarity/ideal diode: bảo vệ cắm ngược.
- TVS/snubber/flyback: transient theo loại tải.
- Thermal fuse/thermostat: ngắt nhiệt độc lập.
- Limit switch và hardware enable: giới hạn cơ khí/năng lượng.

- Enclosure, connector khóa, strain relief: ngăn chạm/tuột.
- Watchdog/brownout: phục hồi một số lỗi phần mềm/nguồn.

Mỗi linh kiện có rating và failure mode; không thêm theo danh sách mà thiếu tính toán.

20.13 FMEA nhỏ

Lập bảng component/function, failure mode, effect, severity, detection, mitigation. Ví dụ sensor độ ẩm đứt trả max/min? Bơm có chạy mãi? Mitigation: plausibility, max runtime hardware/software, flow sensor, bình chống tràn, fuse. Ưu tiên severity trước "xác suất thấp" khi hậu quả lớn.

Lỗi	Hậu quả	Phát hiện	Giảm thiểu
Sensor đứt	bơm bật sai	range/stale	max runtime, state fault
MCU treo	output giữ	watchdog và heartbeat	hardware mặc định tắt
MOSFET chập	tải luôn bật	current/feedback	fuse, cutoff độc lập
Nguồn sụt	reset lặp	reset cause/Vcc	nguồn đủ, UVLO, delayed enable

20.14 Điện lưới, pin và giới hạn sách

Không đấu điện lưới. Pin lithium cần cell có bảo vệ/BMS, charger đúng hóa học, enclosure và kiểm nhiệt; không tự hàn cell không tab. Project liên quan người, y tế, cháy, cửa khóa, phương tiện cần tiêu chuẩn/chuyên môn ngoài giáo trình.

Safe state không luôn là OFF

Một van làm mát tắt có thể nguy hiểm, phanh mất điện có thể cần engage. Safe state phụ thuộc phân tích năng lượng và quá trình. Phần cứng phải đưa hệ về trạng thái đó khi MCU chưa khởi động hoặc mất kiểm soát.

20.15 Bài tập luyện tập

1. Lập power budget cho Uno+OLED+servo+sensor với peak/margin.
2. Tính tổn hao regulator tuyến tính cho ba Vin/I, chọn khi cần buck.
3. Dùng oscilloscope đo Vcc khi servo chạy tải nhẹ, quan sát min/ripple an toàn.
4. Vẽ đường hồi dòng motor và logic; đề xuất bố trí lại.
5. Cài watchdog đúng feed point, mô phỏng main treo và đọc reset cause.
6. Làm FMEA 8 failure mode cho dự án chậu cây; chọn ba mitigation ưu tiên.

20.16 Tóm tắt cuối chương

Bản đồ kiến thức

Nguồn cần budget dòng đỉnh, margin và kiểm nhiệt regulator. Decoupling/bulk/layout xử lý dải nhiễu khác nhau. Brownout/reset loop và back-powering cần power sequencing/default hardware. Watchdog là một lớp, không phải bảo đảm. FMEA nối failure đến effect, detection và mitigation; safe state phụ thuộc hệ thống.

Góc nhìn cá nhân

Một project chỉ được cấp nguồn đúng khi mọi thứ thuận lợi chưa thật sự được cấp nguồn đúng. Sự tin cậy bắt đầu ở khoảnh khắc ta hỏi motor khởi động, dây tuột, sensor chết và MCU reset thì rail điện và output sẽ làm gì.

Phần VI

Xưởng dự án

Chương 21

Đèn giao thông ở ngã tư tí hon – Từ chuỗi lệnh đến hệ thống có luật

Câu chuyện mở đầu

Ba chiếc LED bật lần lượt là một bài tập nhập môn. Một ngã tư có nút cho người đi bộ, thời gian chuyển pha, yêu cầu không bao giờ cho hai hướng xung đột cùng xanh lại là một hệ thống. Khác biệt không nằm ở số LED; khác biệt nằm ở việc ta viết ra luật trước khi viết code.

Dự án này mô phỏng một hướng xe cơ giới và một lối qua đường. Nó đủ nhỏ để dựng trên breadboard nhưng chứa hầu hết câu hỏi quan trọng của hệ nhúng: trạng thái nào hợp lệ, một sự kiện đến giữa pha phải được xử lý ra sao, làm thế nào để không chặn chương trình, mất cảm biến thì hệ về đâu và ta chứng minh điều kiện an toàn bằng cách nào.

Mục tiêu học tập

- Chuyển yêu cầu tự nhiên thành bảng trạng thái và bất biến an toàn.
- Xây máy trạng thái hữu hạn chạy bằng `millis()`.
- Chốt yêu cầu nút nhấn mà không làm rối chu kỳ hiện tại.
- Tách logic quyết định khỏi hàm điều khiển chân.
- Kiểm thử theo kịch bản, biên thời gian và lỗi giả lập.

21.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một ngã tư không được phép cho hai hướng xung đột cùng xanh. Chu kỳ đèn còn cần vàng và khoảng all-red để giải phóng giao lộ. Đây là bài toán bất biến an toàn trước khi là bài Blink nhiều LED.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **đèn giao thông như hệ thống thời gian, trạng thái và yêu cầu an toàn**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

21.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: pha, thời lượng, hướng xung đột, nút người đi bộ, cảm biến, deadline và trạng thái lỗi. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$S_{k+1} = \delta(S_k, t, e), \qquad \neg(G_A \wedge G_B)$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

21.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

21.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là pha hiện tại, thời điểm vào pha và yêu cầu đang chờ. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Không hai hướng xung đột cùng xanh; mọi chuyển xanh đi qua pha giải phóng đã quy định.

21.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Thiết kế ngã tư hai hướng có nút người đi bộ. Ta lập conflict matrix, state table, thời gian min/max, hàng đợi yêu cầu và thử mọi chuyển tiếp kể cả reset giữa pha.

21.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

21.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

21.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

21.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

21.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

21.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

21.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

21.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Viết delay nối tiếp rồi không xử lý nút.
- Không có all-red.
- Cho sự kiện đổi pha ngay phá thời gian tối thiểu.
- Khởi động với output chưa xác định.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

21.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ conflict matrix.
2. Lập state table.
3. Thêm yêu cầu người đi bộ.
4. Thiết kế startup an toàn.
5. Chạy model-based test trên mọi chuyển.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô

và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

21.7 Định nghĩa bài toán trước khi cắm dây

Ta dùng LED đỏ, vàng, xanh cho xe; LED đỏ và xanh cho người đi bộ; một nút nhấn mắc `INPUT_PULLUP`. Luật vận hành:

1. Bình thường xe xanh trong ít nhất 8 giây.
2. Khi người đi bộ nhấn, yêu cầu được ghi nhớ; không đổi đèn tức thời.
3. Hết thời gian tối thiểu, xe chuyển vàng 2 giây rồi tất cả đỏ 1 giây.
4. Người đi bộ xanh 5 giây, sau đó nhấp nháy 3 giây để báo sắp hết.
5. Tất cả đỏ 1 giây trước khi trả xe về xanh.

Các con số chỉ để thực hành. Hệ giao thông thật chịu tiêu chuẩn, chứng nhận, phần cứng dự phòng và phân tích an toàn nghiêm ngặt; breadboard này không được dùng điều khiển giao thông thật.

Ranh giới của mô hình

Đây là mô hình điện áp thấp dùng LED. Không nối với đèn công suất, điện lưới hay cơ cấu ngoài đời. Một chương trình chạy đúng vài phút không phải bằng chứng đủ cho hệ thống liên quan an toàn con người.

21.8 Bất biến – những câu không được phép sai

Yêu cầu thời gian có thể thay đổi, nhưng ba điều sau luôn phải đúng:

- xe xanh và người xanh không cùng bật;
- trước khi quyền đi chuyển từ xe sang người hoặc ngược lại phải có pha tất cả đỏ;
- trạng thái không nhận diện được phải đưa output về tất cả đỏ.

Đó là *bất biến*. Viết bất biến trước giúp review code dễ hơn: ở mọi nhánh của hàm xuất đèn, ta kiểm tra ba câu này thay vì đọc theo cảm giác.

21.9 Vẽ máy trạng thái

Ta chọn sáu trạng thái: `CAR_GREEN`, `CAR_YELLOW`, `ALL_RED_TO_WALK`, `WALK`, `WALK_FLASH` và `ALL_RED_TO_CAR`. Mỗi trạng thái có output xác định và một điều kiện rời đi.

Trạng thái	Output chính	Điều kiện chuyển
Xe xanh	xe xanh, người đỏ	có yêu cầu và đủ 8 s
Xe vàng	xe vàng, người đỏ	đủ 2 s
Tất cả đỏ sang người	mọi quyền đi đỏ	đủ 1 s
Người đi	xe đỏ, người xanh	đủ 5 s
Người nhấp nháy	xe đỏ, LED người chớp	đủ 3 s
Tất cả đỏ sang xe	mọi quyền đi đỏ	đủ 1 s

Nút nhấn không trực tiếp ra lệnh chuyển trạng thái. Nó chỉ đặt `walkRequested = true`. Biến này là một hàng đợi một phần tử: nhiều lần nhấn trước khi được phục vụ vẫn chỉ là một yêu cầu. Khi bắt đầu pha người đi, chương trình xóa cờ.

21.10 Sơ đồ nối và kiểm tra từng output

Mỗi LED đi từ chân số qua điện trở 220–330 Ω tới LED rồi về GND. Nút nối chân 2 xuống GND; chế độ kéo lên làm trạng thái thả là HIGH, nhấn là LOW. Chọn chân 3, 4, 5 cho đèn xe và 8, 9 cho đèn người. Trước khi chạy thuật toán, nạp sketch kiểm tra để bật riêng từng LED một giây. Nếu màu nào sai, sửa dây hoặc hằng chân ngay; đừng sửa thuật toán để bù dây sai.

Phiếu kiểm tra trước khi tích hợp

Đánh dấu năm LED đúng màu, đúng điện trở, chung GND; nút thả đọc HIGH, nhấn đọc LOW; không có LED nào sáng mờ khi đáng lẽ tắt. Chỉ khi từng phần qua kiểm tra mới ghép máy trạng thái.

21.11 Cài đặt không chặn

Listing 21.1: Máy trạng thái đèn giao thông tối giản

```
enum class State : byte {
  CAR_GREEN, CAR_YELLOW, ALL_RED_TO_WALK,
  WALK, WALK_FLASH, ALL_RED_TO_CAR
};

constexpr byte CAR_R = 3, CAR_Y = 4, CAR_G = 5;
constexpr byte PED_R = 8, PED_G = 9, BUTTON = 2;

State state = State::CAR_GREEN;
unsigned long enteredAt = 0;
bool walkRequested = false;

bool elapsed(unsigned long now, unsigned long duration) {
```

```
  return (unsigned long)(now - enteredAt) >= duration;
}

void enter(State next, unsigned long now) {
  state = next;
  enteredAt = now;
}

void setLamps(bool cr, bool cy, bool cg, bool pr, bool pg) {
  digitalWrite(CAR_R, cr); digitalWrite(CAR_Y, cy);
  digitalWrite(CAR_G, cg); digitalWrite(PED_R, pr);
  digitalWrite(PED_G, pg);
}

void render(unsigned long now) {
  switch (state) {
    case State::CAR_GREEN:       setLamps(0,0,1,1,0); break;
    case State::CAR_YELLOW:      setLamps(0,1,0,1,0); break;
    case State::ALL_RED_TO_WALK:
    case State::ALL_RED_TO_CAR:  setLamps(1,0,0,1,0); break;
    case State::WALK:            setLamps(1,0,0,0,1); break;
    case State::WALK_FLASH:
      setLamps(1,0,0,0, ((now / 250UL) % 2UL) == 0UL);
      break;
    default:                     setLamps(1,0,0,1,0); break;
  }
}

void updateState(unsigned long now) {
  switch (state) {
    case State::CAR_GREEN:
      if (walkRequested && elapsed(now, 8000))
        enter(State::CAR_YELLOW, now);
      break;
    case State::CAR_YELLOW:
      if (elapsed(now, 2000)) enter(State::ALL_RED_TO_WALK, now);
      break;
    case State::ALL_RED_TO_WALK:
      if (elapsed(now, 1000)) {
        walkRequested = false; enter(State::WALK, now);
      }
      break;
    case State::WALK:
      if (elapsed(now, 5000)) enter(State::WALK_FLASH, now);
      break;
    case State::WALK_FLASH:
      if (elapsed(now, 3000)) enter(State::ALL_RED_TO_CAR, now);
```

```
      break;
    case State::ALL_RED_TO_CAR:
      if (elapsed(now, 1000)) enter(State::CAR_GREEN, now);
      break;
    default: enter(State::ALL_RED_TO_CAR, now); break;
  }
}
```

Hàm `render()` chỉ biến trạng thái thành output; `updateState()` chỉ quyết định chuyển. Cách tách này làm bất biến dễ quan sát. Phép trừ thời gian dùng kiểu `unsigned long` nên vẫn đúng khi `millis()` tràn, miễn khoảng chờ nhỏ hơn nửa miền đếm.

21.12 Đọc nút như một sự kiện sạch

Nút cơ dội. Ta có thể tái sử dụng lớp debounce ở Chương 5: chỉ khi mức LOW ổn định 30 ms mới phát sự kiện `pressed`. Trong bản đầu, polling mỗi vòng là đủ; không cần interrupt. Khi có cạnh nhấn sạch, đặt cờ yêu cầu.

Listing 21.2: Vòng lặp giữ ba trách nhiệm rõ ràng

```
void loop() {
  const unsigned long now = millis();

  button.update(now, digitalRead(BUTTON));
  if (button.fell()) walkRequested = true;

  updateState(now);
  render(now);
}
```

Thứ tự đọc input, cập nhật trạng thái, xuất output làm cho một vòng lặp giống một bước của mô hình. Nếu cần log, thêm sau chuyển trạng thái chứ không in liên tục mỗi vòng.

21.13 Kiểm thử bằng kịch bản

Không chỉ đứng nhìn đèn. Viết bảng trước:

Kịch bản	Kích thích	Kết quả mong đợi
Không ai nhấn	chạy 30 s	xe luôn xanh, người đỏ
Nhấn giây thứ 2	một cạnh sạch	xe còn xanh đến giây 8 rồi theo đủ chuỗi
Nhấn liên tục	giữ nút LOW	chỉ một yêu cầu, không lặp pha vô hạn
Nhấn lúc WALK	một cạnh	quy ước rõ: bỏ qua hoặc giữ cho chu kỳ sau
Tràn thời gian	đặt thời điểm gần cực đại	thời lượng vẫn đúng
State lỗi	ép giá trị không hợp lệ	mọi quyền đi đỏ

Để không chờ lâu khi test, đặt thời lượng trong cấu trúc cấu hình và dùng bộ giá trị nhanh, chẳng hạn 800 ms thay 8 s. Không rải "số ma thuật" để rồi bỏ sót một chỗ.

21.14 Mở rộng có chủ đích

Thêm hướng xe thứ hai đòi hỏi trạng thái riêng cho hai hướng, khoảng đỏ giao nhau và bất biến "không có hai hướng xung đột cùng xanh". Thêm còi hỗ trợ người khiếm thị cần nhịp không chặn. Chế độ đêm có thể nhấp nháy vàng nhưng phải được kích hoạt qua một yêu cầu chuyển mode có kiểm soát. Mỗi mở rộng bắt đầu bằng bảng trạng thái mới, không bắt đầu bằng thêm một chuỗi `if`.

21.15 Bài tập luyện tập

1. Vẽ sơ đồ trạng thái và ghi guard/action trên từng cạnh.
2. Cài debounce và chứng minh giữ nút không tạo nhiều sự kiện.
3. Thêm hai cấu hình thời lượng: demo và bình thường.
4. Log một dòng chỉ khi trạng thái đổi; gồm state cũ, state mới và thời điểm.
5. Viết hàm `invariantHolds()` kiểm tra các output xung đột trong bản mô phỏng.
6. Thêm chế độ lỗi làm tất cả đỏ nhấp nháy, có cách reset rõ ràng.
7. Thiết kế test cho yêu cầu đến đúng tại biên 7.999 s, 8.000 s và 8.001 s.

21.16 Tóm tắt cuối chương

Bản đồ kiến thức

Dự án đèn giao thông bắt đầu từ luật và bất biến, sau đó mới đến dây và code. Cờ yêu cầu tách sự kiện nút khỏi thời điểm được phục vụ. Máy trạng thái cùng `millis()` giữ hệ phản hồi mà không chặn. Tách quyết định khỏi xuất chân giúp đọc, kiểm thử và đưa hệ về tất cả đỏ khi gặp trạng thái lạ.

Góc nhìn cá nhân

Một chiếc đèn giao thông mô hình dạy ta điều lớn hơn chuyện bật LED: phần mềm nhúng là lời hứa về thứ tự và điều không bao giờ được xảy ra. Khi lời hứa được viết thành trạng thái, bất biến và test, mạch nhỏ bắt đầu có phẩm chất của một hệ thống.

Chương 22

Chậu cây biết chờ – Đo độ ẩm, quyết định tưới và phòng lỗi

Câu chuyện mở đầu

"Đất khô thì bật bơm" nghe như một câu `if`. Nhưng cảm biến ẩm đất trôi theo loại đất, điện cực ăn mòn, nước ngấm có độ trễ, bơm gây nhiễu nguồn và một dây sensor đứt có thể trông giống "rất khô". Nếu chỉ ghép một ngưỡng với một relay, chậu cây thông minh dễ trở thành máy bơm nước vô hạn.

Ta sẽ xây một mô hình điện áp thấp: cảm biến độ ẩm, LED mô phỏng bơm ở giai đoạn đầu, nút tưới thủ công và Serial để quan sát. Khi logic đã qua kiểm thử mới thay LED bằng bơm DC có nguồn riêng, MOSFET logic-level, diode flyback và giới hạn thời gian. Không dùng relay/bơm điện lưới.

Mục tiêu học tập

- Viết yêu cầu tưới dưới dạng dữ liệu, trạng thái và giới hạn.
- Hiệu chuẩn cảm biến theo hệ lắp thật và biểu diễn phần trăm có ý nghĩa.
- Dùng hysteresis, thời gian xác nhận và khoảng nghỉ sau tưới.
- Phát hiện giá trị bất khả tín, sensor stale và runtime quá mức.
- Lập test, log và FMEA cho hệ có nước và cơ cấu chấp hành.

22.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Độ ẩm đất thay đổi chậm, cảm biến dễ ăn mòn và số đo phụ thuộc đất. Bơm lại có dòng lớn và nếu cảm biến rời có thể tưới vô hạn. Một hệ tưới đúng cần hiệu chuẩn, hysteresis, giới hạn thời gian, khoảng nghỉ thấm nước và phát hiện lỗi.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **chậu cây thông minh như hệ thống đo ẩm, ra quyết định và phòng tưới sai**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

22.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: ADC khô/ướt, độ ẩm chuẩn hóa, ngưỡng bật/tắt, thời gian bơm, thời gian nghỉ và mực nước. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$m = \frac{x - x_{\text{dry}}}{x_{\text{wet}} - x_{\text{dry}}}, \qquad 0 \le m \le 1$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

22.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

22.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mode chờ/tưới/nghỉ/lỗi, lần tưới cuối và tổng thời gian bơm. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k, x_k, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Bơm không chạy quá giới hạn; cảm biến lỗi hoặc giá trị bất khả dĩ luôn đưa hệ thống về không tưới và cảnh báo.

22.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Hiệu chuẩn cảm biến trong chính loại đất, lấy nhiều mẫu, chọn hai ngưỡng, bơm theo xung ngắn rồi chờ nước thấm trước khi đo lại. Ta mô phỏng cảm biến rời và bình hết nước.

22.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

22.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

22.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

22.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

22.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

22.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

22.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

22.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Dùng ngưỡng từ Internet.
- Bơm liên tục đến khi số đo đổi.
- Cấp bơm từ chân board.
- Không có phát hiện cảm biến rời.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

22.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Thu đường cong khô–ướt.
2. Thiết kế hysteresis.
3. Lập state machine tưới.
4. Thêm timeout và cooldown.
5. Viết test fault injection.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô

và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

22.7 Tách hệ thống thành năm khối

1. **Quan sát**: lấy mẫu ADC và lọc nhiễu.
2. **Diễn giải**: đổi raw sang mức ẩm đã hiệu chuẩn, gắn cờ hợp lệ.
3. **Quyết định**: xác định cần tưới dựa ngưỡng và lịch sử.
4. **Hành động**: bật bơm có giới hạn năng lượng/thời gian.
5. **Giám sát**: log, cảnh báo, khóa lỗi và cho phép người dùng reset.

Chia khối giúp ta kiểm thử logic bằng dữ liệu giả trước khi có bơm. Nó cũng ngăn việc một lần đọc ADC vô tình điều khiển tải ngay trong cùng biểu thức.

22.8 Hiệu chuẩn thay vì tin số phần trăm trên mạng

Ghi `rawWet` khi sensor ở đất đủ ẩm theo mục tiêu canh tác và `rawDry` khi đất khô theo quy ước của chính chậu. Cảm biến có thể tăng hoặc giảm theo độ ẩm, nên công thức phải biết chiều:

$$m = 100\,\frac{x - d}{w - d}, \qquad m \leftarrow \min(100, \max(0, m)).$$

Ở đây x là raw đã lọc, d là mốc khô, w là mốc ướt. Hai mốc quá gần khiến phép đổi rất nhạy; đó là dấu hiệu cơ khí/sensor chưa ổn. Lưu mốc vào EEPROM kèm version và checksum nếu muốn tồn tại sau reset.

Phần trăm là một thang quy ước

Con số 45% chỉ có ý nghĩa trong cùng sensor, vị trí, đất và quy trình hiệu chuẩn. Nó không tự động là độ ẩm thể tích chuẩn phòng thí nghiệm. Mục tiêu ở đây là quyết định lặp lại được, không phải tạo độ chính xác giả.

22.9 Cảm biến và tuổi thọ

Đầu dò điện trở cấp DC liên tục dễ ăn mòn điện hóa. Có thể chỉ cấp nguồn lúc đo qua một chân/transistor phù hợp, nhưng cảm biến điện dung chất lượng tốt thường phù hợp hơn cho thử nghiệm dài ngày. Dù loại nào, tránh nhúng phần mạch không chống nước, cố định độ sâu/vị trí và quan sát drift nhiều ngày.

Lấy chẳng hạn 15 mẫu cách nhau 10 ms, bỏ vài mẫu đầu sau khi cấp nguồn sensor, dùng median hoặc trimmed mean. Không cần đọc hàng nghìn lần mỗi giây; đất thay đổi chậm.

Chu kỳ vài giây đến vài phút thường đủ tùy mục tiêu.

22.10 Chính sách tưới có trí nhớ

Đặt `START_BELOW = 35%`, `STOP_ABOVE = 45%`; đây là hysteresis. Tuy vậy nước chưa kịp lan đến sensor trong vài giây. Ta không chờ sensor vượt ngưỡng khi bơm đang chạy; thay vào đó dùng xung tưới ngắn rồi nghỉ thấm:

- độ ẩm dưới 35% liên tục 30 s mới yêu cầu tưới;
- bơm chạy tối đa 3 s mỗi xung;
- nghỉ thấm ít nhất 2 phút rồi mới đánh giá lại;
- tối đa ba xung trong một cửa sổ 30 phút;
- sensor không hợp lệ thì không tưới tự động.

Các số phải được hiệu chỉnh theo lưu lượng bơm, thể tích chậu và cây. Giới hạn cuối cùng nên có cả phần cứng: bình chứa nhỏ, khay chống tràn, cầu chì và nếu hậu quả đáng kể thì cảm biến mức/lưu lượng độc lập.

22.11 Máy trạng thái vận hành

Ta dùng `MONITOR`, `CONFIRM_DRY`, `WATERING`, `SOAK`, `FAULT`. Ở `MONITOR`, một lần thấp chỉ bắt đầu xác nhận. Nếu trong lúc xác nhận độ ẩm hồi lên, trở về theo dõi. Hết thời gian xác nhận mới tưới. Hết xung tưới chuyển sang nghỉ thấm bất kể sensor nói gì. Lỗi sensor hoặc vượt quota đi vào `FAULT`; bơm tắt.

Listing 22.1: Khung trạng thái chậu cây, output mặc định tắt

```
enum class PlantState : byte {
  MONITOR, CONFIRM_DRY, WATERING, SOAK, FAULT
};

constexpr byte PUMP = 6;
constexpr unsigned long CONFIRM_MS = 30000UL;
constexpr unsigned long PULSE_MS   = 3000UL;
constexpr unsigned long SOAK_MS    = 120000UL;

PlantState plantState = PlantState::MONITOR;
unsigned long stateSince = 0;

void enterPlantState(PlantState next, unsigned long now) {
  plantState = next;
  stateSince = now;
}

```

```
bool stateElapsed(unsigned long now, unsigned long d) {
  return (unsigned long)(now - stateSince) >= d;
}

void updatePlant(unsigned long now, float moisture, bool valid) {
  digitalWrite(PUMP, LOW);             // default safe action

  if (!valid) enterPlantState(PlantState::FAULT, now);

  switch (plantState) {
    case PlantState::MONITOR:
      if (moisture < 35.0f)
        enterPlantState(PlantState::CONFIRM_DRY, now);
      break;
    case PlantState::CONFIRM_DRY:
      if (moisture >= 40.0f)
        enterPlantState(PlantState::MONITOR, now);
      else if (stateElapsed(now, CONFIRM_MS))
        enterPlantState(PlantState::WATERING, now);
      break;
    case PlantState::WATERING:
      digitalWrite(PUMP, HIGH);
      if (stateElapsed(now, PULSE_MS))
        enterPlantState(PlantState::SOAK, now);
      break;
    case PlantState::SOAK:
      if (stateElapsed(now, SOAK_MS))
        enterPlantState(PlantState::MONITOR, now);
      break;
    case PlantState::FAULT:
    default:
      digitalWrite(PUMP, LOW);
      break;
  }
}
```

Đây mới là khung. Quota số xung, nút reset có chủ ý, phát hiện stale và chế độ manual phải được thêm trước khi điều khiển bơm thật. Điểm quan trọng là output được đặt LOW trước mọi nhánh; lỗi không vô tình giữ trạng thái HIGH từ vòng trước.

22.12 Một phép đo được coi là hợp lệ khi nào?

Giá trị nằm trong 0–1023 chưa chắc hợp lệ. Ta kiểm tra nhiều lớp:

– **Range điện**: raw không kẹt gần 0 hoặc 1023 ngoài dự kiến.

- **Plausibility**: không nhảy 40 điểm phần trăm trong một mẫu nếu không có thao tác.
- **Stale**: giá trị có thay đổi nhỏ trong một khoảng dài; dây chập có thể tạo số đẹp nhưng bất động.
- **Response**: sau xung tưới và thời gian thấm, xu hướng phải tăng ít nhất một mức tối thiểu; nếu không, nghi hết nước, bơm hỏng hoặc sensor lệch vị trí.
- **Freshness**: thời điểm mẫu cuối không quá cũ vì tác vụ lấy mẫu có thể đã ngừng.

Không phải mọi bất động đều là lỗi vì đất thật có thể ổn định. Do đó mỗi luật cần thời gian và bằng chứng phù hợp, tránh biến giám sát thành nguồn báo động giả.

22.13 Driver bơm và bố trí nguồn

Uno không cấp bơm từ chân I/O. Dùng MOSFET kênh N logic-level có rating phù hợp, điện trở cổng, pull-down giữ tắt lúc boot, diode flyback đặt gần tải DC cảm, nguồn bơm đủ dòng khởi động và GND chung theo thiết kế. Đặt tụ bulk gần driver, tách đường dòng bơm khỏi đường ground sensor. Kiểm tra nhiệt MOSFET và dây ở tình huống kẹt bơm.

Trước tiên thay bơm bằng LED. Sau đó dùng tải nhỏ trong một khay khô, cấp nguồn giới hạn dòng, test tay chức năng tắt. Chỉ cuối cùng mới cho nước, đặt mạch cao hơn mặt nước, có strain relief và không để mối nối trần nơi có thể bắn nước.

Nước và điện

Chỉ dùng nguồn DC điện áp thấp, cách ly đạt chuẩn. Không thao tác dây khi tay ướt. Không dùng dự án này cho tưới không giám sát ở nơi rò nước có thể gây thiệt hại; mọi thử nghiệm ban đầu phải có người quan sát và giới hạn lượng nước vật lý.

22.14 Manual không được phá luật

Nút manual không nên viết thẳng HIGH ra bơm. Nó tạo một yêu cầu đi qua cùng lớp quota, max runtime và fault interlock. Có thể yêu cầu nhấn giữ 1 giây để tránh chạm nhầm; LED báo trước 2 giây; nhấn lần nữa dừng ngay. Trong `FAULT`, manual bị từ chối hoặc chỉ cho phép qua quy trình service rõ ràng.

22.15 Log để học từ cây

Mỗi 10 s in CSV: thời gian, raw, moisture, state, pump, pulseCount, valid, faultCode. Khi chuyển trạng thái, in event riêng. Dữ liệu vài ngày cho biết chu kỳ khô, ảnh hưởng ánh nắng, lượng tăng sau một xung và ngưỡng có hợp lý không. Đừng log quá nhanh làm Serial chặn hoặc tạo file khổng lồ.

22.16 Kế hoạch kiểm thử và FMEA

Lỗi giả lập	Kỳ vọng	Cách xác nhận
Rút sensor	bơm tắt, vào FAULT	log mã, đo chân driver LOW
Bơm kẹt	hết max runtime	đồng hồ, kiểm nhiệt có kiểm soát
Bình hết nước	không tăng sau tưới	response fault, không lặp vô hạn
Reset giữa tưới	boot mặc định tắt	quan sát LED/logic analyzer
Nút kẹt	không chạy vô hạn	chỉ một yêu cầu, quota còn hiệu lực
Nhiễu motor	MCU không reset	log boot, đo Vcc tại board

FMEA còn xét ống tuột, chậu tràn, MOSFET chập, firmware treo, sai hiệu chuẩn. Một MOSFET chập khiến phần mềm không thể tắt bơm; nếu hậu quả lớn cần cutoff độc lập hoặc giới hạn lượng nước vật lý.

22.17 Bài tập luyện tập

1. Thu raw trong ba độ ẩm và vẽ phân bố, không chỉ lấy một mẫu.
2. Viết hàm đổi raw sang phần trăm hoạt động cho cả hai chiều sensor.
3. Thêm trimmed mean không dùng `delay()` trong luồng chính.
4. Thêm quota ba xung/30 phút với phép tính an toàn khi `millis()` tràn.
5. Thiết kế mã lỗi riêng cho range, stale, no-response và overrun.
6. Viết 12 test logic dùng chuỗi mẫu giả, không cần bơm thật.
7. Tính lượng nước tệ nhất nếu mọi lớp phần mềm đều yêu cầu tối đa; giảm bằng giới hạn vật lý.
8. Vẽ sơ đồ nguồn/ground, chỉ đường dòng khi bơm khởi động.

22.18 Tóm tắt cuối chương

Bản đồ kiến thức

Chậu cây đáng tin không điều khiển bơm từ một mẫu ADC. Ta hiệu chuẩn trong hệ thật, lọc, xác nhận khô, tưới theo xung, chờ thấm và đặt quota. Sensor lỗi hoặc dữ liệu cũ phải dẫn đến bơm tắt. Manual cũng đi qua interlock. Driver, nguồn và giới hạn lượng nước vật lý cùng tham gia an toàn.

Góc nhìn cá nhân

Thông minh không phải phản ứng thật nhanh với mọi con số. Đôi khi thông minh là biết đất thay đổi chậm, biết nước cần thời gian ngấm và biết một cảm biến im lặng không đáng để ta đặt cược cả căn phòng.

Chương 23

Trợ lý đỗ xe để bàn – Đo khoảng cách mà không hoảng vì một tiếng vọng

Câu chuyện mở đầu

Cảm biến siêu âm thường được demo bằng vài dòng: phát xung, gọi `pulseIn()`, đổi microsecond thành centimet và bật còi. Trên bàn sạch, vật phẳng đặt thẳng, con số trông rất thuyết phục. Ngoài thực tế, bề mặt xiên làm tiếng vọng đi chỗ khác, vật mềm hấp thụ âm, cạnh bàn tạo phản xạ, nhiệt độ đổi tốc độ âm và `pulseIn()` có thể chờ quá lâu.

Dự án này làm một trợ lý đỗ xe mô hình với cảm biến khoảng cách, ba vùng LED và còi có nhịp tăng dần. Mục tiêu không phải độ chính xác công nghiệp mà là học cách biến phép đo có lỗ hổng thành phản hồi bình tĩnh, có ngữ nghĩa và có chế độ lỗi.

Mục tiêu học tập

- Hiểu đường đi của xung siêu âm và timeout bắt buộc.
- Lọc outlier, xác nhận vùng và dùng hysteresis theo khoảng cách.
- Tạo nhịp còi không chặn, tách đo khỏi phản hồi.
- Phân biệt “xa” với “không có phép đo hợp lệ”.
- Kiểm thử góc, vật liệu, tốc độ tiếp cận và lỗi dây.

23.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Cảm biến siêu âm suy khoảng cách từ thời gian xung đi và về. Nhiều bề mặt hấp thụ hoặc phản xạ lệch; tiếng vọng giả và timeout phải được xử lý. Cảnh báo không nên rung theo từng mẫu và phải cho người dùng hiểu mức nguy hiểm bằng nhịp/ánh sáng.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **trợ lý đỗ xe như bài toán thời gian bay, nhiễu và giao diện cảnh báo**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

23.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: độ rộng trigger, thời gian echo, tốc độ âm, khoảng cách, timeout, cửa sổ lọc và ngưỡng cảnh báo. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$d = \frac{v_{\text{sound}}\, t}{2}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

23.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

23.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mẫu hợp lệ gần đây, mức cảnh báo và thời điểm phát âm cuối. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k, x_k, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Không có echo không được diễn giải là vật ở 0 cm; cảnh báo chỉ dựa trên số đo đã xác nhận và có mức fail-safe.

23.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Đo vật ở 20, 50 và 100 cm, lấy median, loại timeout, dùng hysteresis giữa các vùng và ánh xạ khoảng cách sang nhịp buzzer không chặn.

23.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

23.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

23.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

23.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

23.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

23.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

23.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

23.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Không chia thời gian cho hai.
- Dùng pulseIn không timeout phù hợp.
- Tin một echo đơn lẻ.
- Đặt hai cảm biến phát cùng lúc gây xuyên nhiễu.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

23.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Tính khoảng cách từ microsecond.
2. Đo sai số theo bề mặt.
3. Thiết kế median filter.
4. Lập vùng cảnh báo có hysteresis.
5. Thử trường hợp không echo.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô

và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

23.7 Từ thời gian bay đến khoảng cách

Xung đi từ đầu phát đến vật rồi quay về, nên:

$$d = \frac{v\,t}{2}.$$

Với $v \approx 343$ m/s ở khoảng 20°C, có thể dùng gần đúng $d_{cm} = t_{\mu s}/58$. Hệ số thay đổi theo nhiệt độ và môi trường; độ phân giải hiển thị 0,01 cm không tạo ra độ chính xác tương ứng.

Module phổ biến có chân Trigger và Echo; một số board Echo mức 5 V, không an toàn trực tiếp cho MCU 3,3 V. Uno R3 dùng logic 5 V, nhưng luôn đọc datasheet module/board đang cầm. Đặt timeout theo khoảng cách tối đa, chẳng hạn 25 ms tương ứng vài mét; không để hàm chờ vô hạn.

23.8 Kiến trúc theo nhịp

Tác vụ đo chạy 20 Hz, lọc/cập nhật vùng sau mỗi mẫu hợp lệ, LED render mỗi vòng, còi dùng scheduler riêng. Không phát trigger quá dày khiến tiếng vọng cũ lẫn lần mới. Một phép đo có bốn kết quả có nghĩa:

- `VALID`: có echo trong cửa sổ và khoảng cách hợp lý;
- `NO_ECHO`: timeout, chưa biết là quá xa hay vật không phản xạ;
- `TOO_CLOSE`: dưới vùng chết của cảm biến;
- `FAULT`: chuỗi lỗi dài hoặc tín hiệu điện bất thường.

Nếu gộp `NO_ECHO` thành 999 cm, giao diện sẽ báo “an toàn, rất xa” dù sensor vừa bị rút. Đó là lỗi mô hình dữ liệu, không phải lỗi công thức.

23.9 Đọc một mẫu có timeout

Listing 23.1: Đọc xung với thời gian chờ hữu hạn

```
struct RangeSample {
  bool valid;
  float cm;
};

RangeSample readUltrasonic(byte trig, byte echo) {
  digitalWrite(trig, LOW);
  delayMicroseconds(3);
  digitalWrite(trig, HIGH);
```

```
  delayMicroseconds(10);
  digitalWrite(trig, LOW);

  const unsigned long us = pulseIn(echo, HIGH, 25000UL);
  if (us == 0) return {false, 0.0f};

  const float cm = us / 58.0f;
  if (cm < 2.0f || cm > 350.0f) return {false, cm};
  return {true, cm};
}
```

`pulseIn()` vẫn chặn đến timeout; 25 ms có thể chấp nhận cho mô hình đơn giản nhưng không phù hợp hệ có deadline ngắn. Bản nâng cao dùng interrupt/input capture hoặc thư viện không chặn đã được đánh giá. Điều quan trọng là biết ngân sách blocking của mình.

23.10 Lọc mà không che mất vật đang đến

Median của năm mẫu loại tốt một xung quá lớn/nhỏ, nhưng cần thời gian lấy đủ. Exponential moving average phản hồi đều song bị kéo bởi outlier. Một pipeline hợp lý:

1. loại mẫu ngoài range hoặc timeout;
2. median cửa sổ 3–5 mẫu hợp lệ;
3. giới hạn tốc độ thay đổi chỉ để phát hiện nghi vấn, không tự ý sửa dữ liệu thật;
4. xác nhận vùng qua hai hoặc ba cập nhật liên tiếp.

Khi xe mô hình tiến nhanh, lọc quá nặng gây trễ cảnh báo. Đánh giá bằng tốc độ tiếp cận tệ nhất: nếu toàn pipeline trễ 200 ms và vật tiến 1 m/s, điểm phản hồi dịch 20 cm. Bộ lọc luôn có giá phải trả.

23.11 Các vùng và hysteresis

Ta đặt: xanh trên 100 cm, vàng 50–100 cm, đỏ 20–50 cm, dừng dưới 20 cm. Nếu số đo dao động quanh 50 cm, màu/còi sẽ rung. Dùng biên vào/ra khác nhau: từ vàng vào đỏ dưới 47 cm, nhưng từ đỏ về vàng phải trên 53 cm. Tương tự cho các biên khác.

Listing 23.2: Máy trạng thái vùng khoảng cách có hysteresis

```
enum class Zone : byte { FAR, CAUTION, NEAR, STOP, UNKNOWN };

Zone updateZone(Zone oldZone, float cm, bool valid) {
  if (!valid) return Zone::UNKNOWN;

  switch (oldZone) {
    case Zone::FAR:     return cm < 97 ? Zone::CAUTION : oldZone;
```

```
    case Zone::CAUTION:
      if (cm > 103) return Zone::FAR;
      if (cm < 47)  return Zone::NEAR;
      return oldZone;
    case Zone::NEAR:
      if (cm > 53) return Zone::CAUTION;
      if (cm < 18) return Zone::STOP;
      return oldZone;
    case Zone::STOP:    return cm > 22 ? Zone::NEAR : oldZone;
    case Zone::UNKNOWN:
    default:
      if (cm < 20) return Zone::STOP;
      if (cm < 50) return Zone::NEAR;
      if (cm < 100) return Zone::CAUTION;
      return Zone::FAR;
  }
}
```

Để chống một mẫu lạc, chỉ commit vùng mới sau hai đề xuất giống nhau. Riêng đề xuất `STOP` có thể được ưu tiên nhanh hơn nếu hậu quả bỏ lỡ lớn hơn báo nhầm; đây là lựa chọn hệ thống cần ghi rõ.

23.12 Còi là một bộ phát sự kiện, không phải một delay

Vùng xa im; caution beep 80 ms mỗi 1.000 ms; near mỗi 400 ms; stop phát âm liên tục hoặc nhịp rất nhanh. Ta lưu `nextBeepAt` và `beepOffAt`. Mỗi vòng kiểm tra deadline rồi bật/tắt buzzer; sensor vẫn được đọc và nút mute vẫn phản hồi.

Listing 23.3: Bộ tạo beep không chặn

```
void updateBeeper(unsigned long now, Zone zone) {
  static bool on = false;
  static unsigned long nextAt = 0;
  static unsigned long offAt = 0;

  unsigned long period = 0;
  if (zone == Zone::CAUTION) period = 1000;
  if (zone == Zone::NEAR)    period = 400;

  if (zone == Zone::STOP) {
    tone(BUZZER, 2200);
    on = true;
    return;
  }
  if (zone == Zone::UNKNOWN || zone == Zone::FAR) {
    noTone(BUZZER); on = false; nextAt = now;
    return;
```

```
  }

  if (on && (long)(now - offAt) >= 0) {
    noTone(BUZZER); on = false;
  }
  if (!on && (long)(now - nextAt) >= 0) {
    tone(BUZZER, 2200); on = true;
    offAt = now + 80; nextAt = now + period;
  }
}
```

So sánh deadline tương lai bằng ép về `long` hoạt động khi các khoảng nhỏ hơn nửa miền. Một cách nhất quán khác là lưu thời điểm trước và dùng phép trừ unsigned. Trong dự án thật, chọn một quy ước và ghi chú.

23.13 UNKNOWN phải nhìn khác FAR

Khi mất echo một lần, giữ vùng cũ trong một cửa sổ ngắn và gắn chất lượng giảm; tiếng vọng thỉnh thoảng rơi là bình thường. Sau chẳng hạn năm lỗi liên tiếp, chuyển `UNKNOWN`: LED lỗi nhấp nháy theo mẫu riêng, còi phát ba tiếng lúc vào lỗi rồi im hoặc theo chính sách. Không hiển thị xanh.

Nếu sensor khôi phục, yêu cầu ba mẫu hợp lệ trước khi rời lỗi. Điều này tránh mode rung giữa lỗi/khỏe. Log số lỗi liên tiếp và tuổi mẫu cuối để phân biệt dây lỏng với vật khó phản xạ.

23.14 Bố trí cơ khí là một phần thuật toán

Hai mắt siêu âm cần mặt thoáng; khung vỏ không che chùm. Đặt sensor vuông góc với bề mặt dự kiến, tránh sàn/bàn phản xạ sát. Thử tấm bìa phẳng, chai tròn, vải, bề mặt xiên và vật hẹp. Nếu ứng dụng có nhiều sensor, phát lần lượt để tránh crosstalk.

Khoảng cách dừng không chỉ là ngưỡng sensor. Nó gồm sai số, độ trễ lọc, tốc độ di chuyển và margin. Mô hình để bàn không được quảng bá như thiết bị hỗ trợ lái xe thật.

Lỗi thường gặp

Không timeout; coi timeout là rất xa; gọi đo liên tục; beep bằng `delay()`; không có hysteresis; tin hai chữ số thập phân; lắp sensor lệch; dùng một vật phẳng duy nhất để "hiệu chuẩn". Mỗi lỗi làm demo đẹp nhưng hệ kém trung thực.

23.15 Kế hoạch thử nghiệm

Lập ma trận khoảng cách 10, 20, 50, 100, 200 cm với bốn vật liệu và ba góc. Ở mỗi ô, thu ít nhất 30 mẫu, ghi tỷ lệ hợp lệ, median, độ phân tán. Sau đó thử động: tiến vật với tốc độ

lặp lại, quay video kèm thước để đo thời điểm đổi vùng. Cuối cùng rút Echo/Trigger, che sensor và reset MCU trong mỗi vùng.

Tiêu chí chấp nhận ví dụ: không treo quá 30 ms do một phép đo; không báo FAR sau chuỗi mất echo; vùng ổn định khi vật đứng; STOP xuất hiện trước vạch margin ở tốc độ thử; nút mute chỉ tắt âm, không tắt LED lỗi.

23.16 Bài tập luyện tập

1. Tính timeout cho khoảng cách tối đa 2 m, thêm margin hợp lý.
2. So sánh median-5 và EMA trên một chuỗi có hai outlier.
3. Cài bộ xác nhận vùng hai mẫu và đo độ trễ phát sinh.
4. Thêm cảm biến nhiệt độ để bù tốc độ âm, rồi đánh giá cải thiện thực tế.
5. Tạo scheduler đo hai sensor luân phiên, không phát cùng lúc.
6. Thiết kế giao thức Serial xuất distance, valid, zone, age và error count.
7. Viết 15 ca kiểm thử hàm `updateZone()` ở đúng các biên hysteresis.
8. Đề xuất trạng thái giao diện cho NO_ECHO ngắn và FAULT kéo dài.

23.17 Tóm tắt cuối chương

Bản đồ kiến thức

Đo siêu âm là phép đo thời gian bay có timeout và giới hạn vật lý. Timeout không đồng nghĩa vật ở rất xa. Lọc/outlier và xác nhận vùng làm phản hồi ổn định nhưng tạo trễ cần tính. Hysteresis ngăn rung ở biên. Còi phải chạy không chặn; trạng thái UNKNOWN phải khác FAR; bố trí cơ khí và ma trận thử nghiệm quyết định chất lượng thật.

Góc nhìn cá nhân

Cảm biến không nói “vật cách 43 cm”; nó đưa cho ta một dấu vết điện có thể là tiếng vọng. Kỹ thuật bắt đầu khi ta giữ được sự khác nhau giữa điều đã đo, điều đã suy ra và điều hoàn toàn chưa biết.

Chương 24

Trạm môi trường mini – Khi nhiều cảm biến phải cùng kể một câu chuyện

Câu chuyện mở đầu

Một cảm biến nhiệt độ in số lên Serial là demo. Một trạm môi trường chạy nhiều ngày phải biết lúc nào đo, dữ liệu nào còn mới, hiển thị ra sao, ghi bản ghi nào, làm gì khi một cảm biến mất, và tại sao hai giá trị đặt cạnh nhau chưa chắc cùng đại diện cho một thời điểm.

Dự án này xây trạm để bàn gồm cảm biến nhiệt độ/độ ẩm số, cảm biến ánh sáng analog, OLED I2C, nút đổi trang và ghi CSV qua Serial. Ta không trói vào một mã module cụ thể: thư viện và địa chỉ phải chọn theo phần cứng thật. Điều quan trọng là kiến trúc thu thập dữ liệu có timestamp, quality và freshness.

Mục tiêu học tập

- Lập lịch nhiều tác vụ có chu kỳ khác nhau mà không dùng `delay()`.
- Biểu diễn mỗi phép đo bằng value, timestamp và trạng thái chất lượng.
- Thiết kế pipeline acquire–validate–derive–present–log.
- Xử lý sensor lỗi cục bộ thay vì làm cả hệ thống sụp.
- Viết log có schema, version và khả năng phân tích lại.

24.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Nhiệt độ, độ ẩm, ánh sáng và chất lượng không khí có chu kỳ, đơn vị và thời gian ổn định khác nhau. Gom tất cả đọc mỗi loop tạo dữ liệu thừa và làm nóng cảm biến. Trạm cần scheduler, timestamp, validation, trạng thái cảm biến và định dạng log có schema.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **trạm môi trường như hệ thống đa cảm biến với đồng bộ thời gian và chất lượng dữ liệu**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

24.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: chu kỳ từng cảm biến, timestamp, đơn vị, quality flag, hiệu chuẩn, lưu trữ và chu kỳ hiển thị. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$z_i = \frac{x_i - \mu_i}{\sigma_i}, \qquad r_t = (t, x_1, \ldots, x_m, q_1, \ldots, q_m)$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

24.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

24.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là mẫu mới nhất từng kênh, tuổi mẫu, lỗi liên tiếp và hàng đợi ghi. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k, x_k, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Mỗi bản ghi nói rõ thời điểm/đơn vị/chất lượng; dữ liệu cũ hoặc cảm biến lỗi không được trình bày như số đo mới hợp lệ.

24.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Thiết kế trạm bốn kênh: scheduler khác chu kỳ, validate miền, giữ last-known-good kèm tuổi, ghi CSV và hiển thị cảnh báo khi dữ liệu stale.

24.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

24.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

24.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

24.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

24.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

24.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

24.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

24.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Ghi số không timestamp.
- Trộn chu kỳ cảm biến.
- Thay lỗi bằng 0 hợp lệ.
- Không kiểm ảnh hưởng nhiệt từ board.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

24.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Lập sampling schedule.
2. Thiết kế schema record.
3. Thêm quality flag.
4. Phát hiện stale data.
5. Kiểm thử một cảm biến mất kết nối.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô

và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

24.7 Yêu cầu và ngân sách thời gian

Ta chọn nhịp minh họa:

Tác vụ	Chu kỳ	Lý do
Đọc nút	10 ms	phản hồi giao diện, debounce
Đọc ánh sáng	200 ms	thay đổi vừa, lọc được
Đọc nhiệt/ẩm	2 s	tôn trọng giới hạn sensor
Cập nhật OLED	250 ms	đủ mượt, không chiếm bus quá mức
Ghi log	10 s	dữ liệu gọn cho xu hướng chậm
Health check	1 s	phát hiện stale/lỗi liên tiếp

Chu kỳ đọc thật phải theo datasheet. “Loop chạy nhanh” không có nghĩa mọi sensor được phép đọc nhanh. Một số thư viện tự chờ hoặc trả lại mẫu cũ; phải đọc tài liệu và đo runtime.

24.8 Mô hình dữ liệu biết nói thật

Đừng lưu nhiệt độ chỉ bằng một `float`. Ta cần biết nó còn dùng được không.

Listing 24.1: Một phép đo có giá trị, tuổi và chất lượng

```
enum class Quality : byte { UNKNOWN, GOOD, SUSPECT, FAILED };

struct Reading {
  float value = 0.0f;
  unsigned long sampledAt = 0;
  Quality quality = Quality::UNKNOWN;
  byte consecutiveErrors = 0;
};

bool isFresh(const Reading& r, unsigned long now,
             unsigned long maxAge) {
  return r.quality == Quality::GOOD &&
         (unsigned long)(now - r.sampledAt) <= maxAge;
}
```

Nếu đọc thất bại một lần, có thể giữ giá trị cuối nhưng đổi quality thành `SUSPECT`; UI hiển thị ký hiệu và log không giả nó là mẫu mới. Sau ngưỡng lỗi, `FAILED`. Khi hồi phục, yêu cầu vài mẫu liên tiếp hợp lệ trước `GOOD`.

24.9 Pipeline năm tầng

24.9.1 Acquire

Driver nói chuyện với sensor và trả raw/status. Tầng này biết I2C, ADC, CRC nhưng chưa biết "phòng quá nóng".

24.9.2 Validate

Kiểm tra mã trả về, NaN, range vật lý và tốc độ thay đổi hợp lý. Range rộng để bắt lỗi dây, không dùng như bộ lọc tùy tiện. Nhiệt độ phòng 150°C là lỗi rõ; tăng 3°C trong một giây có thể là sensor vừa bị chạm tay, nên cần ngữ cảnh.

24.9.3 Derive

Từ nhiệt độ và độ ẩm có thể tính đại lượng dẫn xuất nếu công thức/phạm vi phù hợp, hoặc đơn giản tính min/max, EMA và xu hướng. Dữ liệu dẫn xuất phải mang quality của các đầu vào; một đầu vào stale thì kết quả không thể là GOOD.

24.9.4 Present

OLED hiển thị trang tổng quan, trang chi tiết và health. Dùng dấu gạch/biểu tượng lỗi khi thiếu dữ liệu; không in `0.0` vì số không có thể bị hiểu là phép đo thật.

24.9.5 Log

CSV gồm schema version, uptime, mỗi value, quality và age. Header cố định giúp Python/pandas đọc lại. Tách log event (boot, sensor fail/recover, đổi cấu hình) khỏi log định kỳ.

24.10 Một scheduler nhỏ, dễ nhìn

Listing 24.2: Lập lịch theo thời điểm đến hạn

```
struct TaskClock {
  unsigned long last = 0;
  unsigned long period;

  bool due(unsigned long now) {
    if ((unsigned long)(now - last) < period) return false;
    last += period;
    if ((unsigned long)(now - last) >= period) last = now;
    return true;
  }
};

TaskClock lightClock{0, 200};
TaskClock climateClock{0, 2000};
```

```
TaskClock displayClock{0, 250};
TaskClock logClock{0, 10000};

void loop() {
  unsigned long now = millis();
  updateButton(now);
  if (lightClock.due(now))   acquireLight(now);
  if (climateClock.due(now)) acquireClimate(now);
  updateHealth(now);
  if (displayClock.due(now)) renderPage(now);
  if (logClock.due(now))     writeCsv(now);
}
```

Cách cộng `last += period` giảm drift khi tác vụ hơi trễ. Nhánh bắt kịp đặt `last = now` tránh chạy dồn vô hạn sau khi bị chặn lâu. Scheduler này không bảo đảm deadline cứng; hãy đo thời gian tệ nhất của từng tác vụ và tổng tải.

24.11 I2C khi có hơn một thiết bị

OLED và sensor có thể dùng chung SDA/SCL nếu địa chỉ không xung đột, điện áp tương thích và pull-up phù hợp. Quét địa chỉ chỉ là chẩn đoán, không thay datasheet. Dây ngắn, ground tốt, tốc độ bus hợp lý; breadboard/dây dài làm cạnh tín hiệu xấu.

Khi giao dịch I2C treo hoặc NACK, driver phải trả lỗi. Không để UI dừng mãi vì một sensor. Có thể retry hữu hạn với backoff, reset riêng driver/bus nếu nền tảng hỗ trợ, rồi đánh dấu thiết bị thất bại. Log thời lượng giao dịch để phát hiện thư viện blocking bất thường.

24.12 Ánh sáng: từ ADC đến chỉ số ổn định

LDR trong cầu chia áp cho raw phụ thuộc điện trở cố định, Vcc và đặc tuyến phi tuyến. Nếu chỉ cần mức tương đối, hiệu chuẩn `darkRaw`/`brightRaw`, lọc median/EMA rồi đổi thành 0–100%. Đừng gọi nó là lux. Muốn lux, dùng sensor/quang học và quy trình hiệu chuẩn thích hợp.

Do OLED và MCU có thể làm nhiễu nguồn ADC, lấy mẫu ở nhịp cố định, bố trí tụ/ground và so sánh khi OLED bật/tắt. Median loại xung; EMA tạo xu hướng. Lưu cả raw trong log giai đoạn phát triển để không mất bằng chứng.

24.13 Thiết kế màn hình nhỏ

Màn 128x64 không phải nơi nhét mọi số. Trang 1: nhiệt độ, độ ẩm, ánh sáng và biểu tượng health. Trang 2: min/max từ lúc boot và tuổi mẫu. Trang 3: số lỗi, uptime, địa chỉ I2C/version firmware. Nút nhấn đổi trang bằng sự kiện debounce; nhấn giữ có thể reset min/max sau xác nhận.

Chỉ vẽ lại vùng thay đổi hoặc dùng buffer thư viện đúng cách. Theo dõi thời gian render; nếu một frame mất 30 ms, nó ảnh hưởng nhịp khác. UI ưu tiên thông tin hành động được: dấu lỗi rõ hơn một đồ thị tí hon khó đọc.

24.14 CSV có hợp đồng

Ví dụ header:

Listing 24.3: Một bản ghi tự mô tả đủ để phân tích

```
# envstation schema=1 firmware=1.0.0
uptime_ms,temp_c,temp_q,temp_age_ms,rh_pct,rh_q,
light_pct,light_q,page,health
10000,27.31,G,214,63.2,G,58.4,G,0,OK
```

Trong output thật, header ở một dòng; listing xuống dòng để vừa trang. Chọn dấu chấm thập phân, không thêm đơn vị trong ô, không đổi thứ tự âm thầm. Nếu schema đổi, tăng version. Serial có buffer hữu hạn; viết một dòng gọn theo chu kỳ, tránh nối nhiều đối tượng `String` gây phân mảnh RAM trên AVR.

24.15 Min/max, reset và mất điện

Min/max từ boot dễ làm, nhưng một outlier có thể phá cả ngày. Chỉ cập nhật từ mẫu GOOD sau validate. Nếu muốn min/max theo ngày, cần thời gian thực đáng tin hoặc máy tính gắn timestamp khi nhận log. EEPROM không nên ghi mỗi mẫu; lưu cấu hình khi người dùng đổi, với wear reduction và CRC.

Sau reset, health bắt đầu UNKNOWN cho đến khi có mẫu. UI không tái sử dụng số ngẫu nhiên/cũ. In boot reason/version để phân biệt mất nguồn với lỗi sensor.

24.16 Kịch bản lỗi

Sự kiện	Hành vi mong đợi	Điều cần quan sát
Rút sensor khí hậu	ánh sáng/UI vẫn chạy	quality, age, event log
OLED mất kết nối	thu/log vẫn chạy	không block, mã health
Serial không đọc	loop không treo dài	buffer/tần suất log
ADC chập rail	light FAILED	raw, range counter
`millis()` tràn	scheduler tiếp tục	khoảng giữa các mẫu
Reset giữa giao dịch	boot sạch	không dữ liệu giả GOOD

24.17 Thử nghiệm dài hạn

Một giờ không đại diện một tuần. Chạy soak test 24–72 giờ ở bàn an toàn: lưu log trên máy tính, theo dõi reset, gap timestamp, tuổi mẫu tối đa, RAM nếu có công cụ, số lỗi bus và drift. Tạo heat nhẹ bằng thay đổi môi trường an toàn, che/mở ánh sáng, rút/cắm sensor khi đã tắt nguồn theo yêu cầu phần cứng.

Phân tích log bằng Python: vẽ value và quality theo thời gian, histogram khoảng cách giữa bản ghi, tìm duplicate/gap, đánh dấu event. Đây là cây cầu tự nhiên giữa sách Python Tập 2 và Arduino: board thu nhận có kỷ luật, Python kiểm tra và trực quan hóa.

24.18 Bài tập luyện tập

1. Viết lớp `Reading` tổng quát hơn có unit và source ID mà không tốn RAM quá mức.
2. Đo thời gian tệ nhất của từng tác vụ bằng `micros()` và lập utilization budget.
3. Thêm quality SUSPECT sau một lỗi, FAILED sau năm lỗi, hồi phục sau ba mẫu tốt.
4. Viết CSV parser Python kiểm tra schema, gap và value ngoài range.
5. Tạo ba trang OLED; bảo đảm mất sensor không làm nhấp nháy cả màn.
6. Mô phỏng `millis()` tràn trong hàm scheduler độc lập.
7. So sánh log 200 ms và 10 s về dung lượng, thông tin và nguy cơ blocking.
8. Viết báo cáo soak test gồm tiêu chí, đồ thị, lỗi thấy được và thay đổi thiết kế.

24.19 Tóm tắt cuối chương

Bản đồ kiến thức

Trạm nhiều cảm biến cần scheduler và mô hình dữ liệu có value, timestamp, quality. Pipeline tách thu, kiểm, suy ra, hiển thị và log. Lỗi một thiết bị phải được cô lập. CSV cần schema/version; UI phải phân biệt số thật với dữ liệu thiếu. Kiểm thử dài hạn và phân tích log mới cho thấy gap, drift và lỗi phục hồi.

Góc nhìn cá nhân

Một trạm tốt không phải nơi có thật nhiều con số. Nó là người kể chuyện trung thực: biết mẫu nào vừa được đo, mẫu nào đã cũ, chương nào bị mất và chỗ nào ta chỉ đang suy đoán.

Chương 25

Robot dò đường – Điều khiển là cuộc đối thoại liên tục

Câu chuyện mở đầu

Robot dò vạch thường xuất hiện trong video như một chiếc xe "biết nhìn". Thực ra nó lặp một cuộc đối thoại: phát ánh sáng xuống mặt đường, đọc phản xạ, ước lượng vạch nằm lệch bên nào, điều chỉnh hai bánh, rồi quan sát hậu quả. Nếu vòng đối thoại quá chậm, nhiễu hoặc bão hòa, robot lắc, bay khỏi đường hoặc quay tại chỗ.

Dự án này đi từ thử motor trên giá kê bánh đến điều khiển tỷ lệ/PD, tìm lại đường và test pin yếu. Nó chỉ dùng robot mini điện áp thấp trong khu vực chắn an toàn; không biến board học tập thành phương tiện chở người.

Mục tiêu học tập

- Hiệu chuẩn mảng cảm biến và ước lượng vị trí vạch có trọng số.
- Điều khiển hai motor qua driver, không cấp từ chân Arduino.
- Xây vòng điều khiển theo chu kỳ cố định, tránh `delay()`.
- Điều chỉnh P/PD có hệ thống, hiểu bão hòa và giới hạn tốc độ.
- Thiết kế trạng thái LOST/SEARCH/STOP và quy trình test an toàn.

25.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Robot không chỉ đọc trái/phải rồi quay. Cảm biến tạo ước lượng vị trí vạch; controller biến sai lệch thành chênh tốc hai bánh; động học và ma sát tạo phản hồi mới. Chu kỳ đủ nhanh, hiệu chuẩn cảm biến và giới hạn motor quyết định độ ổn định.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **robot dò đường như vòng điều khiển phản hồi theo sai lệch vị trí**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

25.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: giá trị cảm biến, trọng số vị trí, error, tốc độ nền, correction, saturation và chu kỳ điều khiển. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$e = \frac{\sum_i w_i s_i}{\sum_i s_i}, \qquad u = K_p e + K_d \frac{\Delta e}{\Delta t}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

25.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

Hiện tượng $\longrightarrow$ cảm biến/mạch vào $\longrightarrow$ ADC hoặc chân số $\longrightarrow$ thuật toán $\longrightarrow$ mạch công suất $\longrightarrow$ tác động.

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

25.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là error hiện tại/trước, mode tìm vạch và lệnh motor. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k, x_k, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Lệnh motor nằm trong giới hạn; mất vạch kích hoạt chiến lược phục hồi thay vì dùng phép chia hoặc error không hợp lệ.

25.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Hiệu chuẩn dải đen/trắng từng cảm biến, tính vị trí trọng số, bắt đầu P, thêm D để giảm dao động, log error và motor command trên nhiều đoạn cua.

25.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

25.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

25.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

25.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

25.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

25.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

25.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

25.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Đặt ngưỡng giống nhau cho mọi sensor.
- Tăng Kp khi lỗi do chu kỳ chậm.
- Không giới hạn lệnh motor.
- Mất vạch nhưng vẫn dùng error cũ vô hạn.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

25.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Vẽ control loop.
2. Hiệu chuẩn mảng sensor.
3. Tính weighted position.
4. Tune P rồi PD.
5. Thiết kế recovery khi mất line.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô

và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

25.7 Kiến trúc năng lượng và cơ khí

Hai motor lấy nguồn qua driver H-bridge có rating dòng chạy/kẹt phù hợp. Arduino phát logic PWM/direction; nguồn motor không đi qua chân 5 V board. Ground logic/driver chung theo sơ đồ nhà sản xuất, tụ chống nhiễu và dây motor bố trí xa sensor. Pin phải có bảo vệ, cầu chì hoặc current limit, công tắc dễ với tới.

Khung xe cân, bánh không trượt trên trục, sensor song song mặt sàn ở độ cao ổn định. Sai cơ khí tạo "bug" phần mềm: một motor ma sát lớn hơn khiến cùng PWM nhưng tốc độ khác. Trước control, cho từng bánh chạy tiến/lùi khi robot được kê chắc, xác nhận chiều và đo ngưỡng PWM bắt đầu quay.

Kê bánh trước, chạy chậm sau

Mọi test driver đầu tiên thực hiện khi bánh không chạm bàn. Có công tắc ngắt nguồn motor, khu vực trống và giới hạn PWM thấp. Không chạm bánh/trục đang quay; buộc dây gọn để không cuốn vào cơ cấu.

25.8 Cảm biến phản xạ và hiệu chuẩn

Mảng năm sensor đặt từ trái sang phải. Trên nền và vạch, raw có thể tăng hoặc giảm tùy module. Quét robot bằng tay qua cả hai bề mặt, ghi min/max từng kênh. Chuẩn hóa riêng từng sensor:

$$n_i = 1000 \ \mathrm{clamp}\left(\frac{x_i - min_i}{max_i - min_i}, 0, 1\right).$$

Nếu vạch cho giá trị thấp, đảo $n_i \leftarrow 1000 - n_i$. Không dùng một min/max chung: LED/photransistor và góc lắp khác nhau. Mẫu có biên quá nhỏ báo sensor bẩn, độ cao sai hoặc hiệu chuẩn thiếu.

Có thể bật emitter theo nhóm nếu phần cứng hỗ trợ, đo on/off để trừ ánh sáng môi trường. Che robot khỏi ánh nắng trực tiếp trong bài đầu, rồi chủ động test thay đổi ánh sáng sau khi logic ổn.

25.9 Ước lượng vị trí bằng trọng số

Gán vị trí $p_i = -2000, -1000, 0, 1000, 2000$. Với cường độ vạch n_i:

$$position = \frac{\sum_i n_i p_i}{\sum_i n_i}.$$

Error là vị trí mong muốn 0 trừ position, hoặc ngược lại tùy quy ước. Kiểm tra dấu bằng tay: vạch ở bên trái phải làm robot quay trái. Nếu tổng cường độ dưới ngưỡng, vạch mất; không chia cho số gần 0 và không giả position bằng 0.

Listing 25.1: Ước lượng vị trí và phát hiện mất vạch

```
struct LineEstimate {
  bool seen;
  float position;
};

LineEstimate estimateLine(const int level[5]) {
  const int weight[5] = {-2000, -1000, 0, 1000, 2000};
  long weighted = 0;
  long total = 0;
  for (byte i = 0; i < 5; ++i) {
    weighted += (long)level[i] * weight[i];
    total += level[i];
  }
  if (total < 350) return {false, 0.0f};
  return {true, (float)weighted / (float)total};
}
```

Ngưỡng `total` phải lấy từ dữ liệu. Giao lộ hoặc vạch rất rộng cho tổng lớn và nhiều sensor cùng thấy; có thể là một event khác, không chỉ position.

25.10 Từ sai lệch đến tốc độ bánh

Với base speed B và correction u:

$$left = B - u, \qquad right = B + u.$$

Nếu dấu sai, robot rời vạch nhanh hơn. Bắt đầu nhấc bánh hoặc base rất thấp. Điều khiển P dùng $u = K_p e$. K_p thấp quay chậm, cao gây lắc. Thành phần D phản ứng tốc độ thay đổi:

$$u = K_p e + K_d \frac{e - e_{prev}}{\Delta t}.$$

Vì noise bị khuếch đại bởi đạo hàm, chu kỳ điều khiển phải tương đối ổn định và có thể lọc D. Đừng thêm I ngay: line follower thường không cần integral trong bản đầu, và windup khi mất vạch gây khó phục hồi.

Listing 25.2: Bước điều khiển PD theo chu kỳ đo được

```
float previousError = 0.0f;

void controlStep(float position, float dtSeconds) {
  const float error = -position;
```

```
  const float derivative = (error - previousError) / dtSeconds;
  previousError = error;

  float correction = KP * error + KD * derivative;
  correction = constrain(correction, -MAX_CORR, MAX_CORR);

  int left = constrain((int)(BASE - correction), -255, 255);
  int right = constrain((int)(BASE + correction), -255, 255);
  setMotorSpeeds(left, right);
}
```

`setMotorSpeeds()` chịu trách nhiệm direction/PWM và deadband. Khi đổi chiều H-bridge, có thể cần tắt ngắn theo datasheet để tránh shoot-through; dùng driver có bảo vệ phù hợp.

25.11 Chu kỳ vòng điều khiển

Chọn 5–10 ms tùy sensor, MCU và cơ khí. Scheduler gọi đúng nhịp; đo Δt thực bằng `micros()` thay vì giả vĩnh viễn. Log không được in hàng dài trong vòng control; lưu buffer nhỏ hoặc log decimated ở 10–20 Hz.

Ngân sách một bước gồm đọc 5 sensor, chuẩn hóa, ước lượng, PD và cập nhật PWM. Nếu tổng gần chu kỳ, UI/Serial sẽ gây jitter. Đo worst case, không chỉ average.

25.12 Bù lệch motor

Ở error 0, robot có thể vẫn cong vì motor/bánh khác nhau. Có thể đặt base trái/phải khác nhau từ bài test đường thẳng, hoặc dùng encoder cho điều khiển tốc độ vòng trong. Không "bù" bằng sửa sensor center nếu nguyên nhân là actuator; như vậy robot chỉ thẳng ở một mức pin/tốc độ.

Điện áp pin giảm làm cùng PWM cho tốc độ khác. Đo pin qua divider đúng dải và cảnh báo low battery; tốt hơn nữa là điều khiển tốc độ bằng encoder. Với sách này, giảm base khi pin thấp và dừng dưới ngưỡng an toàn của pack là đủ để học.

25.13 Mất vạch và tìm lại

Khi `seen=false`, PD không có error hợp lệ. Ta nhớ dấu error cuối:

- dưới 100 ms: giữ correction hạn chế để vượt khe nhỏ;
- sau đó vào `SEARCH`, quay chậm về phía vạch cuối thấy;
- nếu tìm quá 2 s hoặc quay quá phạm vi: `STOP_FAULT`;
- thấy vạch ổn định ba mẫu: trở lại FOLLOW với derivative được reset.

Reset `previousError` khi vào lại tránh derivative kick. Search phải có timeout; nếu không robot có thể quay mãi rơi khỏi bàn. Đường thử nên đặt trên sàn hoặc bàn có gờ, không ở mép cao.

25.14 Giao lộ, vạch đích và ngữ cảnh

Nhiều sensor cùng mạnh có thể là giao lộ, vạch ngang hoặc cua rộng. Chỉ một snapshot không phân biệt. Dùng thời gian/chuỗi: vạch ngang làm tổng lớn trong quãng ngắn; giao lộ có pattern đặc trưng. Trong bài cơ bản, đặt vạch đích rộng, yêu cầu tổng lớn liên tiếp 50 ms rồi dừng. Ghi rõ giới hạn để không gọi thuật toán "nhận mọi giao lộ".

25.15 Quy trình chỉnh P/PD

1. Tắt D, base thấp, tăng K_p từ nhỏ đến khi robot bám nhưng bắt đầu dao động.
2. Giảm nhẹ K_p, thêm K_d nhỏ để giảm lắc và overshoot.
3. Thử đoạn thẳng, cua rộng, cua gắt; không tune chỉ một đoạn.
4. Tăng base từng bước; mỗi tốc độ có thể cần gain khác.
5. Test pin đầy/gần ngưỡng thấp, ánh sáng khác, mặt đường bẩn.
6. Ghi gain, firmware, đường thử và kết quả; đừng tune bằng ký ức.

Đồ thị error/correction cho biết bão hòa bao lâu, jitter và dao động. Nếu correction liên tục chạm giới hạn, tăng gain không giúp; cần giảm base, cải thiện sensor/cơ khí hoặc thiết kế đường.

25.16 Ma trận kiểm thử

Điều kiện	Chỉ số	Tiêu chí ví dụ
Đường thẳng 3 m	thời gian/lệch	không rời vạch, ít dao động
Cua ba bán kính	tỷ lệ hoàn thành	10/10 ở base đã công bố
Khe vạch 3 cm	phục hồi	không vào search dài
Mất vạch thật	fail-safe	dừng trước 2,5 s
Pin thấp	ổn định	cảnh báo và giảm tốc/dừng
Sensor che/bẩn	detection	không quay vô hạn

25.17 Bài tập luyện tập

1. Thu min/max từng sensor và in thanh bar Serial để nhìn độ tương phản.
2. Viết test hàm position cho vạch trái, giữa, phải, hai sensor và mất vạch.

3. Đo chu kỳ control thực; vẽ histogram jitter bằng Python.
4. Tune P theo quy trình, ghi lại bằng bảng thay vì nhận xét “tốt hơn”.
5. Thêm D có lọc bậc một và so sánh error RMS/độ ổn định.
6. Cài state FOLLOW–SEARCH–STOP, test khe nhỏ và mất đường.
7. Thêm vạch đích rộng với xác nhận theo thời gian.
8. Lập power budget motor stall, driver, Arduino và sensor; chọn cầu chì/nguồn phù hợp.
9. Đề xuất vòng tốc độ encoder bên trong và vòng position bên ngoài.

25.18 Tóm tắt cuối chương

Bản đồ kiến thức

Robot dò đường là vòng quan sát–ước lượng–điều khiển lặp theo chu kỳ. Sensor phải được hiệu chuẩn riêng; vị trí trọng số chỉ hợp lệ khi đủ tín hiệu. P/PD biến error thành chênh tốc hai bánh nhưng chịu bão hòa, noise và cơ khí. Mất vạch là trạng thái riêng có tìm kiếm hữu hạn rồi dừng. Tune phải dựa dữ liệu và ma trận đường thử.

Góc nhìn cá nhân

Khoảnh khắc robot bám được vạch rất vui, nhưng bài học sâu hơn nằm ở những lần nó lắc và lao ra ngoài. Mỗi thất bại đang nói về độ trễ, dấu, năng lượng hay một giả định bị giấu. Nghe được lời đó chính là học điều khiển.

Chương 26

Đồ án tốt nghiệp trong chăn – Tự biến một mong muốn thành hệ thống có bằng chứng

Lời mời cuối cùng

Đến đây, bạn đã bật đèn, đọc sensor, lái actuator, bỏ `delay()`, viết máy trạng thái, giao tiếp bus, lưu dữ liệu và phân tích lỗi. Đồ án cuối không nhằm dùng tất cả linh kiện. Nó nhằm chứng minh bạn có thể chọn một vấn đề vừa sức, viết yêu cầu đo được, chia rủi ro, xây từng lát, kiểm thử và kể lại trung thực.

Bạn có thể chọn một trong bốn gợi ý: hộp nhắc nghỉ mắt, tủ ươm mini chỉ giám sát, bộ đếm sản phẩm mô hình, hoặc tác phẩm tương tác ánh sáng–âm thanh. Bạn cũng có thể tự đề xuất, miễn chỉ dùng điện áp thấp và không điều khiển quá trình mà lỗi có thể gây thương tích, cháy, ngập, khóa người hay hư hại lớn.

Mục tiêu học tập

- Viết project brief, yêu cầu chức năng/phi chức năng và ranh giới.
- Thiết kế kiến trúc, interface, state machine và ngân sách tài nguyên.
- Lập kế hoạch theo vertical slice cùng tiêu chí hoàn thành.
- Tạo traceability từ yêu cầu đến test và bằng chứng.
- Trình bày demo, báo cáo lỗi còn lại và hướng phát triển trung thực.

26.1 Từ hiện tượng vật lý đến một hệ thống đo–quyết định–tác động

Hiện tượng cần hiểu trước khi nối dây

Một ý tưởng như hộp thuốc nhắc giờ chỉ có giá trị kỹ thuật khi yêu cầu đo được: sai số thời gian, độ lớn cảnh báo, thời lượng pin, phản ứng khi mất nguồn và tiêu chí chấp nhận. Đồ án là quá trình tích lũy bằng chứng rằng hệ thống đáp ứng yêu cầu trong các trường hợp bình thường và lỗi.

Một sketch không thể sửa một mạch điện được mô hình hóa sai. Với chủ đề **đồ án nhúng như chu trình yêu cầu–mô hình–prototype–xác minh**, ta luôn đi qua bốn miền: hiện tượng vật lý, tín hiệu điện, con số trong vi điều khiển và hành động của cơ cấu chấp hành. Mỗi lần chuyển miền đều có thể làm mất thông tin, đưa thêm nhiễu hoặc tạo độ trễ. Vì vậy, code chỉ là một mắt xích trong chuỗi bằng chứng.

26.1.1 Bước 1: gọi tên đại lượng và đơn vị

Các đại lượng cần theo dõi gồm: stakeholder, requirement, interface, ngân sách nguồn/thời gian, rủi ro, test case và artifact. Mỗi đại lượng phải có đơn vị, miền hợp lý và cách đo. Nếu một giá trị ADC là 620, con số đó chưa tự nói lên nhiệt độ, độ ẩm hay khoảng cách. Ta phải biết điện áp tham chiếu, đặc tuyến cảm biến, sai số và điều kiện môi trường đã tạo ra nó.

Mô hình tối thiểu dùng trong chương là:

$$\text{need} \rightarrow \text{requirement} \rightarrow \text{design} \rightarrow \text{test} \rightarrow \text{evidence}$$

Công thức không nhằm làm mạch trở nên hàn lâm. Nó giúp kiểm tra thứ nguyên và dự đoán trước hành vi. Nếu vế trái là volt mà vế phải cho ra ampere, mô hình đã sai trước khi nạp sketch. Nếu mô hình dự đoán dòng LED vượt giới hạn, không nên "thử xem có cháy không".

26.1.2 Bước 2: vẽ chuỗi tín hiệu

Ta theo dõi tín hiệu theo tuyến:

$$\text{Hiện tượng} \longrightarrow \text{cảm biến/mạch vào} \longrightarrow \text{ADC hoặc chân số} \longrightarrow \text{thuật toán} \longrightarrow \text{mạch công suất} \longrightarrow \text{tác động}.$$

Ở mỗi mũi tên, hãy ghi ba điều: miền giá trị, tần suất thay đổi và lỗi có thể xuất hiện. Một dây dài có thể bắt nhiễu; tiếp điểm cơ khí có thể dội; ADC lượng tử hóa; phép lọc tạo trễ; relay có thời gian đóng cắt; nguồn có thể sụt áp khi motor khởi động. Khi biết lỗi nằm ở đoạn nào, ta đo đúng điểm thay vì sửa code ngẫu nhiên.

26.1.3 Bước 3: trạng thái và thời gian

Hệ nhúng không chỉ tính một kết quả rồi kết thúc. Nó quan sát lặp lại trong thời gian. Trạng thái cần nhớ ở chương này là phiên bản thiết kế, yêu cầu đã xác minh, lỗi mở và quyết định kỹ thuật. Ta mô tả một chu kỳ bằng:

$$S_{k+1} = F(S_k,\, x_k,\, \Delta t),$$

trong đó x_k là mẫu đo mới và Δt là thời gian từ lần cập nhật trước. Nếu bỏ Δt, cùng một thuật toán có thể cư xử khác nhau khi vòng `loop()` chạy nhanh hoặc chậm. Đây là lý do thiết kế thời gian quan trọng ngang với logic đúng–sai.

Bất biến an toàn và chức năng

Mỗi yêu cầu có ít nhất một phép thử; mọi thay đổi thiết kế được truy vết đến lý do và không làm mất yêu cầu an toàn.

26.2 Thiết kế thí nghiệm trước khi thiết kế sản phẩm

Ca sử dụng dẫn đường

Xây hồ sơ đồ án hộp thuốc: user story, requirement định lượng, sơ đồ khối, pin map, state machine, prototype từng khối, test matrix, log lỗi và demo theo kịch bản.

26.2.1 1. Tách giả thuyết

Đừng thử cả hệ thống cùng lúc. Nếu mục tiêu là xác nhận cảm biến, hãy tạm bỏ relay, motor và giao diện. Nếu mục tiêu là xác nhận cơ cấu chấp hành, dùng một tín hiệu thử đã biết thay cho cảm biến. Mỗi thí nghiệm chỉ nên trả lời một câu hỏi. Cách tách này làm giảm số nguyên nhân khả dĩ khi kết quả sai.

26.2.2 2. Lập ngân sách điện và thời gian

Ngân sách điện ghi điện áp làm việc, dòng trung bình, dòng đỉnh và công suất tiêu tán. Ngân sách thời gian ghi chu kỳ lấy mẫu, thời gian xử lý, thời gian truyền và độ trễ chấp nhận được. Một hệ thống có thuật toán đúng nhưng nguồn sụt hoặc phản ứng trễ vẫn là hệ thống sai.

Hạng mục	Câu hỏi đo lường	Bằng chứng cần lưu
Nguồn	Điện áp có giữ trong giới hạn khi tải đổi không?	Số đo không tải, tải thường và tải đỉnh
Tín hiệu vào	Mức thấp/cao hoặc dải analog thực tế là bao nhiêu?	Min, max, nhiễu đỉnh–đỉnh và đơn vị
Thời gian	Mẫu đến nhanh đến mức nào, thuật toán trễ bao lâu?	Timestamp và chu kỳ quan sát
Đầu ra	Cơ cấu chấp hành có nhận đúng lệnh không?	Trạng thái lệnh và trạng thái thực
Lỗi	Khi dây rời hoặc cảm biến bão hòa, hệ thống làm gì?	Log lỗi và trạng thái an toàn

26.2.3 3. Đo đường cơ sở

Trước khi đặt ngưỡng, thu một dãy số đo khi điều kiện ổn định. Tính giá trị nhỏ nhất, lớn nhất, trung bình và độ dao động. Sau đó thay đổi đúng một yếu tố rồi đo lại. Ngưỡng phải được chọn từ phân bố quan sát và khoảng an toàn, không phải từ một con số đẹp nhìn thấy trong một lần đọc Serial Monitor.

26.2.4 4. Truy vết một chu kỳ

Với mỗi chu kỳ, ghi lần lượt thời điểm, số đo thô, số đo sau hiệu chuẩn, trạng thái quyết định và lệnh đầu ra. Nếu hệ thống có máy trạng thái, ghi cả trạng thái trước và sau. Dữ liệu này cho phép dựng lại quyết định. Không có truy vết, câu "thỉnh thoảng nó lỗi" gần như không thể kiểm chứng.

26.2.5 5. Thử lỗi có chủ đích

Rút cảm biến, tạo giá trị sát ngưỡng, làm tải thay đổi nhanh và khởi động lại giữa một chu kỳ. Các thử nghiệm này không phá hoại nếu được thiết kế trong giới hạn an toàn; chúng cho thấy hệ thống có về trạng thái dự đoán được hay không. Trong sản phẩm thật, đường đi khi lỗi quan trọng không kém đường đi bình thường.

26.3 Từ phép đo đến quyết định có độ tin cậy

Một mẫu đơn lẻ hiếm khi đủ. Ta cần cân bằng ba yếu tố: nhiễu, độ trễ và chi phí tính toán. Lọc mạnh làm số đo êm nhưng phản ứng chậm; lấy mẫu quá nhanh tạo nhiều dữ liệu tương quan và tốn thời gian; lấy mẫu quá chậm có thể bỏ lỡ sự kiện. Chu kỳ lấy mẫu phải xuất phát từ tốc độ thay đổi của hiện tượng, không từ tốc độ tối đa của vi điều khiển.

Quyết định cũng cần vùng bất định. Nếu ngưỡng là (T), việc bật khi (x>T) và tắt ngay khi (x<T) có thể làm đầu ra rung quanh (T). Hysteresis, xác nhận nhiều mẫu hoặc khoảng chờ tối thiểu là ba cách phổ biến để ổn định. Lựa chọn nào cũng phải được giải thích bằng đặc tính vật lý và yêu cầu sử dụng.

Khi điều khiển tải, phải phân biệt *lệnh* với *trạng thái thật*. Gọi `digitalWrite` chỉ cho biết phần mềm đã đặt bit đầu ra; nó không chứng minh motor quay, van mở hay relay đóng. Muốn khẳng định trạng thái thật, cần cảm biến phản hồi hoặc ít nhất phép đo điện phù hợp.

26.4 An toàn là một phần của thuật toán

An toàn bắt đầu từ giới hạn điện áp, dòng và nhiệt, nhưng không dừng ở đó. Phần mềm phải có trạng thái mặc định khi khởi động, timeout khi mất tín hiệu, giới hạn thời gian kích tải và cách phục hồi sau reset. Một đầu ra nguy hiểm không nên chờ chương trình chạy qua nhiều bước mới được đưa về mức an toàn.

Với nguồn ngoài, motor, relay hoặc tải cảm, phải dùng tầng công suất và bảo vệ phù hợp; không lấy dòng tải trực tiếp từ chân I/O. Nối chung mốc tham chiếu khi cần, kiểm tra cực tính và không thao tác mạch đang cấp nguồn nếu việc đó có thể tạo chập. Cuốn sách dùng mô hình điện áp thấp trên bàn học; mọi kết nối điện lưới cần thiết bị đạt chuẩn và người có chuyên môn.

26.5 Sai rồi sửa bằng phép đo

Những giả định dễ làm hệ thống sai

- Bắt đầu mua linh kiện trước khi chốt yêu cầu.
- Demo một lần được xem là kiểm thử.
- Không có baseline và tiêu chí pass/fail.
- Chỉ lưu bản code cuối mà mất lịch sử quyết định.

Quy tắc gỡ lỗi là đi từ năng lượng đến thông tin: kiểm tra nguồn và GND trước, kiểm tra tín hiệu điện sau, rồi mới kiểm tra giá trị số và logic. Nếu đảo thứ tự, ta có thể dành hàng giờ sửa một điều kiện `if` trong khi cảm biến chưa từng nhận đủ nguồn.

26.6 Bài tập ba tầng và hồ sơ bằng chứng

1. Viết năm requirement đo được.
2. Lập risk register.
3. Tạo verification matrix.
4. Thiết kế milestone theo rủi ro.
5. Viết báo cáo thất bại và bài học.

Mỗi bài phải kèm sơ đồ khối, bảng chân, công thức hoặc quy tắc chuyển đổi, ba ca thử và một đoạn mô tả trạng thái an toàn. Với bài có cảm biến, nộp thêm ít nhất mười mẫu thô

và giải thích cách chọn ngưỡng. Với bài có tải, nêu rõ nguồn cấp và tầng công suất. Đây là hồ sơ chứng minh hệ thống hoạt động, không chỉ là ảnh LED đang sáng.

26.7 Bước 1: Một trang mô tả đủ rõ

Project brief trả lời:

- **Người dùng**: ai tương tác, trong bối cảnh nào?
- **Nỗi khó**: điều gì hiện bất tiện hoặc dễ sai?
- **Kết quả**: hệ giúp quan sát/quyết định/hành động thế nào?
- **Không làm**: ba điều nằm ngoài phạm vi.
- **Ràng buộc**: board, nguồn, thời gian, ngân sách, môi trường.
- **Tiêu chí thành công**: con số hoặc hành vi có thể kiểm tra.

Ví dụ yếu: "làm hộp thông minh". Ví dụ tốt hơn: "mỗi 25 phút học, hộp phát ánh sáng dịu và beep ngắn; người dùng xác nhận nghỉ bằng nút; lịch vẫn phản hồi nút trong 50 ms; mất sensor ánh sáng không làm timer dừng; thiết bị chỉ dùng USB 5 V."

26.8 Bước 2: Yêu cầu có mã định danh

Đặt FR cho chức năng, NFR cho chất lượng, SAF cho an toàn:

ID	Yêu cầu	Cách chấp nhận
FR-01	Bắt đầu chu kỳ bằng một lần nhấn sạch	20/20 lần, không double event
FR-02	Báo khi hết 25 phút và giữ trạng thái chờ	clock giả nhanh + test thật
NFR-01	Phản hồi nút dưới 50 ms trong mọi mode	logic analyzer/log timestamp
NFR-02	Chạy liên tục 24 giờ không reset ngoài ý muốn	soak log không có boot event
SAF-01	Mọi output tắt khi boot hoặc state lỗi	đo chân trước/sau reset

Mỗi câu dùng "shall" theo tinh thần bắt buộc, tránh "thân thiện", "nhanh", "ổn định" mà không có thước đo. Nếu chưa biết ngưỡng, ghi giả định và lên thí nghiệm tìm nó.

26.9 Bước 3: Sơ đồ khối và interface

Vẽ nguồn, MCU, input, output và đường truyền. Với mỗi interface ghi voltage, direction, protocol, rate và failure. Lập pin map trước khi code:

Khối	Chân/bus	Điện/nhịp	Hành vi khi lỗi
Nút	D2 pull-up	active LOW, 10 ms	stuck không lặp vô hạn
LED RGB	D5/D6/D9	PWM, common cathode	tắt khi boot
Buzzer	D10	tone theo event	mute vẫn giữ visual
OLED	I2C	5 V/địa chỉ kiểm tra	timer vẫn chạy nếu mất

Kiểm tra xung đột timer/chân trong thư viện; trên AVR, `tone()`, Servo và PWM có thể dùng tài nguyên timer ảnh hưởng nhau. Đọc tài liệu core/thư viện phiên bản đang dùng và làm spike nhỏ trước khi khóa thiết kế.

26.10 Bước 4: State machine và data flow

Liệt kê mode bằng danh từ/trạng thái quan sát được: `IDLE`, `FOCUS`, `BREAK_DUE`, `BREAK`, `FAULT`. Với mỗi state ghi entry action, output, event được nhận, guard và next state. Tách event khỏi mức điện: `BUTTON_PRESSED`, `TIMER_EXPIRED`, `SENSOR_FAILED`.

Dòng dữ liệu đi acquire → validate → model → decision → actuator/UI → log. Mỗi bước có kiểu dữ liệu và quy ước đơn vị. Tên như `durationMs`, `temperatureC` giúp ngăn lỗi đơn vị.

26.11 Bước 5: Ngân sách

26.11.1 Thời gian

Liệt kê chu kỳ/deadline/worst-case của mỗi task. Tổng runtime định kỳ không được lấp hết CPU; để margin cho event, Serial và thư viện. Đo bằng chân debug hoặc `micros()`.

26.11.2 Bộ nhớ

Uno R3 có tài nguyên hữu hạn. Xem báo cáo compile flash/RAM, tránh `String` tăng giảm khó đoán, lưu chuỗi cố định trong flash khi phù hợp, giới hạn buffer và kiểm tra tràn. Không tối ưu mù; theo dõi số đo qua từng mốc.

26.11.3 Nguồn

Ghi dòng idle/typical/peak của từng tải, hiệu suất regulator, nhiệt và margin. Output cảm/động cơ có nguồn/driver riêng theo thiết kế. Xác định trạng thái chân trước `setup()` và khi reset.

26.11.4 Sai số và độ trễ

Với sensor, ghi range, resolution, accuracy thực tế, rate, filter delay. Với điều khiển, tính thời gian từ thế giới đổi đến actuator phản hồi. Một con số đẹp không thay ngân sách sai số.

26.12 Bước 6: Xây theo vertical slice

Đừng lắp toàn bộ rồi nạp một sketch 500 dòng. Sáu lát gợi ý:

1. **Bring-up**: boot, heartbeat, version, mọi output safe.
2. **Input slice**: một input thành event sạch và log được.
3. **State slice**: event làm chuyển hai state, output bằng LED.
4. **Timing slice**: scheduler/timer thật, vẫn phản hồi input.
5. **Peripheral slice**: thêm từng sensor/display với fault injection.
6. **Integration**: full state, logging, soak và enclosure.

Mỗi lát có Definition of Done: code build sạch, sơ đồ cập nhật, test qua, log/bằng chứng lưu, commit/tag nếu dùng Git. Một lát chưa test chưa "xong 90%".

26.13 Bước 7: Thiết kế code

Một cấu trúc vừa sức:

Listing 26.1: Khung vòng lặp của đồ án

```
void setup() {
  configureSafeOutputs();
  beginSerialAndReportVersion();
  beginInputs();
  beginPeripherals();
  runPowerOnSelfTest();
  enterState(State::IDLE, millis());
}

void loop() {
  const unsigned long nowMs = millis();
  const unsigned long nowUs = micros();

  acquireInputs(nowMs);
  validateReadings(nowMs);
  Event event;
  while (nextEvent(event)) dispatch(event, nowMs);
  runScheduledTasks(nowMs, nowUs);
  renderOutputs(nowMs);
```

```
20   monitorHealth(nowMs);
21 }
```

Giữ driver phần cứng, logic domain và UI/log tách vừa đủ. Không biến mọi thứ thành class nếu chưa giúp test/đọc. Hằng cấu hình ở một chỗ; pin map không rải. Error code là enum; log state transition có nguyên nhân.

26.14 Bước 8: Verification plan

Tạo ma trận yêu cầu–test:

Req	Test	Phương pháp	Bằng chứng
FR-01	T-BTN-03	20 nhấn với dội giả	event log, video
FR-02	T-TIME-02	clock tăng tốc + mốc biên	transition timestamps
NFR-01	T-LAT-01	pulse input và debug output	ảnh logic analyzer
NFR-02	T-SOAK-01	chạy 24 giờ	CSV, không boot gap
SAF-01	T-RST-04	reset ở mọi state	đo output LOW

Test happy path, boundary, invalid input, timing, reset/power cycle, disconnected peripheral và recovery. Fault injection có kiểm soát: dùng dữ liệu giả/driver stub trước; chỉ rút dây khi phần cứng cho phép và đã tắt nguồn nếu cần.

26.15 Bước 9: Quan sát và chẩn đoán

Firmware in banner gồm project, semantic version, build ID nếu có, reset cause và schema. Event log ngắn: timestamp, severity, module, code, data. Có heartbeat chậm hoặc health page; không dùng print dày làm thay đổi timing.

Đặt pin debug HIGH quanh task quan trọng để logic analyzer đo runtime/period. Đếm deadline miss, sensor error, queue high-water. Một hệ có quan sát giúp báo cáo bằng bằng chứng thay vì "em thấy chạy ổn".

26.16 Bước 10: Review an toàn và FMEA

Hỏi cho từng output: nếu mắc HIGH, mắc LOW, MCU reset, dây đứt, transistor chập, nguồn sụt thì hậu quả gì? Safe state được bảo đảm bằng pull-up/down, enable, fuse/current limit và giới hạn cơ khí/năng lượng khi cần. Watchdog chỉ là một lớp.

Nếu project liên quan nhiệt, nước, chuyển động, pin, vật nuôi hay người, giảm scope đến mức hậu quả chấp nhận được và luôn giám sát. Không dùng đồ án học tập làm thiết bị y tế, báo cháy, khóa an ninh, điều khiển xe hoặc tải điện lưới.

26.17 Bước 11: Báo cáo kỹ thuật

Báo cáo 12–20 trang chất lượng hơn 100 trang ảnh code. Cấu trúc:

1. vấn đề, người dùng, scope và tiêu chí;
2. yêu cầu, kiến trúc, pin/power budget;
3. thuật toán/state/data model và các lựa chọn;
4. cách triển khai theo lát, công cụ và version;
5. kế hoạch test, kết quả số liệu, failure/recovery;
6. giới hạn, rủi ro còn lại và hướng phát triển;
7. phụ lục sơ đồ, BOM, mã nguồn, log/schema.

Ảnh mạch có nhãn, sơ đồ nguồn rõ, đồ thị có đơn vị, bảng test có expected/actual/pass. Nêu lỗi gặp và cách tìm ra; đó là bằng chứng học tập, không phải điểm trừ.

26.18 Bước 12: Demo không phụ thuộc may mắn

Chuẩn bị kịch bản 5–7 phút: 30 giây vấn đề, 60 giây kiến trúc, 2 phút happy path, 1 phút lỗi và recovery, 1 phút số liệu test, phần còn lại giới hạn. Mang video dự phòng nhưng vẫn demo trực tiếp nếu có thể. Đánh dấu vị trí dây, sạc pin, mang cáp, không cập nhật thư viện phút cuối.

Một demo mạnh chủ động rút một sensor mô phỏng hoặc chọn nút "inject fault", cho thấy hệ không treo, UI báo lỗi và phục hồi theo thiết kế. Đừng tạo lỗi nguy hiểm với motor/nước trước khán giả.

26.19 Rubric tự chấm 100 điểm

Hạng mục	Điểm
Vấn đề, scope, yêu cầu đo được	12
Kiến trúc, pin/interface, nguồn	14
Thuật toán/state/data model	16
Chất lượng code và tổ chức cấu hình	12
Test, traceability và fault injection	18
An toàn, FMEA và recovery	12
Dữ liệu/log và bằng chứng định lượng	8
Báo cáo/demo/giới hạn trung thực	8
Tổng	100

Không có điểm riêng cho "nhiều linh kiện". Một project nhỏ được kiểm chứng kỹ tốt hơn một hệ phức tạp chỉ chạy khi tác giả đứng cạnh sửa dây.

26.20 Nhật ký 14 mốc

1. Chọn vấn đề và ranh giới an toàn.
2. Viết brief, phỏng vấn/thử ngữ cảnh nếu có.
3. Khóa yêu cầu phiên bản 0.1 và tiêu chí.
4. Thử rủi ro lớn nhất bằng spike.
5. Sơ đồ khối, pin map, power budget.
6. Bring-up và safe boot.
7. Input/event slice.
8. State/timing slice.
9. Peripheral từng cái cùng error path.
10. Tích hợp và đo timing/RAM.
11. Test matrix và fault injection.
12. Soak test, phân tích log, sửa.
13. Đóng enclosure/tài liệu, regression test.
14. Demo và retrospective.

Mỗi mốc ghi "đã nghĩ gì, đã đo gì, kết quả, quyết định tiếp theo". Nhật ký giữ lại đường suy luận mà code cuối không kể được.

26.21 Bộ câu hỏi phản biện cuối

1. Ba bất biến quan trọng nhất là gì, được bảo vệ ở đâu?
2. Dữ liệu nào có timestamp/quality, dữ liệu nào chưa?
3. Worst-case loop/task là bao nhiêu, đo bằng gì?
4. Output làm gì trong 100 ms đầu sau cấp nguồn?
5. Sensor mất, kẹt, trả số hợp lệ giả thì hệ phản ứng thế nào?
6. `millis()` tràn, pin yếu, Serial đầy có làm sai state không?
7. Có failure phần cứng nào phần mềm không thể cứu? Giới hạn hậu quả ra sao?
8. Test nào thất bại gần nhất và bạn học gì từ nó?
9. Người khác có thể dựng lại từ BOM, sơ đồ và README không?
10. Điều gì project tuyệt đối chưa được dùng để làm?

26.22 Bài tập – chính là đồ án

1. Nộp project brief một trang và bảng ít nhất 10 yêu cầu có ID.
2. Nộp block diagram, pin map, BOM, power/timing/memory budget.
3. Nộp state table/diagram và định nghĩa event/data quality.
4. Nộp repository hoặc gói mã nguồn có README và version phụ thuộc.
5. Nộp test matrix ít nhất 25 ca, trong đó có 8 lỗi/biên.
6. Nộp log soak tối thiểu 8 giờ và bản phân tích; mục tiêu tốt là 24 giờ.
7. Nộp FMEA ít nhất 10 failure mode và ba biện pháp ưu tiên.
8. Nộp video demo happy path, fault, recovery và safe shutdown.
9. Viết retrospective: ba quyết định đúng, ba sai, ba việc sẽ làm tiếp.

26.23 Tóm tắt cuối chương

Bản đồ kiến thức

Đồ án hoàn chỉnh đi từ brief đến yêu cầu có ID, kiến trúc, ngân sách, state/data model, vertical slice, verification và báo cáo. Traceability nối lời hứa với test/bằng chứng. Safe state, FMEA và giới hạn hậu quả thuộc thiết kế từ đầu. Giá trị không nằm ở số module mà ở tính tái tạo, đo được và trung thực.

Góc nhìn cá nhân

Tốt nghiệp không phải lúc bạn không còn gặp lỗi. Đó là lúc một lỗi xuất hiện, bạn biết cần quan sát ở đâu, giả thuyết nào đáng thử, bằng chứng nào đủ để kết luận và ranh giới nào phải giữ an toàn. Từ chiếc LED đầu tiên đến đây, điều ta thật sự xây là cách suy nghĩ ấy.

Phụ lục A

Bàn linh kiện và pinout – Tra cứu trước khi cắm

Phụ lục này là chiếc bàn kiểm nhanh, không thay datasheet. Cùng tên thương mại có thể tồn tại nhiều module, pinout và mức logic khác nhau. Luôn đọc chữ in trên chính board, đối chiếu sơ đồ nhà sản xuất và ngắt nguồn trước khi đổi dây.

A.1 Arduino Uno R3 nhìn theo vai trò

Uno R3 dùng ATmega328P, logic danh định 5 V, có 14 chân digital (6 chân hỗ trợ PWM theo ký hiệu trên board), 6 analog input, USB và jack nguồn. Giới hạn điện/dòng chi tiết phải lấy từ tài liệu chính thức và datasheet vi điều khiển; không thiết kế tải sát giới hạn tuyệt đối.

Nhóm	Chân quen thuộc	Ghi chú thiết kế
Digital	D0–D13	HIGH/LOW, input/output; kiểm tra xung đột shield/thư viện.
UART	D0 RX, D1 TX	Dùng cho USB-Serial trên Uno; thiết bị ngoài có thể cản upload/log.
Interrupt ngoài	D2, D3	Mapping theo board/core; dùng `digitalPinToInterrupt`.
PWM	D3, D5, D6, D9, D10, D11	Dấu ~ trên board; tần số phụ thuộc timer/core.
SPI	D10–D13, ICSP	SS, MOSI, MISO, SCK; ưu tiên header ICSP khi shield cần portability.
I2C	SDA/SCL, tương ứng A4/A5	Bus open-drain cần pull-up phù hợp; kiểm tra địa chỉ.
Analog	A0–A5	ADC; cũng có thể dùng digital theo core, nhưng ghi rõ ý định.
Reference	AREF	Chỉ dùng theo cấu hình analog reference đúng; nối sai có thể hỏng board.

Nhóm	Chân quen thuộc	Ghi chú thiết kế
Nguồn	5V, 3.3V, GND, Vin, IOREF, RESET	Không phải các chân tải công suất; hiểu đường nguồn/USB trước khi nối.
LED tích hợp	D13	Có tải/đường mạch sẵn; tiện bring-up nhưng không hoàn toàn "trống".

Absolute maximum không phải operating target

Giới hạn tuyệt đối là ranh giới hư hỏng, không phải mức nên dùng. Dòng mỗi chân, tổng theo port/chip, công suất regulator và nhiệt đều phải xét. Dùng transistor/MOSFET/driver cho tải, đặt margin và đo trong tình huống xấu.

A.2 Bảng chọn điện trở LED

Với nguồn logic V_S, sụt áp LED V_F và dòng mục tiêu I:

$$R \geq \frac{V_S - V_F}{I}, \qquad P_R = I^2 R.$$

Ví dụ 5 V, LED đỏ V_F khoảng 2 V, chọn 8 mA: $R \geq 375\,\Omega$; chọn giá trị chuẩn 390 Ω hoặc cao hơn. V_F thay đổi theo màu, linh kiện và nhiệt; dùng datasheet/đo. LED chỉ báo thường không cần 20 mA.

Nguồn	V_F giả định	Dòng mục tiêu	Điện trở tính/chọn
5 V	2,0 V	5 mA	600 Ω; chọn 620/680 Ω
5 V	2,0 V	8 mA	375 Ω; chọn 390 Ω
5 V	3,0 V	5 mA	400 Ω; chọn 470 Ω
3,3 V	2,0 V	5 mA	260 Ω; chọn 270/330 Ω
3,3 V	3,0 V	3 mA	100 Ω; cần kiểm tra độ sáng/margin

Không dùng bảng như chân lý vì V_F thật khác. Chọn dòng thấp trước, quan sát độ sáng rồi mới giảm R trong giới hạn.

A.3 Mã màu điện trở bốn vạch

Hai vạch đầu là chữ số, vạch ba là hệ số nhân, vạch bốn là sai số. Đen 0, nâu 1, đỏ 2, cam 3, vàng 4, lục 5, lam 6, tím 7, xám 8, trắng 9. Vàng kim nhân 0,1 và sai số 5%; bạc nhân 0,01 và sai số 10% theo vị trí. Khi không chắc chiều, dùng đồng hồ; màu trên điện trở nhỏ dễ nhầm.

Mẫu	Chữ số	Nhân	Kết quả
nâu–đen–đỏ–vàng kim	10	10^2	1 kΩ, 5%
đỏ–đỏ–nâu–vàng kim	22	10^1	220 Ω, 5%
vàng–tím–cam–vàng kim	47	10^3	47 kΩ, 5%
cam–cam–đen–vàng kim	33	10^0	33 Ω, 5%

A.4 Tụ điện theo công việc

- **Decoupling**: tụ gốm nhỏ đặt sát cặp Vcc–GND của IC, đường vòng ngắn.
- **Bulk**: tụ lớn hơn gần tải/rail để đỡ biến động chậm và dòng đỉnh.
- **Timing/filter**: giá trị, dung sai, leakage và loại điện môi ảnh hưởng đáp ứng.
- **Snubber**: chọn theo tải/transient; không sao chép mù giá trị.

Tụ điện phân có cực; cắm ngược nguy hiểm. Điện áp định mức phải cao hơn điện áp làm việc với margin. Tụ vẫn có thể tích năng lượng sau ngắt; trong dự án điện áp thấp nhỏ, thói quen kiểm tra/xả đúng cách vẫn cần được học.

A.5 Nút, công tắc và input không trôi

Input CMOS hở có thể đọc ngẫu nhiên. Dùng pull-up/pull-down. Với `INPUT_PULLUP`, nút nối GND, thả HIGH, nhấn LOW. Logic active-low không phải lỗi; đặt tên `buttonPressed` thay vì rải so sánh khó đọc.

Nút cơ có dội vài đến hàng chục millisecond. Debounce phần mềm theo thời gian ổn định thường đủ; interrupt không làm dội biến mất. Với dây dài/môi trường nhiễu, cần xem RC, Schmitt trigger, shielding và bảo vệ ESD.

A.6 BJT, MOSFET và relay theo câu hỏi

Phần tử	Hợp với	Câu hỏi bắt buộc
BJT NPN low-side	tải nhỏ/vừa	base current, saturation, công suất, diode tải cảm
MOSFET N low-side	tải DC hiệu quả	logic-level ở V_{GS} thật, $R_{DS(on)}$, gate resistor/pulldown, nhiệt
Driver H-bridge	motor hai chiều	dòng liên tục/đỉnh/stall, voltage drop, dead time, bảo vệ
Relay module	cách ly tiếp điểm theo rating	coil/driver, contact rating theo loại tải, flyback, safe default
Solid-state relay	đóng cắt phù hợp loại AC/DC	leakage, heat, zero-cross, fail mode

"MOSFET chịu 30 A" trên tiêu đề module không cho biết nó có mát ở gate 5 V/3,3 V, PCB đó và điều kiện của bạn. Tính $P \approx I^2 R_{DS(on)}$, cộng switching loss nếu PWM cao, xem nhiệt trở và đo nhiệt có kiểm soát.

A.7 Diode cần nhớ

Diode flyback song song ngược với coil/motor DC low-side tạo đường cho dòng cảm khi tắt. Đặt gần tải/driver, chọn dòng, điện áp và tốc độ. Diode thường làm dòng giảm chậm; ứng dụng cần release nhanh có thể dùng TVS/Zener/snubber theo thiết kế.

Diode bảo vệ đảo cực làm sụt áp; Schottky thấp hơn nhưng leakage khác. TVS kẹp transient không thay fuse. Mỗi linh kiện bảo vệ giải quyết một failure mode cụ thể.

A.8 Cảm biến: bảng đọc nhãn trước

Loại	Interface thường gặp	Cần xác minh
Biến trở/LDR	analog divider	dải raw, chiều tăng, trở kháng, Vref
Nhiệt/ẩm số	proprietary/I2C	điện áp, rate, checksum, self-heating, thư viện
Siêu âm	trigger/echo	mức Echo, vùng chết, timeout, góc chùm
IR phản xạ	analog/digital	ánh sáng môi trường, độ cao, ngưỡng module
Hall/reed	digital/analog	từ cực, debounce, pull-up, khoảng cách
PIR	digital	warm-up, retrigger, vùng nhìn, active level
Encoder	hai kênh số	xung/vòng, mức logic, tốc độ cạnh, interrupt/polling
Load cell	bridge + ADC	excitation, amplifier, cơ khí, calibration, drift
Khí/VOC hobby	analog/digital	warm-up, heater current, cross-sensitivity; không dùng an toàn

Module "gas sensor" hobby không phải thiết bị báo rò rỉ/cháy đạt chuẩn. Không suy ra nồng độ đáng tin từ một đồ thị mạng và không dùng bảo vệ tính mạng.

A.9 Bus và mức logic

Bus	Dây cốt lõi	Điều hay quên
UART	TX, RX, GND	nối chéo TX–RX, baud/frame, mức TTL không phải RS-232
I2C	SDA, SCL, GND	open-drain, pull-up/voltage, địa chỉ, chiều dài/capacitance
SPI	SCK, MOSI, MISO, CS	mode, max clock, mỗi CS, MISO release, level
OneWire	data, GND, nguồn	pull-up, topology/cáp, timing, parasite power
Analog	signal, reference/ground	source impedance, noise, range và common ground

Level shifter không phải lúc nào cũng cần và không phải loại nào cũng phù hợp mọi bus/tốc độ. Mạch MOSFET hai chiều phổ biến hợp I2C open-drain, không mặc nhiên hợp SPI tốc độ cao. Đọc schematic module: nhiều board đã có regulator nhưng chưa chắc đã shift logic.

A.10 Dây, breadboard và connector

Breadboard tiện học nhưng tiếp xúc không chắc, rail có thể bị ngắt giữa, không phù hợp dòng lớn/rung. Quy ước màu: đỏ nguồn dương, đen ground, màu khác signal; đây là thói quen chứ không chứng minh điện áp. Dây ngắn, cắm phẳng, kéo thử nhẹ, dán nhãn hai đầu ở dự án lớn.

Khi chuyển sang perfboard/PCB, thêm connector khóa, strain relief, test point, lỗ bắt cơ khí, khoảng cách đường công suất và silkscreen chỉ cực/pin 1. Một connector tốt ngăn nhiều lỗi hơn một đoạn code phòng thủ.

A.11 Đồng hồ số: bốn phép đo nền tảng

1. **Continuity khi mất nguồn**: tìm đứt/chập, biết buzzer có ngưỡng chứ không bằng 0 Ω tuyệt đối.
2. **Điện trở khi mất nguồn**: tháo/ảnh hưởng mạch song song; không đo ohm trên mạch đang cấp.
3. **Điện áp khi có nguồn**: que đen ở reference đúng, đo rail tại nguồn và tại tải lúc chuyển trạng thái.
4. **Dòng**: mắc nối tiếp, cắm đúng jack/range; không đặt thang A trực tiếp ngang nguồn vì gần như chập mạch.

Luôn trả que đỏ về jack voltage sau đo dòng. Với dòng motor đỉnh nhanh, đồng hồ có thể bỏ lỡ; oscilloscope/current probe hoặc shunt đúng thiết kế cần thiết.

A.12 Bộ linh kiện học tập đề xuất

Nhóm	Số lượng gợi ý	Mục đích
Uno R3 hoặc board tương thích rõ nguồn gốc	1	nền tảng bài học
Breadboard + dây jumper tốt	1 bộ	prototype điện áp thấp
LED nhiều màu	10–20	output/diagnostic
Điện trở 220 Ω–100 kΩ	bộ E12	LED, pull, divider, RC
Nút nhấn + biến trở	4 + 2	digital/analog input
Tụ gốm 100 nF, điện phân nhiều cỡ	10 + vài	decoupling/bulk/thí nghiệm
Photoresistor/sensor ánh sáng	2	analog/calibration
Buzzer passive/active	mỗi loại 1	waveform/event
Servo micro	1	actuator vị trí, nguồn riêng khi cần
MOSFET logic-level + diode	vài	tải DC low-side
Driver motor + motor nhỏ	1 bộ	H-bridge/robot
OLED I2C	1	UI/bus
Sensor nhiệt/ẩm có datasheet	1	data quality/scheduler
Đồng hồ số có cầu chì	1	đo điện áp, điện trở, continuity
Nguồn bench giới hạn dòng hoặc USB bảo vệ	1	bring-up an toàn

Không mua mọi module cùng lúc. Mua theo chương giúp bạn đọc kỹ hơn, tránh một hộp linh kiện không rõ pinout. Datasheet, schematic và một đồng hồ tốt đáng giá hơn nhiều sensor rẻ.

A.13 Checklist trước cấp nguồn

1. Điện áp nguồn và cực tính đã xác nhận bằng nhãn/sơ đồ?
2. Rail breadboard có ngắt giữa hay đảo màu ở đoạn nào?
3. Có chập Vcc–GND khi đo continuity lúc mất nguồn?
4. IC/diode/LED/tụ phân cực và pin 1 đúng chiều?
5. Mọi LED/tín hiệu cần hạn dòng đã có điện trở?

6. Tải có driver, diode, nguồn và common ground đúng?
7. Pin output có trạng thái safe trong reset/boot nhờ phần cứng?
8. Logic 3,3 V/5 V tương thích hai chiều?
9. Current limit đặt thấp hợp lý cho lần bring-up đầu?
10. Tay khô, bàn sạch, không có dây trần chạm nhau?

Trật tự bring-up

Kiểm tra mất nguồn → cấp nguồn giới hạn dòng không MCU nếu phù hợp → đo rail → gắn MCU với output safe → test từng ngoại vi → test tải nhỏ → test tình huống đỉnh/lỗi. Mỗi bước chỉ thay một biến.

Phụ lục B

Bảng lỗi tại bàn – Tìm nguyên nhân mà không đoán mò

Chẩn đoán là quá trình thu hẹp không gian nguyên nhân. Đừng thay dây, code, nguồn và thư viện cùng lúc: nếu mạch chạy, bạn không biết điều gì sửa nó; nếu vẫn hỏng, bằng chứng càng rối.

B.1 Quy trình tám bước

1. **Đóng băng hiện trạng**: chụp dây, lưu code/build log, ghi triệu chứng chính xác.
2. **Đưa về an toàn**: tắt tải, ngắt nguồn nếu có nhiệt/mùi/chập, kê cơ cấu.
3. **Tái hiện**: điều kiện nào luôn gây lỗi, điều kiện nào không?
4. **Chia đôi**: nguồn, phần cứng, giao tiếp, driver, logic hay timing?
5. **Đo**: rail, pin, event, timestamp; không dùng "có vẻ".
6. **Thay một biến**: sketch tối thiểu, một dây, một cấu hình.
7. **Xác nhận ngược**: trả thay đổi về cũ hoặc lặp nhiều lần để chứng minh quan hệ.
8. **Ghi nguyên nhân gốc**: triệu chứng, cơ chế, sửa, regression test.

B.2 Dấu hiệu phải ngắt nguồn ngay

Mùi khét, linh kiện quá nóng chạm không an toàn, khói, pin phồng/rò, dòng chạm giới hạn bất ngờ, tia lửa, tụ cắm ngược, motor kẹt hoặc nước gần mạch: ngắt ở nguồn/công tắc, không tiếp tục "xem thử". Pin lithium hỏng cần quy trình của nhà sản xuất/cơ sở địa phương; không đâm, bóp hay tự xử lý tùy tiện.

B.3 Không upload được

Quan sát	Khả năng	Phép thử ít rủi ro
Không thấy port	cáp chỉ sạc, driver/quyền, board chưa cấp	thử cáp dữ liệu biết tốt, cổng khác, xem Device Manager/system log
Port có nhưng upload timeout	sai board/port, D0/D1 bị tải, reset/bootloader	chọn lại board/port, tháo ngoại vi D0/D1, thử Blink
Signature mismatch	processor/core/board sai, bootloader/chip	đối chiếu chip/board package, thử board biết tốt
Upload được lúc được lúc không	cáp/connector/nguồn nhiễu, auto-reset	dây ngắn, bỏ tải, đo 5 V, thử USB khác
Compile lỗi trước upload	code/library/core	đọc dòng lỗi đầu có ý nghĩa, kiểm version và minimal example

Đừng nhấn reset liên tục ngẫu nhiên. Tách compile khỏi upload: nếu compile fail thì cáp không phải nguyên nhân; nếu compile xong và port fail thì code logic chưa chạy.

B.4 Board không có dấu hiệu sống

1. Ngắt ngoại vi, chỉ USB/cáp biết tốt.
2. Quan sát LED nguồn, đo 5 V và 3,3 V so với GND.
3. Nếu nguồn bench current-limit ngay, không tăng limit mù; tắt và đo chập.
4. Nạp Blink tối thiểu, không thư viện.
5. Nếu Blink chạy, gắn lại từng khối; đo rail sau mỗi bước.
6. Nếu regulator/chip nóng khi board trống, ngắt và thay/kiểm tra chuyên sâu.

B.5 LED không sáng hoặc sáng ngược

Kiểm tra cực LED, điện trở nối tiếp, pin map, mode OUTPUT, common ground và active-high/low của module. Dùng sketch bật cố định, đo điện áp hai phía LED. Nếu pin HIGH nhưng không có chênh áp đúng, dây/cực/LED; nếu pin không HIGH, code/pin/xung đột. Với RGB common-anode, mức LOW có thể bật màu; ghi abstraction thay vì đảo khắp code.

B.6 Nút tự nhấn, nhấn một lần ra nhiều lần

Input tự đổi khi thả thường là floating hoặc dây dài nhiễu. Bật pull-up/pull-down, kiểm common ground và connector. Nhiều event sát nhau là dội; log raw với timestamp, debounce theo ổn định. Nếu giữ nút tạo lặp, bạn đang xử lý level thay event; thêm edge detection/repeat policy rõ ràng.

B.7 Analog nhảy hoặc luôn 0/1023

Mẫu lỗi	Nguyên nhân hay gặp	Kiểm tra
Luôn 0	signal chập GND, sensor mất nguồn, pin sai	đo voltage tại chân, dùng biến trở biết tốt
Luôn max	chập Vref/Vcc, pin hở kéo lên, vượt range	đo signal, ngắt module, kiểm divider
Nhảy vài count	noise/quantization bình thường	histogram, average/median đúng nhu cầu
Nhảy khi motor chạy	ground bounce, rail dip, EMI	scope rail, tách đường dòng, decoupling, nguồn
Sai sau đổi kênh	source impedance/sample cap	bỏ mẫu đầu, giảm impedance theo datasheet
Tỷ lệ sai	Vref khác giả định, calibration sai	đo Vref/Vcc, dùng reference đúng, hiệu chuẩn

Lọc phần mềm không chữa rail sụt hoặc ground sai. Luôn xem raw trước khi map phần trăm.

B.8 Serial ký tự rác hoặc không có dữ liệu

Hai đầu phải cùng baud, frame và mức điện TTL; GND chung. TX nối RX. RS-232 điện áp thật không nối thẳng TTL. Nếu log mất đoạn, kiểm tốc độ in/buffer và code blocking. Nếu Serial Monitor mở làm reset Uno do DTR, nhận biết boot banner thay vì tưởng firmware crash.

Đóng Serial Monitor/app khác trước upload nếu port bị giữ. Dữ liệu nhị phân không đọc như text. Kết thúc dòng thống nhất; parser nên chịu `\r\n` và record hỏng.

B.9 I2C không thấy thiết bị

1. Xác nhận Vcc/GND và mức điện của module.
2. Xác nhận SDA/SCL đúng chân board; không suy pin Uno cho board khác.
3. Chạy scanner để thấy ACK, nhưng đối chiếu địa chỉ datasheet/jumper.
4. Kiểm pull-up tồn tại và kéo lên rail an toàn cho mọi thiết bị.
5. Rút bớt thiết bị, dây ngắn, giảm clock.
6. Dùng oscilloscope/logic analyzer: đường có lên đủ, bị giữ LOW, cạnh quá chậm?

7. Kiểm xung đột địa chỉ; đổi jumper/multiplexer nếu cần.

Bus bị giữ LOW có thể do thiết bị mất nguồn/back-power, giao dịch dở hoặc wiring. Power cycle an toàn có thể phục hồi nhưng chưa phải sửa nguyên nhân.

B.10 SPI đọc ID sai hoặc toàn 0xFF

Kiểm CS đúng và chỉ một thiết bị active, SCK/MOSI/MISO, mode 0–3, bit order, max clock, mức logic. 0xFF thường là MISO bị kéo lên/không ai lái; 0x00 có thể chập thấp, thiết bị chưa nguồn/reset. Đặt logic analyzer và giải mã transaction; so byte lệnh/response với datasheet. Dùng transaction API để thư viện trả cấu hình bus sau mỗi thiết bị.

B.11 Servo rung và Arduino reset

Nguồn servo thiếu dòng đỉnh hoặc ground/layout nhiễu là nghi phạm đầu. Không cấp nhiều servo từ 5 V Uno. Dùng nguồn riêng đủ dòng với ground chung đúng, tụ bulk gần tải, dây ngắn, đo Vcc tại MCU bằng scope. Tách thử: servo không tải, nguồn riêng, signal cố định. Cơ khí kẹt làm stall/current/heat; ngắt ngay.

Nếu rung chỉ ở một số góc, kiểm pulse range an toàn và liên kết cơ khí. Không ép servo vào hard stop. Thư viện Servo dùng timer có thể ảnh hưởng PWM pin khác; kiểm tài nguyên.

B.12 Motor yếu, driver nóng hoặc chỉ quay một chiều

Đo nguồn tại driver khi chạy, không chỉ khi không tải. Kiểm stall current so rating thật, voltage drop driver, PWM/direction table và ground. Driver tuyến tính cũ có sụt áp lớn; MOSFET bridge hiệu quả hơn nhưng vẫn cần nhiệt.

Chỉ một chiều: một input/half-bridge/wire/motor terminal có lỗi. Kê bánh, chạy bốn tổ hợp chân theo datasheet ở PWM thấp. Driver nóng nhanh: ngắt, tìm shoot-through, wiring sai, quá dòng, tần số/decay mode. Không đặt tản nhiệt như cách che thiết kế sai.

B.13 Relay click nhưng tải không chạy

Coil click chỉ chứng minh phần cơ có thể chuyển. Xác định COM/NO/NC bằng continuity khi mất nguồn; kiểm contact rating theo DC/AC và tải cảm/inrush. Module có jumper/active-low/opto wiring khác nhau. Với tải điện lưới, dừng: sách không hướng dẫn đấu; dùng thiết bị được chứng nhận và người có chuyên môn.

B.14 Chương trình treo “ngẫu nhiên”

“Ngẫu nhiên” thường nghĩa chưa quan sát đúng. Thêm boot counter/reset cause, heartbeat, state transition, deadline miss và pin debug. Các nguồn phổ biến:

- nguồn sụt/nhiễu khi tải chuyển;
- buffer overflow, index ngoài mảng, stack/RAM cạn;
- chuỗi động phân mảnh trên MCU nhỏ;
- ISR quá dài/biến shared không đúng atomic;
- chờ bus/Serial không timeout;
- state thiếu transition hoặc deadline sai do kiểu số;
- watchdog feed sai chỗ tạo che giấu hoặc reset loop.

Giảm về minimal reproducible case nhưng giữ điều kiện gây lỗi. Thay tải thật bằng tải giả để phân biệt điện và logic. Dùng compiler warnings cao và static analysis nếu workflow cho phép.

B.15 Timing chạy sai sau nhiều ngày

Tìm phép so sánh `now >= last + interval` dễ lỗi tràn; dùng `(unsigned long)(now-last) >= interval`. Kiểm kiểu `int` bị tràn, literal thiếu hậu tố `UL`, drift do đặt `last=now` thay cộng period, RTC trôi/mất pin, task blocking. Test bằng clock giả gần điểm wrap; đừng đợi 49 ngày.

B.16 EEPROM đọc cấu hình kỳ lạ

EEPROM mới/khác firmware có byte không theo schema. Record cần magic, version, length, payload, CRC; nếu invalid dùng default an toàn. Kiểm packing/endian/size khi đổi struct. Không ghi mỗi loop; chỉ ghi khi thay đổi và có debounce/commit. Power loss giữa write cần hai slot hoặc protocol commit.

B.17 Cảm biến trả số hợp lý nhưng sai

Lỗi nguy hiểm nhất không phải NaN mà là số "đẹp". So với dụng cụ tham chiếu phù hợp, đổi điều kiện có kiểm soát, đặt hai sensor cạnh nhau, kiểm placement/self-heating/calibration. Theo dõi drift và response. Plausibility dựa quan hệ: nhiệt đổi mà sensor đứng yên, bơm chạy nhưng ẩm không có xu hướng, motor lệnh đổi nhưng encoder không đổi.

B.18 Buzzer/`tone()` làm PWM khác biến mất

Timer là tài nguyên chia sẻ. Tra core và thư viện cho board/version; lập bảng timer owner. Tạo sketch tối thiểu chỉ gồm hai chức năng để chứng minh xung đột. Đổi pin không luôn đủ vì timer gắn nhóm pin. Có thể dùng timer khác, driver phần cứng, scheduler toggle hoặc board nhiều timer hơn tùy yêu cầu.

B.19 Compiler báo lỗi dài

Đọc lỗi đầu tiên trong mã của bạn, không nhảy xuống hàng chục lỗi hệ quả. Xác nhận dấu ngoặc, dấu chấm phẩy, tên/kiểu, include và trùng thư viện. Bật verbose compile để thấy thư viện được chọn. Ghi version IDE/core/library trong README; "latest" hôm nay không tái tạo được mai sau.

B.20 Bảng ghi chẩn đoán

Mỗi lỗi nên có một phiếu:

Mục	Nội dung cần ghi
Triệu chứng	quan sát định lượng, tần suất, trạng thái, timestamp
Điều kiện	nguồn, tải, firmware, nhiệt, cách kích hoạt
Kỳ vọng	hành vi theo yêu cầu nào
Giả thuyết	tối đa 3 giả thuyết đang ưu tiên và lý do
Phép thử	một biến, dụng cụ, kết quả dự đoán cho từng giả thuyết
Kết quả	số đo/log/ảnh, không chỉ pass/fail
Nguyên nhân gốc	cơ chế giải thích toàn bộ bằng chứng
Sửa	phần cứng/code/tài liệu, rủi ro mới
Regression	test giữ lỗi không quay lại

B.21 Mười câu hỏi trước khi cầu cứu

1. Board, core, IDE, thư viện và nguồn chính xác là gì?
2. Sơ đồ/pin map thật có giống code không?
3. Minimal sketch có tái hiện không?
4. Rail tại tải khi lỗi là bao nhiêu?
5. Log/state cuối trước lỗi là gì?
6. Lỗi theo thời gian, nhiệt, tải hay thao tác?
7. Đã thay đúng một biến nào, kết quả?
8. Có timeout cho mọi chờ ngoại vi không?
9. Output khi reset/boot là gì?
10. Người khác cần file/log/ảnh nào để tái hiện?

Một câu hỏi kỹ thuật tốt đã chứa nửa lời giải: tiêu đề cụ thể, sơ đồ tối giản, code có thể biên dịch, expected/actual và số đo. Nó giúp người khác hỗ trợ mà không phải đoán hệ thống bạn đang có.

Phụ lục C

Đáp án gợi ý và phòng thí nghiệm thu nhỏ

Đáp án không phải con đường duy nhất. Hãy tự làm trước, ghi dự đoán, đo kết quả rồi so. Các giá trị linh kiện/thời gian là điểm khởi đầu; phần cứng thật và datasheet có quyền phủ quyết.

C.1 Chương 1 – Input, state, output

Bài toán. Mô tả đèn hành lang có nút bật/tắt và tự tắt sau 30 giây.

Input vật lý là mức nút; sau debounce/edge detection ta có event `PRESS`. State tối thiểu gồm `OFF`, `ON`; dữ liệu kèm `turnedOnAt`. Output là đèn. Trong OFF, PRESS vào ON và lưu thời điểm. Trong ON, PRESS hoặc elapsed 30 s về OFF. Bất biến: khi state OFF, output luôn LOW kể cả vừa reset.

Listing C.1: Logic độc lập với chân phần cứng

```
enum class LampState : byte { OFF, ON };

void updateLamp(unsigned long now, bool pressedEvent) {
  if (lampState == LampState::OFF && pressedEvent) {
    lampState = LampState::ON;
    turnedOnAt = now;
  } else if (lampState == LampState::ON &&
             (pressedEvent || (unsigned long)(now - turnedOnAt) >= 30000UL)) {
    lampState = LampState::OFF;
  }
  digitalWrite(LAMP_PIN, lampState == LampState::ON ? HIGH : LOW);
}
```

Test bằng clock giả tại 29.999 s, 30.000 s, nhấn hai lần và reset trong ON. Bài này cho thấy một yêu cầu đời thường có thể biểu diễn không cần delay.

C.2 Chương 2 – Tính điện trở và công suất

Với nguồn 5 V, LED xanh có V_F đo được 2,8 V, dòng mong muốn 6 mA:

$$R \geq \frac{5 - 2,8}{0,006} = 366,7\,\Omega.$$

Chọn 390 Ω. Dòng ước tính $(5 - 2,8)/390 = 5,64$ mA. Công suất trên điện trở $I^2R \approx 0,0124$ W, rất dưới 0,25 W. Tuy vậy phải kiểm dòng chân/tổng MCU và độ sáng thật. Nếu LED vẫn đủ sáng ở 680 Ω, lựa chọn đó giảm tải hơn.

Với divider 10 kΩ–10 kΩ ở 5 V, điểm giữa lý tưởng 2,5 V và dòng divider 0,25 mA. Khi gắn tải 10 kΩ từ giữa xuống GND, nhánh dưới thành 5 kΩ, output còn $5 \times 5/(10+5) = 1,67$ V. Đây là lý do "đo không tải đúng" chưa bảo đảm khi nối ADC/mạch sau; phải xét trở kháng tải.

C.3 Chương 3 – Bring-up LED có bằng chứng

Quy trình đúng không bắt đầu bằng thay ngẫu nhiên:

1. Mất nguồn, đo điện trở giữa rail dương và GND để phát hiện chập rõ.
2. Xác định khe giữa breadboard, rail có bị ngắt và cực LED bằng datasheet/đồng hồ diode.
3. Cắm LED + 390 Ω từ 5 V xuống GND, chưa dùng chân MCU; xác nhận mạch tải.
4. Chuyển đầu vào D8, nạp sketch HIGH/LOW 1 s; đo điện áp pin.
5. Đảo phần mềm chỉ khi wiring/polarity đã biết, không dùng code che dây sai.

Nếu LED không sáng nhưng D8 đo 5 V: đo hai đầu LED. Chênh áp gần 0 cho thấy LED bị nối tắt/sai hàng; chênh gần 5 V có thể LED ngược/hỏng hoặc đường về đứt. Một số đo chia không gian nguyên nhân tốt hơn việc đổi năm linh kiện.

C.4 Chương 4 – Serial parser không dựa may mắn

Yêu cầu nhận lệnh `LED 1` và `LED 0`. Không dùng `readString()` với thời gian chờ dài trong hệ cần phản hồi. Tích lũy đến newline vào buffer cố định, bảo vệ tràn, sau đó parse.

Listing C.2: Thu một dòng vào buffer hữu hạn

```
char line[32];
byte used = 0;

void pollSerial() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\r') continue;
```

```
    if (c == '\n') {
      line[used] = '\0';
      handleLine(line);
      used = 0;
    } else if (used < sizeof(line) - 1) {
      line[used++] = c;
    } else {
      used = 0;
      Serial.println(F("ERR,line_too_long"));
    }
  }
}
```

`handleLine` so khớp đầy đủ, kiểm tham số và trả `OK` hoặc mã lỗi. Test dòng rỗng, CRLF, ký tự lạ, dài 31/32/100 byte và hai lệnh dính trong một lần nhận.

C.5 Chương 5 – Debounce và edge

Một lời giải giữ raw cuối, stable state, thời điểm raw đổi. Khi raw khác, cập nhật `changedAt`; chỉ khi raw giữ ít nhất 30 ms và khác stable mới commit. Event nhấn xuất hiện khi stable chuyển HIGH sang LOW với pull-up.

Listing C.3: Debounce phát event một vòng

```
bool updateButton(unsigned long now, bool rawHigh) {
  static bool lastRaw = HIGH, stable = HIGH;
  static unsigned long changedAt = 0;
  bool pressed = false;

  if (rawHigh != lastRaw) {
    lastRaw = rawHigh;
    changedAt = now;
  }
  if (rawHigh != stable &&
      (unsigned long)(now - changedAt) >= 30UL) {
    bool old = stable;
    stable = rawHigh;
    pressed = old == HIGH && stable == LOW;
  }
  return pressed;
}
```

Chuỗi test nên chứa dội LOW/HIGH trong 0–12 ms, ổn LOW từ 20–100 ms, dội lúc thả và wrap time. Kỳ vọng đúng một event press, không event khi giữ.

C.6 Chương 6 – Hiệu chuẩn analog hai chiều

Hàm đổi raw với hai mốc bất kỳ không giả chiều:

Listing C.4: Scale có kiểm tra mốc và clamp

```
bool scalePercent(int raw, int atZero, int atHundred, float& out) {
  const int span = atHundred - atZero;
  if (abs(span) < 20) return false;
  out = 100.0f * (raw - atZero) / span;
  out = constrain(out, 0.0f, 100.0f);
  return true;
}
```

Test $(0, 100, 900)$ cho khoảng 0%, $(1000, 100, 900)$ clamp 100%; đổi mốc $(900, 100)$ vẫn chạy đúng chiều. Nếu span nhỏ, trả invalid thay vì khuếch đại noise. Nên lưu cả raw và percent trong log để truy nguyên calibration.

Sai lầm phổ biến là dùng `map()` số nguyên rồi tin độ chính xác. `map()` không tự clamp, chia nguyên và không biết mốc có hợp lệ. Một hàm domain rõ giúp hành vi lỗi có chỗ đứng.

C.7 Chương 7 – Gamma cho LED và nhịp buzzer

Mắt không cảm độ sáng tuyến tính theo duty. Một bảng gamma 2,2 có thể biến mức người dùng 0–255 thành PWM. Trên AVR nhỏ, tạo bảng trước và lưu flash; không cần `pow()` mỗi vòng. Tuy nhiên LED/mạch/ánh sáng môi trường cũng ảnh hưởng, nên gamma là cảm nhận chứ không hiệu chuẩn quang học.

Với buzzer nhịp 100 ms mỗi 700 ms, dùng hai deadline hoặc state OFF/ON. Nếu đến hạn muộn, chọn chính sách: giữ phase bằng cộng period hoặc đặt theo now. Với cảnh báo, tránh phát dồn để "bắt kịp". Test loop cố tình bận 250 ms và xác nhận hệ không bật buzzer liên tục ngoài ý muốn.

PWM không giảm điện áp tức thời; tải thấy xung. LED/motor chấp nhận theo thiết kế, nhưng thiết bị analog khác có thể cần lọc hoặc DAC. Đừng gọi `analogWrite()` là analog voltage chung cho mọi pin/tải.

C.8 Chương 8 – Driver tải và năng lượng

Một motor 6 V có dòng chạy 200 mA và stall 1,2 A. Nguồn/driver phải xét ít nhất 1,2 A cùng margin và nhiệt, không chỉ 200 mA. Với MOSFET $R_{DS(on)} = 0,08\,\Omega$ tại đúng gate voltage, conduction loss ở stall xấp xỉ $1,2^2 \times 0,08 = 0,115$ W; còn switching/PCB/diode.

Trạng thái khi boot cần pull-down cổng để MOSFET tắt trước khi firmware cấu hình. Diode flyback song song motor, cực ngược lúc cấp bình thường. Common ground cho logic reference, nhưng dòng motor không chạy chung đoạn mảnh với ground sensor.

Test theo bậc: LED giả tải; motor kê tự do PWM thấp; motor tải bình thường; tình huống

gần stall thật ngắn với current limit và nhiệt theo dõi. Không cố tình giữ stall lâu. Đo rail MCU và driver ở mỗi bậc.

C.9 Chương 9 – Scheduler và wraparound

Hai chính sách scheduler:

Listing C.5: Giữ phase và bỏ lần quá trễ

```
bool due(unsigned long now, unsigned long& last,
         unsigned long period) {
  if ((unsigned long)(now - last) < period) return false;
  last += period;
  if ((unsigned long)(now - last) >= period) last = now;
  return true;
}
```

Nếu task 100 ms được gọi ở 101, 201, 302 ms, cộng period giữ phase gần 100/200/300. Nếu hệ treo 5 s, vòng `if` chỉ chạy một lần và nhảy tới now, không chạy 50 lần dồn. Với task cần xử lý mọi mẫu, một scheduler đơn giản không đủ; cần queue/buffer hoặc hardware sampling.

Test wrap: đặt `last=0xFFFFFFF0`, period 32, cho now đi qua 0x00000010. Phép trừ unsigned cho 32 và task đến hạn. Không cần đợi thời gian thật.

C.10 Chương 10 – State table thành test

Máy cửa mô hình có CLOSED, OPENING, OPEN, CLOSING, FAULT; event OPEN_REQ, CLOSE_REQ, LIMIT_OPEN, LIMIT_CLOSED, TIMEOUT. Bất biến: không bao giờ drive mở và đóng cùng lúc; FAULT tắt cả hai; mọi chuyển động có timeout.

Mỗi ô state/event tạo ít nhất một test. Ví dụ OPENING + LIMIT_OPEN → OPEN; OPENING + TIMEOUT → FAULT; OPENING + CLOSE_REQ có thể bị bỏ hoặc đảo có sequence an toàn, nhưng chính sách phải ghi. Một event không hợp lệ không được rơi vào state rác.

Entry action lưu `enteredAt`; render output từ state. Limit switch là input nhưng cũng có thể cần interlock phần cứng trong hệ năng lượng cao. Mô hình chỉ dùng LED/motor nhỏ kê an toàn.

C.11 Chương 11 – Median và EMA bằng dữ liệu

Chuỗi 500, 502, 501, 900, 503: mean là 581,2, median là 502. Median chống một outlier. Nếu tín hiệu thật bước từ 500 lên 700, median-5 cần ba mẫu mới phản ánh; nó tạo trễ.

EMA $y_k = y_{k-1} + \alpha(x_k - y_{k-1})$. Với $\alpha = 0,2$, bắt đầu 500 và input mới 700, các output là 540, 572, 597,6, 618,1... Mượt nhưng chậm. Chọn alpha theo sample period; đổi nhịp mà

giữ alpha sẽ đổi đáp ứng theo giây.

Phòng thí nghiệm: phát chuỗi raw qua Serial hoặc mảng cố định, chạy raw/median/EMA, xuất CSV, vẽ ba đường bằng Python. Đo noise standard deviation và thời gian đạt 90% bước. Chọn bằng con số, không chỉ "đường đẹp".

C.12 Chương 12 – Hysteresis và PID chống windup

Điều khiển quạt on/off: bật khi $T \geq 30°C$, tắt khi $T \leq 28°C$. Trong khoảng 28–30, giữ trạng thái cũ. Đây là memory một bit; một biểu thức `T > 29` không tương đương.

Với PID output giới hạn 0–255, integral chỉ tích khi output chưa bão hòa hoặc error kéo nó trở lại. Một dạng conditional integration:

Listing C.6: Ý tưởng anti-windup tối giản

```
float candidateI = integral + error * dt;
float raw = kp * error + ki * candidateI + kd * derivative;
float limited = constrain(raw, 0.0f, 255.0f);

bool pushesOut = (raw > 255.0f && error > 0.0f) ||
                 (raw < 0.0f && error < 0.0f);
if (!pushesOut) integral = candidateI;
```

PID nhiệt cần sensor/actuator, sample period và cơ cấu an toàn thích hợp; không thử với heater nguy hiểm. Mô phỏng plant hoặc dùng LED/quạt điện áp thấp trước.

C.13 Chương 13 – Frame, checksum và parser phục hồi

Chọn frame text: `@TYPE,SEQ,PAYLOAD*CS\n`. Checksum XOR trên bytes giữa @ và *. Parser state: WAIT_START, IN_BODY, IN_CHECKSUM; có max length và inter-byte timeout. Gặp @ mới trong body có thể resync từ đầu.

Test: frame đúng; sai checksum; thiếu newline; dài quá; noise trước @; hai frame liền; mất một byte; duplicate sequence. Receiver trả ACK theo sequence nếu protocol cần và xử lý duplicate idempotent. Checksum nhỏ chỉ bắt lỗi truyền tình cờ, không bảo mật/chống giả mạo.

Một thiết kế tốt nói rõ encoding, delimiter escaping, range, timeout, retry và version. "Gửi chuỗi qua Serial" chưa phải giao thức cho đến khi lỗi cũng có hành vi xác định.

C.14 Chương 14 – Chẩn đoán I2C/SPI bằng tầng

Với I2C, scanner thấy địa chỉ 0x3C nhưng OLED vẫn trắng. Điều đó chỉ chứng minh thiết bị ACK địa chỉ. Kiểm tiếp điện áp, reset, constructor kích thước, địa chỉ truyền vào thư viện, init return, buffer/display call và xung đột RAM. Thu logic analyzer để thấy địa chỉ/ACK/data; so ví dụ tối thiểu của đúng thư viện/version.

Với hai thiết bị SPI, lỗi chỉ xuất hiện khi cùng nối thường do CS floating hoặc một module không nhả MISO. Đặt pull-up CS, cấu hình tất cả CS OUTPUT/HIGH trước begin, bọc mỗi giao dịch trong `SPI.beginTransaction()` và `endTransaction()`, quan sát mode/clock. Rút từng MISO giúp khoanh vùng nhưng sửa cuối phải theo schematic/datasheet.

Bài nộp nên có bảng tầng: physical (rail/wire/waveform), link (ACK/mode), driver (return code), application (state). Mỗi số đo chỉ hỗ trợ tầng tương ứng; đừng dùng scanner để kết luận "màn hình hỏng".

C.15 Chương 15 – UI từ nhu cầu hành động

Một màn giám sát bơm cần hiển thị state, moisture, age/quality và fault, không cần raw chiếm vị trí chính. Quy tắc gợi ý:

- dòng lớn: trạng thái hành động `MONITOR/WATER/SOAK/FAULT`;
- dòng hai: ẩm 42% hoặc "–" nếu invalid;
- biểu tượng nhỏ: tuổi mẫu, manual/auto, kết nối;
- fault page: mã + hành động an toàn "PUMP OFF – CHECK SENSOR".

Nút ngắn đổi trang, nhấn giữ 1,5 s mở xác nhận reset lỗi; không để một chạm xóa cảnh báo. Test bằng state giả, số dài, NaN, mất bus, chuyển nhanh. Màn mất không được kéo logic điều khiển ngừng; render là consumer của model.

Độ nhấp nháy thường do clear toàn màn mỗi vòng hoặc frame rate không ổn. Chỉ render định kỳ, dùng buffer đúng thư viện và cập nhật khi model/page đổi. Đo thời gian giao dịch OLED để đưa vào budget.

C.16 Chương 16 – Record EEPROM hai slot

Record gồm magic, version, sequence, payload và CRC. Hai slot A/B. Khi load, validate cả hai, chọn sequence mới hơn theo quy tắc wrap. Khi save, ghi slot cũ hơn: invalid marker trước, payload/CRC, rồi magic/valid marker cuối. Mất nguồn giữa chừng để lại slot cũ hợp lệ.

Listing C.7: Cấu trúc cấu hình có dấu vết phiên bản

```
struct ConfigV1 {
  uint16_t magic;
  uint8_t version;
  uint8_t length;
  uint32_t sequence;
  int16_t dryRaw;
  int16_t wetRaw;
  uint16_t crc;
};
```

Không phụ thuộc layout struct giữa compiler nếu lưu lâu; serialize từng field hoặc kiểm size/endianness. Test EEPROM toàn 0xFF, CRC sai, version cũ, mất điện giả sau từng byte, sequence wrap. Default phải safe và UI/log nói config đã fallback.

Ước tính endurance: nếu datasheet cho N chu kỳ/ô và ghi mỗi phút, tuổi lý tưởng $N/(60\times 24)$ ngày; còn nhiệt/phân bố. Ghi mỗi thay đổi người dùng thay vì mỗi loop làm chênh lệch nhiều bậc.

C.17 Chương 17 – ISR tối thiểu và snapshot nguyên tử

Encoder sinh cạnh nhanh. ISR chỉ tăng đếm và lưu thời điểm nếu cần; không `Serial.print()`, không delay, không I2C.

Listing C.8: Sao chụp biến nhiều byte từ ISR

```
volatile uint32_t pulseCount = 0;

void onPulse() { ++pulseCount; }

uint32_t snapshotPulses() {
  noInterrupts();
  uint32_t copy = pulseCount;
  interrupts();
  return copy;
}
```

`volatile` buộc truy cập bộ nhớ nhưng không làm phép đọc 32-bit nguyên tử trên AVR 8-bit. Critical section ngắn giải quyết snapshot. Nếu cần lấy rồi reset, làm cả hai trong cùng vùng.

Test tốc độ cạnh tối đa bằng signal generator hoặc sketch thứ hai, so count kỳ vọng. Đo ISR width bằng toggle pin chỉ trong test. Nếu edge quá nhanh, dùng hardware counter/input capture hoặc giảm tín hiệu; không nhồi thêm logic vào ISR.

C.18 Chương 18 – Ranh giới module

Tách dự án chậu cây thành:

- `SoilSensor`: driver lấy raw/quality;
- `MoistureModel`: calibration/filter/freshness;
- `IrrigationController`: state/quota, không biết chân;
- `PumpDriver`: safe output, max hardware-facing checks;
- `Console`: command/log;
- `App`: scheduler và nối các phần.

Interface controller có thể là `update(now, moisture, valid, event)` trả `PumpCommand`. Trong test trên máy tính, bơm chỉ là giá trị; feed chuỗi dữ liệu và kiểm state. Driver thật chuyển command thành pin.

Không tạo singleton/global ở mọi module làm phụ thuộc ẩn. Truyền cấu hình hoặc reference cần thiết. Tuy nhiên một project Uno nhỏ không cần framework DI đồ sộ; ranh giới nên làm code ngắn hơn để suy luận, không phải thêm nghi thức.

C.19 Chương 19 – Từ triệu chứng đến nguyên nhân

Triệu chứng: board reset khi servo bắt đầu. Ba giả thuyết ưu tiên: rail sụt, nhiễu reset/ground, bug code. Phép thử:

1. Ghi boot banner/reset count để chứng minh reset, không chỉ màn nhấp nháy.
2. Thay servo bằng LED nhưng giữ logic: nếu hết lỗi, nghi năng lượng/EMI.
3. Cấp servo nguồn riêng đủ dòng với common ground đúng; đo Vcc tại MCU bằng scope.
4. Giữ servo nhưng chạy pulse cố định minimal sketch; loại tầng ứng dụng.
5. Thử tải cơ khí rời/kê; xem stall/current.

Nếu Vcc tụt dưới ngưỡng đúng thời điểm reset và nguồn riêng sửa được lặp lại, bằng chứng mạnh cho root cause nguồn. Sau sửa, regression test nhiều góc/tải, pin thấp và power cycle; không dừng ở một lần chạy.

C.20 Chương 20 – Power budget và FMEA

Ví dụ: Uno 50 mA giả định đo, OLED 25 mA, sensor 10 mA, servo idle 20 mA/move 250 mA/stall 900 mA. Rail logic không được tính đơn giản 50+25+10+250 rồi chọn 500 mA nếu servo có thể stall 900 mA. Tách nguồn servo 5–6 V đủ peak với margin, ground chung thiết kế, tụ bulk; nguồn logic riêng. Tổng công suất và connector/wire cũng phải chịu.

FMEA chậu cây mẫu:

Failure	Effect	Detection	Mitigation
Sensor đứt	đọc khô giả	range/stale	auto off, fault
MOSFET chập	bơm luôn on	feedback/dòng	fuse, cutoff/giới hạn nước
Ống tuột	nước sai chỗ	không tăng ẩm/khay	clamp, flow/leak, bình nhỏ
MCU treo	output giữ	watchdog và heartbeat	mặc định tắt, timer độc lập
Nguồn sụt	reset lặp	reset cause/scope	budget, rail tách, enable delay

Severity cao có thể đòi giảm scope dù xác suất thấp. Safe state không mặc nhiên OFF;

phải theo quá trình, nhưng với mô hình bơm để bàn này OFF là lựa chọn bảo thủ.

C.21 Chương 21 – Test bất biến đèn giao thông

Ta tách hàm `outputsFor(state, phase)` trả năm boolean, không gọi pin. Test duyệt mọi state và phase nhấp nháy:

Listing C.9: Bất biến có thể chạy như một phép kiểm

```
bool safe(const Lamps& x) {
  bool conflictingGreen = x.carGreen && x.pedGreen;
  bool carHasTwo = (x.carRed + x.carYellow + x.carGreen) > 1;
  return !conflictingGreen && !carHasTwo;
}
```

Trong C++, bool cộng thành số 0/1. Bổ sung yêu cầu đúng một đèn xe sáng, trừ mode fault nếu thiết kế cho tất cả đỏ (vẫn một đỏ). Test transition trace từ mọi state, nhấn ở từng millisecond biên bằng clock giả. Nếu đổi duration, test không cần chờ thật.

Một chi tiết chính sách: nhấn trong WALK. Nếu giữ yêu cầu cho chu kỳ sau, người giữ nút có thể tạo phục vụ liên tiếp; nếu bỏ, người đến muộn phải nhấn lại. Chọn và ghi theo mô hình người dùng, thêm cooldown/debounce phù hợp.

C.22 Chương 22 – Quota tưới dạng cửa sổ

Với tối đa ba xung trong 30 phút, lưu timestamp ba lần gần nhất trong ring buffer. Trước tưới, đếm timestamp có tuổi dưới cửa sổ. Không cần RTC; uptime đủ cho giới hạn vận hành giữa reset, nhưng reset sẽ xóa lịch sử. Nếu reset để lách quota là rủi ro, lưu bền có kiểm soát hoặc dùng hardware/giới hạn nước độc lập.

Sau mỗi xung, lưu moisture trước. Sau SOAK, nếu tăng dưới `MIN_RESPONSE` trong hai chu kỳ liên tiếp, vào fault NO_RESPONSE. Đừng kết luận từ một lần vì vị trí sensor/nước. Log pulse volume ước lượng từ lưu lượng đã đo, không từ nhãn bơm.

Test chuỗi: khô thoáng qua; khô 30 s; wet trong confirm; ba xung; xung thứ tư bị từ chối; sensor mất khi đang tưới; reset giữa HIGH; no-response. Mỗi case xác nhận chân bơm và state, không chỉ log.

C.23 Chương 23 – Bộ lọc vùng khoảng cách

Median-3 trên chuỗi 80, 79, 250 cho 80; nhưng ba timeout liên tiếp không có median hợp lệ. Model giữ last good cùng age, quality chuyển SUSPECT sau một lỗi và UNKNOWN sau năm lỗi. UI có thể giữ khoảng cách cũ với dấu “ ” trong 200 ms, sau đó “–”.

Hysteresis test cho CAUTION: 99 giữ caution, 104 vào FAR; từ FAR 101 vẫn FAR, 96 vào caution. Test đúng bằng `<` hay `<=` ở 97/103 để đặc tả không mơ hồ.

Độ trễ tệ nhất: sensor 20 Hz (đợi trung bình 25 ms), median-3 cần đến 100 ms để đa số đổi, xác nhận hai kết quả thêm 50 ms, loop/beep 10 ms; xấp xỉ 160 ms. Ở 0,5 m/s, vật đi 8 cm. Cộng sai số/margin vào ngưỡng STOP, không chỉ nhìn con số 20 cm.

C.24 Chương 24 – Health model trạm môi trường

Mỗi Reading có chất lượng riêng; system health là phép tổng hợp có mức độ:

- OK: mọi dịch vụ bắt buộc GOOD/fresh;
- DEGRADED: sensor phụ/display lỗi nhưng logging lõi còn;
- FAILED: sensor bắt buộc mất hoặc scheduler miss kéo dài;
- STARTING: chưa đủ mẫu sau boot.

Không lấy worst của mọi module nếu OLED mất mà đo/log vẫn có giá trị; định nghĩa criticality. Log transition health một lần, không spam mỗi vòng. Khi sensor hồi, cần ba mẫu GOOD; health recovery có timestamp.

Python kiểm log: parse `uptime_ms`; tính difference; flag gap lớn hơn 1,5 chu kỳ; nhóm theo quality; không nội suy qua FAILED như dữ liệu thật. Vẽ đường value cùng vùng nền thể hiện quality để người đọc thấy độ tin cậy.

C.25 Chương 25 – Dấu correction của robot

Giả sử position âm khi vạch bên trái, error $= -position$ nên dương. Công thức left $= B - u$, right $= B + u$: bánh phải nhanh hơn làm xe quay trái hay phải phụ thuộc chiều/hình học. Với xe differential chạy tiến, bánh phải nhanh hơn làm xe cong trái, đúng. Tuy nhiên wiring motor có thể đảo; test kê bánh và đẩy sensor qua vạch để xác nhận bằng quan sát.

Unit test position: [0,0,1000,0,0] $\rightarrow$ 0; [1000,0,0,0,0] $\rightarrow -2000$; [0,500,500,0,0] $\rightarrow -500$; toàn 0 $\rightarrow$ unseen. Test normalization trước vì raw chiều ngược sẽ đảo nghĩa.

Khi vào SEARCH, lưu last sign, giảm PWM, đặt timeout. Khi reacquire, reset derivative và ramp base trong 200 ms để tránh giật. Nếu mất vạch ở giao điểm đối xứng, last sign có thể 0; chọn search pattern hữu hạn rồi dừng.

C.26 Chương 26 – Một mẫu traceability nhỏ

Project hộp nhắc nghỉ mắt có requirement FR-02: “hết focus duration phải vào trạng thái báo nghỉ trong sai số 100 ms”. Test T-TIME-02 dùng duration 2 s trong test build, clock giả bước 10 ms, kỳ vọng transition ở [2000,2100] ms; test wrap khởi đầu gần cực đại. Bằng chứng là event trace và assertion, không video đồng hồ duy nhất.

NFR-01 “button latency dưới 50 ms” đo bằng logic analyzer: kênh 1 raw button, kênh 2 debug pin set khi event dispatch. Debounce 30 ms nghĩa latency chủ động đã khoảng 30–40 ms; nếu vượt, kiểm task blocking. Requirement giúp lộ mâu thuẫn sớm.

SAF-01 “output tắt khi boot/state lỗi”: điện trở pull-down phần cứng, setup đặt LOW trước OUTPUT theo driver phù hợp, test power-cycle 20 lần và reset ở mọi state. Nếu actuator vẫn có glitch 2 ms, software test đã tìm ra giới hạn cần hardware enable.

C.27 Cách dùng đáp án để học sâu hơn

Với mỗi lời giải, hãy thay một giả định: board 3,3 V; sensor chậm gấp đôi; loop bị block 80 ms; pin gần cạn; dữ liệu mất 10%; người dùng giữ nút. Dự đoán trước, chạy test sau. Nếu lời giải gãy, đừng vá một dòng; quay lại requirement, state và ngân sách. Chính phần “nếu như” biến ví dụ thành kỹ năng chuyển giao được.

Phụ lục D

Từ điển bỏ túi Arduino và hệ thống nhúng

Các thuật ngữ được giải thích theo ngữ cảnh cuốn sách. Một số từ có nghĩa rộng hơn trong điện tử/chuyên ngành; khi thiết kế thật, dùng định nghĩa trong datasheet và tiêu chuẩn áp dụng.

Thuật ngữ	Giải thích thực hành
ADC	Bộ chuyển đổi analog sang số; ánh xạ điện áp trong range tham chiếu thành mã rời rạc.
Actuator	Phần tử tác động ra thế giới: LED, buzzer, motor, van, heater; thường cần driver và power budget.
Active-high	Chức năng được kích hoạt khi signal HIGH.
Active-low	Chức năng được kích hoạt khi signal LOW; thường viết dấu gạch trên tên hoặc hậu tố `_N`.
Aliasing	Mẫu quá chậm làm tín hiệu nhanh xuất hiện sai tần số/hình dạng; cần sample rate và lọc phù hợp.
Analog	Đại lượng liên tục theo mô hình; ADC chỉ quan sát thành mức rời rạc có sai số.
API	Hợp đồng hàm/kiểu mà module cung cấp cho phần khác sử dụng.
AREF	Điểm/đầu vào tham chiếu của ADC trên một số board; dùng sai cấu hình có thể gây hỏng.
Atomic	Thao tác không bị quan sát ở trạng thái dở; quan trọng khi main và ISR chia sẻ dữ liệu nhiều byte.
Back-powering	Thiết bị bị cấp ngược qua chân tín hiệu khi rail chính tắt.
Baud	Tốc độ ký hiệu; trong UART cấu hình phổ biến gần với bit/s nhưng không phải khái niệm hoàn toàn đồng nhất.
Blocking	Lời gọi giữ CPU/chờ khiến tác vụ khác chưa chạy; phải đưa vào ngân sách latency.

Thuật ngữ	Giải thích thực hành
Bootloader	Chương trình nhỏ hỗ trợ nạp firmware trước khi ứng dụng chạy.
Breadboard	Bảng cắm thử không hàn; nhanh nhưng tiếp xúc/layout không hợp dòng lớn/tần số cao/rung.
Brownout	Nguồn thấp khiến MCU chạy ngoài điều kiện hoặc được mạch brown-out reset.
Buffer	Vùng nhớ tạm cho bytes/mẫu/event; phải có kích thước và chính sách khi đầy.
Bus	Đường truyền dùng chung giữa nhiều thành phần, thí dụ I2C/SPI.
Calibration	Tìm quan hệ giữa raw và đại lượng/mốc tham chiếu trong điều kiện đã biết.
Checksum	Giá trị kiểm lỗi truyền/lưu đơn giản; không cung cấp tính xác thực bảo mật.
Clock drift	Sai lệch tích lũy của đồng hồ/nhịp so với thời gian tham chiếu.
Common ground	Điểm tham chiếu điện chung cần cho nhiều tín hiệu single-ended; bố trí đường hồi dòng vẫn rất quan trọng.
Composition	Tạo đối tượng/hệ bằng cách chứa các thành phần có trách nhiệm riêng, thay vì kế thừa mọi thứ.
Concurrency	Nhiều công việc tiến triển xen kẽ hoặc song song; trên MCU nhỏ thường là loop + interrupt/peripheral.
CRC	Kiểm tra dư vòng, bắt nhiều mẫu lỗi tốt hơn checksum đơn giản nhưng vẫn không phải chữ ký bảo mật.
Critical section	Đoạn cần bảo vệ khỏi truy cập xen kẽ; phải càng ngắn càng tốt.
Current limit	Giới hạn dòng của nguồn/driver để giảm hậu quả chập/quá tải.
DAC	Bộ chuyển số sang điện áp/dòng analog thật; không phải board nào cũng có.
Data quality	Nhãn GOOD/SUSPECT/FAILED cho biết mức tin cậy, không chỉ giá trị.
Deadband	Khoảng input/output không tạo thay đổi; ở motor có thể là PWM thấp chưa thắng ma sát.
Deadline	Thời điểm chậm nhất công việc phải hoàn tất để yêu cầu còn đúng.
Debounce	Loại nhiều chuyển mức cơ học trong một lần nhấn/nhả.
Decoupling capacitor	Tụ đặt gần chân nguồn IC để cung cấp dòng biến thiên nhanh và giảm nhiễu cục bộ.
Derivative kick	Thành phần D nhảy mạnh do setpoint/error đổi đột ngột.

Thuật ngữ	Giải thích thực hành
Digital	Biểu diễn bằng mức/trạng thái rời rạc; mức điện vẫn là analog và có ngưỡng.
Duty cycle	Tỷ lệ thời gian tín hiệu ở trạng thái bật trong một chu kỳ PWM.
EEPROM	Bộ nhớ không mất dữ liệu khi tắt nguồn, có endurance ghi hữu hạn.
EMA	Exponential moving average, bộ lọc $y \leftarrow y + \alpha(x - y)$.
EMI	Nhiễu điện từ do cạnh nhanh, motor, dây/layout; có thể phát hoặc bị thu.
Encoder	Cảm biến biến chuyển động thành xung/vị trí; quadrature có hai kênh để biết chiều.
Endianness	Thứ tự byte của số nhiều byte khi lưu/truyền.
Event	Điều vừa xảy ra có thời điểm, thí dụ cạnh nhấn; khác với level đang giữ.
Fail-safe	Thiết kế hướng hệ về trạng thái giảm rủi ro khi lỗi đã xét; không đồng nghĩa miễn nhiễm mọi lỗi.
Fault injection	Chủ động tạo lỗi có kiểm soát để kiểm detection/recovery.
FMEA	Phân tích failure mode, effect, severity, detection và mitigation.
Flyback diode	Diode tạo đường cho dòng tải cảm khi driver tắt, giảm xung áp.
Freshness	Độ mới của dữ liệu, thường tính bằng tuổi từ timestamp mẫu cuối.
FSM	Finite-state machine: state hữu hạn, event/guard và transition.
Ground bounce	Điện áp ground cục bộ dịch do dòng thay đổi và trở kháng đường hồi.
Guard	Điều kiện phải đúng để một transition được phép xảy ra.
H-bridge	Mạch driver cho dòng qua motor theo hai chiều, thường hỗ trợ PWM/brake/coast.
Hysteresis	Dùng ngưỡng vào và ra khác nhau để tránh rung quanh một biên.
I2C	Bus đồng bộ hai dây SDA/SCL, open-drain, địa chỉ và pull-up.
Idempotent	Xử lý lại cùng lệnh không làm hậu quả nhân đôi ngoài ý muốn.
Input capture	Phần cứng timer chụp timestamp cạnh tín hiệu chính xác hơn polling.
Interrupt	Cơ chế tạm chuyển CPU sang ISR khi sự kiện; không phải luồng phép thuật.

Thuật ngữ	Giải thích thực hành
ISR	Interrupt service routine; ngắn, xác định, tránh blocking/I/O phức tạp.
Jitter	Biến thiên của chu kỳ hoặc latency quanh giá trị mong muốn.
Latency	Thời gian từ kích thích/event đến phản ứng được quan sát.
Level shifter	Mạch chuyển mức logic giữa các miền điện áp; topology phải hợp bus/tốc độ.
Logic analyzer	Dụng cụ ghi nhiều tín hiệu số theo thời gian và có thể giải mã protocol.
Loop rate	Tần suất vòng lặp hoàn thành; không ổn định nếu có tác vụ blocking/biến thiên.
Magic number	Hằng số rải trong code không tên/ý nghĩa; cũng có thể chỉ signature trong record.
Margin	Khoảng dự phòng giữa điều kiện thiết kế và giới hạn.
Median filter	Chọn trung vị cửa sổ, mạnh với outlier đơn lẻ nhưng tạo trễ.
Metastability	Trạng thái không ổn định khi tín hiệu vi phạm timing ở phần tử số; đồng bộ input bất đồng bộ khi cần.
MOSFET logic-level	MOSFET có thông số dẫn thấp tại gate voltage logic thực, không chỉ threshold.
Non-blocking	Mỗi lần gọi làm ít việc rồi trả, tiến trình được giữ bằng state/deadline.
NVM	Non-volatile memory, nhóm bộ nhớ giữ dữ liệu khi mất nguồn.
Outlier	Mẫu lệch xa phần còn lại do nhiễu/lỗi hoặc sự kiện thật; không được xóa mà thiếu lý do.
Polling	Kiểm tra input/peripheral lặp trong chương trình thay vì nhận interrupt.
Power budget	Bảng dòng/công suất theo rail, idle/typical/peak, hiệu suất và margin.
PWM	Điều chế độ rộng xung; thay duty để đổi giá trị trung bình/năng lượng trên tải phù hợp.
Race condition	Kết quả phụ thuộc thứ tự/timing truy cập chung ngoài dự kiến.
Rail	Đường phân phối một mức nguồn/reference trong mạch.
Rate limiting	Giới hạn tần suất sự kiện/lệnh để tránh spam/quá tải.
Raw value	Giá trị gần đầu ra phần cứng trước map/filter/calibration; rất hữu ích cho debug.
Recovery	Quá trình rời fault/degraded về vận hành, thường cần điều kiện ổn định và reset state.

Thuật ngữ	Giải thích thực hành
Regression test	Test giữ lỗi đã sửa không xuất hiện lại.
Resolution	Bước nhỏ nhất biểu diễn/quan sát; không đồng nghĩa accuracy.
Retry	Thử lại hữu hạn sau lỗi; cần timeout/backoff và không che lỗi kéo dài.
Ring buffer	Buffer vòng có head/tail, hiệu quả cho stream/event hữu hạn.
Rollover	Bộ đếm quay về 0 sau giá trị cực đại; phép trừ unsigned xử lý interval đúng.
Safe state	Trạng thái giảm rủi ro khi lỗi/boot; phải xác định theo năng lượng/quá trình.
Sampling rate	Số mẫu mỗi giây; quyết định khả năng quan sát tín hiệu và tải xử lý.
Saturation	Output chạm giới hạn vật lý/số; PID integral có thể windup.
Scheduler	Thành phần quyết định tác vụ nào đến hạn chạy; không nhất thiết là hệ điều hành.
Schema	Cấu trúc/version của record hoặc message để hai phía hiểu cùng nghĩa.
Sensor fusion	Kết hợp nguồn đo theo mô hình/độ tin cậy, không chỉ lấy trung bình.
Setpoint	Giá trị mục tiêu của vòng điều khiển.
Shield	Board cắm chồng theo chuẩn chân Arduino; vẫn cần kiểm nguồn/pin conflict.
Signal integrity	Chất lượng dạng sóng tại receiver: cạnh, ringing, crosstalk, noise và reference.
Soak test	Chạy dài trong điều kiện kiểm soát để tìm leak, drift, reset và lỗi hiếm.
SPI	Bus đồng bộ SCK/MOSI/MISO cùng chip-select, nhanh nhưng nhiều dây hơn I2C.
Stall current	Dòng motor khi trục không quay; thường lớn hơn dòng chạy và quan trọng cho driver/nguồn.
State	Điều hệ thống đang là, quyết định cách diễn giải event và output.
Static analysis	Phân tích code không chạy để tìm một số lỗi/vi phạm quy tắc.
Timeout	Giới hạn chờ; biến "có thể treo mãi" thành nhánh lỗi có thể xử lý.
Timestamp	Mốc thời gian gắn với mẫu/event; phải biết clock, unit và rollover.
Traceability	Liên kết requirement tới thiết kế, code, test và bằng chứng.

Thuật ngữ	Giải thích thực hành
Transient	Biến động ngắn điện áp/dòng do switching, tải cảm, ESD hoặc nguồn.
UART	Giao tiếp nối tiếp bất đồng bộ với TX/RX và cấu hình baud/frame.
Unit test	Test một đơn vị logic nhỏ với input/kỳ vọng kiểm soát.
Uptime	Thời gian từ boot; hữu ích cho lịch nội bộ nhưng không phải giờ lịch.
Validation	Kiểm dữ liệu/sản phẩm phù hợp nhu cầu/nghĩa thực tế; thường phân biệt với verification.
Verification	Kiểm sản phẩm thực hiện đúng đặc tả/yêu cầu đã viết.
Volatile	Từ khóa báo compiler giá trị có thể đổi ngoài luồng bình thường; không bảo đảm atomic.
Watchdog	Bộ đếm độc lập reset/cảnh báo nếu phần mềm không chứng minh còn tiến triển đúng hạn.
Wear leveling	Phân tán lần ghi qua nhiều ô/slot để tăng tuổi bộ nhớ.
Worst-case execution time	Thời gian chạy tệ nhất được đo/phân tích cho một tác vụ trong điều kiện đã định.

D.1 Năm cặp dễ nhầm

1. **Resolution và accuracy**: ADC nhiều bit có bước nhỏ, nhưng tham chiếu, nhiễu và hiệu chuẩn vẫn có thể làm sai số lớn.
2. **Raw và value**: raw là bằng chứng gần phần cứng; value đã mang giả định chuyển đổi.
3. **Level và event**: LOW đang giữ khác "vừa chuyển sang LOW".
4. **Timeout và fault**: timeout là quan sát; fault là diễn giải sau chính sách/lịch sử.
5. **Fail-safe và fault-tolerant**: fail-safe giảm hậu quả khi lỗi; fault-tolerant vẫn duy trì chức năng qua một số lỗi. Một hệ có thể có cái này mà không có cái kia.

Phụ lục E

Sổ tay thực hành – Mười bốn phiếu biến "đã chạy" thành bằng chứng

In các phiếu này hoặc sao chép vào nhật ký. Mỗi phiếu cố ý buộc ta dự đoán trước khi đo, ghi điều kiện và chỉ thay một biến. Dòng trống không phải bài văn; hãy điền số, đơn vị, phiên bản và ảnh/log tham chiếu.

E.1 Phiếu 1 – Kiểm tra trước cấp nguồn

Dự án/phiên bản: _____ **Ngày:** _____

Nguồn dự kiến: _____ **Current limit:** _____

STT	Điểm kiểm	Kết quả
1	Part number, pin 1, cực LED/diode/tụ đã đối chiếu	_____
2	Vcc–GND không chập rõ khi mất nguồn	_____
3	Rail breadboard, khe giữa và common ground đúng	_____
4	Mức logic giữa mọi module tương thích	_____
5	Tải có điện trở/driver/flyback/current limit	_____
6	Output có trạng thái safe trong boot/reset	_____
7	Dây công suất, signal và cơ khí không chạm/rối	_____
8	Công tắc ngắt nguồn dễ với; khu vực test an toàn	_____

Dự đoán dòng không tải: _____ **Dòng đo:** _____

Nếu chênh quá margin đã định, không tăng giới hạn để ép chạy. Ghi giả thuyết:

Quyết định cấp nguồn/không cấp và lý do:

E.2 Phiếu 2 – Bring-up một output

Output: _____ **Pin/active level:** _____

1. Viết trạng thái mong đợi trước `setup()`: _____
2. Vẽ đường dòng khi ON và OFF; đánh dấu điện trở/driver.
3. Nạp sketch tối thiểu bật 1 s, tắt 1 s; không thư viện khác.
4. Đo điện áp pin và hai đầu tải trong cả hai trạng thái.
5. Reset/cắm nguồn 10 lần, quan sát glitch bằng LED hoặc logic analyzer.

Trạng thái	V pin	V tải	Quan sát
Mất nguồn	_____	_____	
Boot	_____	_____	
OFF	_____	_____	
ON	_____	_____	
Reset khi ON	_____	_____	

Tiêu chí pass: output đúng logic, không vượt rating, boot/reset safe. Nếu fail, ghi một thay đổi duy nhất và phép xác nhận ngược.

E.3 Phiếu 3 – Nút nhấn và debounce

Wiring: pull-up/pull-down _____; thả _____; nhấn _____.

Ghi raw edge bằng timestamp trong 20 lần nhấn. Ước lượng bounce dài nhất thay vì chọn 50 ms theo thói quen.

Lần	Cạnh raw	Event sạch	Ghi chú
1–5	_____	_____	
6–10	_____	_____	
11–15	_____	_____	
16–20	_____	_____	

Debounce chọn: _____ ms. **Latency p95:** _____ ms.

Test bắt buộc: chạm cực ngắn, nhấn bình thường, giữ 5 s, nhấn nhanh hai lần, dội giả, wrap time. Chính sách giữ nút: _____

Pass khi: mỗi hành động người dùng tạo đúng số event theo chính sách, không có event khi thả và latency nằm trong requirement.

E.4 Phiếu 4 – Khảo sát một kênh analog

Sensor/kênh/Vref: _____

Thu ít nhất 100 mẫu ở ba điều kiện ổn định. Không map phần trăm trước khi xem raw.

Điều kiện	Min	Median	Max	Ghi chú
Mốc thấp				
Mốc giữa				
Mốc cao				
Tải bật				

Noise peak-to-peak: _____ count; **span hữu ích:** _____ count.

Thử median-5 và EMA hai alpha. Ghi độ lệch chuẩn sau lọc và thời gian đạt 90% khi tín hiệu bước. Bộ lọc chọn phải dựa yêu cầu noise/latency.

Calibration equation + unit: _____

Điều kiện invalid/stale: _____

Giữ file CSV raw, config và đồ thị. Nếu số đổi khi motor/OLED chạy, quay về nguồn/layout trước khi tăng lọc.

E.5 Phiếu 5 – PWM, tải và nhiệt

Pin/timer/tần số dự kiến: _____

Tải/driver/nguồn: _____

Đo hoặc quan sát duty 0, 25, 50, 75, 100%. Với LED ghi cảm nhận sáng; motor ghi voltage/current/tốc độ nếu có. Xác định deadband và điểm bắt đầu quay.

Duty	Dòng	Nhiệt driver	Output đo	Ghi chú
0%				
25%				
50%				
75%				
100%				

Kiểm tra xung đột với `tone()`, Servo hoặc thư viện timer. Đo waveform khi chức năng kia bật/tắt. Ghi version core vì mapping/timer phụ thuộc nền tảng.

Giới hạn PWM vận hành và lý do: _____

Nếu driver/tải nóng ngoài dự kiến, ngắt nguồn và xem I^2R, switching, stall, sụt áp và tản nhiệt; không tiếp tục chỉ để điền bảng.

E.6 Phiếu 6 – Scheduler không chặn

Liệt kê task trước khi code:

Task	Period	Deadline	WCET đo	Overrun policy
_____				
_____				
_____				
_____				
_____				

Đặt pin debug quanh task dài, thu ít nhất 1.000 chu kỳ. Ghi period min/mean/max, jitter và số deadline miss. Chèn tác vụ giả bận 2x bình thường để xem chính sách skip/catch-up.

Test wraparound với clock giả. Tìm và sửa mọi biểu thức dựa `now >= last + period`. Không dùng một biến `previousMillis` cho các task độc lập.

Utilization quan sát: _____%; **margin:** _____%.

Pass khi: deadline yêu cầu giữ ở worst case đã định, event khẩn/UX vẫn được đọc, không burst vô hạn sau block và wrap không đổi chu kỳ.

E.7 Phiếu 7 – Review máy trạng thái

Tên hệ: _____ **Safe state:** _____

Liệt kê state, entry action, output và timeout. Mỗi state cần câu trả lời "làm sao rời đi?"; mọi state chuyển động/năng lượng có deadline.

State	Entry/output	Event/guard	Next/fault

Ba bất biến:

(1) _____

(2) _____

(3) _____

Tạo test cho mọi cạnh hợp lệ, event không hợp lệ, timeout đúng biên, reset trong mỗi state và state enum rác. Log transition gồm old, event, new, time; không log mỗi vòng.

E.8 Phiếu 8 – Giao thức Serial

Mục đích/phiên bản: _____

Frame grammar: _____

Quyết định	Đặc tả
Encoding và delimiter	
Độ dài frame/payload	
Range và unit	
Checksum/CRC	
Timeout giữa byte/frame	
ACK, retry, duplicate	
Version và tương thích	
Error response	

Fuzz tối thiểu 100 frame: rỗng, noise, byte mất, delimiter trong payload, quá dài, checksum sai, hai frame dính, delay giữa byte. Parser không treo, không tràn và resync ở frame tốt tiếp theo.

Golden frame và kết quả parse mong đợi:

Một frame lỗi và error/recovery mong đợi:

E.9 Phiếu 9 – Bring-up I2C hoặc SPI

Thiết bị/part/revision: _____

Voltage, địa chỉ hoặc mode/max clock: _____

1. Đối chiếu pinout trên module với datasheet/schematic.
2. Đo rail; kiểm pull-up và mức logic.
3. Chỉ nối một thiết bị, chạy example tối thiểu đúng library version.
4. Thu trace: start/address/ACK với I2C hoặc CS/SCK/MOSI/MISO với SPI.
5. Đọc ID/register biết trước; lưu bytes raw.
6. Thêm thiết bị thứ hai, lặp trace và kiểm conflict.
7. Rút/mất ACK có kiểm soát; firmware timeout, báo lỗi và tiếp tục phần còn lại.

Library/core version: _____

Thời gian giao dịch typical/max: _____

Recovery policy: retry _____ lần, backoff _____ ms.

Sau đó: _____

Đính kèm ảnh trace có nhãn; decoder tự động và waveform phải đồng ý.

E.10 Phiếu 10 – Dữ liệu bền qua mất nguồn

Dữ liệu nào thật sự cần lưu? _____

Tần suất thay đổi: _____ **Endurance budget:** _____

Record: magic _____; version _____; length _____; sequence _____; CRC _____.

Tình huống	Kỳ vọng	Kết quả
EEPROM toàn 0xFF	default + event	
CRC sai slot A	chọn B/default	
Hai slot hợp lệ	chọn sequence mới	
Version cũ	migrate/reject rõ	
Mất nguồn giữa write	còn ít nhất một record tốt	
Giá trị ngoài range	clamp/reject, không unsafe	

Đếm số byte ghi trong một thao tác và số thao tác/ngày tệ nhất. Nếu lưu mỗi phút chỉ để “cho chắc”, thiết kế lại: RAM cho runtime, EEPROM cho thay đổi cấu hình có chủ ý.

E.11 Phiếu 11 – Power budget và đo rail

Tải	Rail	Idle	Typical	Peak/stall	Nguồn số liệu
_____					
_____					
_____					
_____					

Regulator input/output/current: _____

P_{loss} dự tính _____ W; hiệu suất/thermal margin _____.

Đo Vcc tại nguồn, MCU và driver khi idle, switching, khởi động, tải tệ nhất an toàn. Oscilloscope ground phải nối đúng; không đo mạch điện lưới bằng quy trình hobby.

Điểm	Min	Max	Ripple/transient
Nguồn			
MCU Vcc			
Driver rail			
Sensor rail			

Pass/fail và margin còn lại: _____

E.12 Phiếu 12 – Fault injection và recovery

Chỉ tạo lỗi không nguy hiểm, ưu tiên stub/dữ liệu giả. Xác định trước safe output và cách dừng thử.

Lỗi	Phát hiện và hạn	Hành động an toàn	Phục hồi, test
Sensor mất			
Sensor kẹt số			
Bus timeout			
Input stuck			
Output feedback sai			
Nguồn sụt/reset			
Queue/buffer đầy			
State không hợp lệ			

Ghi thời gian từ lỗi đến detection và từ detection đến safe state. Xác nhận bằng chân thật, không chỉ biến/log. Recovery cần điều kiện ổn định, giới hạn retry và thông báo; nếu phải power-cycle thủ công, ghi rõ.

Lỗi phần cứng phần mềm không thể cứu:

Lớp giới hạn hậu quả độc lập: _____

E.13 Phiếu 13 – Soak test và phân tích log

Thời lượng/môi trường/firmware: _____

Tiêu chí dừng ngay: nhiệt, dòng, chuyển động, nước hoặc error nào?

Log tối thiểu: uptime, state, health, sample age, error counters, deadline misses, boot event và các đại lượng chính. Máy tính nhận log gắn timestamp riêng.

Chỉ số	Kết quả	Tiêu chí/pass
Số boot/reset ngoài ý muốn		
Gap log lớn nhất		
Sample age lớn nhất		
Sensor error/recovery		
Deadline miss		
Memory/high-water nếu có		
Nhiệt/dòng cực đại		

Vẽ value + quality, histogram interval và timeline event. Chọn ba bất thường, giải thích bằng bằng chứng hoặc ghi "chưa biết". Sau sửa, chạy lại cùng protocol để so sánh.

Kết luận giới hạn phạm vi đã test:

E.14 Phiếu 14 – Nghiệm thu đồ án

Tên/version/repository hoặc gói nguồn: _____

STT	Bằng chứng bàn giao	Vị trí/trạng thái
1	Brief, scope, người dùng và điều không làm	
2	Requirement có ID và tiêu chí đo	
3	Block diagram, schematic, pin map, BOM	
4	Power/timing/memory/error budget	
5	State table/diagram, event và bất biến	
6	Source build được, dependency/version khóa	
7	Safe boot, timeout, error/recovery path	
8	Test matrix liên kết requirement	
9	Fault injection và regression result	
10	Soak log/đồ thị và phiên bản firmware	
11	FMEA, rủi ro còn lại, giới hạn sử dụng	
12	README dựng/nạp/vận hành/dừng an toàn	
13	Video/demo script và phương án dự phòng	
14	Retrospective và backlog có ưu tiên	

Ba lỗi còn biết:

(1) _____

(2) _____

(3) _____

Điều kiện sản phẩm chưa được dùng:

Người review/ngày/kết luận: _____

Nghiệm thu không tuyên bố "không còn lỗi". Nó nói phiên bản nào đã được kiểm dưới điều kiện nào, bằng chứng ở đâu và rủi ro nào vẫn còn.

Phụ lục F

Tài liệu tham khảo và lộ trình đọc tiếp

Tài liệu phần cứng thay đổi theo board, core và thư viện. Các đường dẫn dưới đây là điểm bắt đầu chính thức; hãy lưu cả phiên bản/ngày truy cập trong README của dự án. Ngày truy cập của bản thảo: 04/09/2026.

F.1 Arduino chính thức

1. Arduino Documentation, *Language Reference*: https://docs.arduino.cc/language-reference/
2. Arduino Documentation, *UNO R3*: https://docs.arduino.cc/hardware/uno-rev3
3. Arduino Help Center, *Download and install Arduino IDE*: https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE
4. Arduino Help Center, *Upload a sketch in Arduino IDE*: https://support.arduino.cc/hc/en-us/articles/4733418441116-Upload-a-sketch-in-Arduino-IDE
5. Arduino built-in examples trong IDE và tài liệu: dùng ví dụ tối thiểu của đúng board/core làm mốc bring-up.
6. Kho mã nguồn Arduino AVR Boards/core: dùng để tra mapping chân, timer và implementation theo phiên bản đã cài.

F.2 Vi điều khiển và điện tử

1. Microchip, *ATmega328P Data Sheet*: thông số điện, GPIO, ADC, timer, interrupt, EEPROM và fuse/brown-out.
2. Horowitz, Paul; Hill, Winfield, *The Art of Electronics*, Cambridge University Press.
3. Scherz, Paul; Monk, Simon, *Practical Electronics for Inventors*, McGraw-Hill.
4. Williams, Tim, *The Circuit Designer's Companion*, Newnes.

5. Ott, Henry W., *Electromagnetic Compatibility Engineering*, Wiley.

Giá trị pin, absolute maximum, timing và peripheral luôn ưu tiên datasheet đúng mã/grade/package/revision. Sách giúp đặt câu hỏi; datasheet trả lời cho linh kiện đang dùng.

F.3 Hệ thống nhúng và kiến trúc phần mềm

1. Barr, Michael; Massa, Anthony, *Programming Embedded Systems*, O'Reilly.
2. Pont, Michael J., *Patterns for Time-Triggered Embedded Systems*, Addison-Wesley.
3. Samek, Miro, *Practical UML Statecharts in C/C++*, Newnes.
4. Koopman, Philip, *Better Embedded System Software*, Drumnadrochit Education.
5. Jack Ganssle, các bài viết về debounce, watchdog, firmware quality và embedded debugging tại `https://www.ganssle.com/`

Những nguồn này đi xa hơn sketch: deadline, state machine, defensive design, code review và reliability. Đọc với tinh thần so sánh yêu cầu của hệ mình, không bê nguyên pattern.

F.4 Điều khiển và xử lý tín hiệu

1. Åström, Karl J.; Murray, Richard M., *Feedback Systems*, Princeton University Press; bản điện tử của tác giả: `https://fbsbook.org/`
2. Smith, Steven W., *The Scientist and Engineer's Guide to Digital Signal Processing*: `https://www.dspguide.com/`
3. Tài liệu/application note chính thức của nhà sản xuất sensor về calibration, noise, layout và sampling.

PID chỉ là một phần điều khiển. Hãy học mô hình plant, sampling, stability, saturation, delay và measurement quality trước khi tăng gain.

F.5 Đo lường và dụng cụ

Manual của chính đồng hồ, oscilloscope, logic analyzer và nguồn bench là tài liệu bắt buộc. Học category/rating, giới hạn input, cách nối ground của oscilloscope và cầu chì thang dòng trước khi đo. Các video ngắn không thay manual an toàn.

Đối với logic protocol, đối chiếu trace với datasheet thiết bị và chuẩn bus. Decoder tự động có thể đặt sai threshold/mode; waveform thô vẫn là bằng chứng nền.

F.6 Cách đọc một datasheet

1. Xác nhận part number, package, revision và điều kiện nhiệt/nguồn.
2. Đọc pinout và block diagram để dựng mô hình.
3. Tách *absolute maximum* khỏi *recommended operating conditions*.
4. Xem electrical characteristics với điều kiện test, min/typ/max.
5. Xem timing diagram trước khi viết protocol/driver.
6. Đọc application circuit/layout, errata và note liên quan.
7. Ghi lại giả định/chọn giá trị vào design note; dẫn trang/bảng.

F.7 Lộ trình sau cuốn sách

1. **Đường firmware**: C/C++ sâu hơn, build system, unit test trên host, static analysis, Git/CI.
2. **Đường phần cứng**: schematic/PCB, nguồn chuyển mạch, signal integrity, EMC và thiết kế cho sản xuất.
3. **Đường điều khiển**: toán hệ động lực, discrete control, encoder, system identification.
4. **Đường kết nối**: protocol binary, CAN/RS-485, security, update firmware an toàn.
5. **Đường sản phẩm**: requirement, risk, standards, verification/validation, serviceability và lifecycle.

Hãy chọn một đường trong ba tháng, làm một dự án nhỏ có log và báo cáo. Độ sâu đến từ vòng lặp thiết kế–đo–sửa, không từ việc mua board mới liên tục.

Lời kết

Một board Arduino không làm thế giới đơn giản đi. Nó chỉ đặt những câu hỏi của điện tử và phần mềm lên một chiếc bàn đủ nhỏ để ta có thể chạm vào. Khi bạn đo trước khi đoán, viết trạng thái trước chuỗi `if`, đặt timeout trước khi chờ, và nghĩ đến lỗi trước lúc demo, bạn đã bước ra khỏi bài tập. Từ đây, linh kiện có thể đổi; cách học ấy vẫn đi cùng bạn.

Nguồn chính thống dùng để kiểm chứng bản sâu

1. Arduino. *Language Reference*: https://docs.arduino.cc/language-reference/
2. Arduino. *Getting Started with Arduino*: https://docs.arduino.cc/learn/starting-guide/getting-started-arduino/

3. Arduino. *Arduino Uno Rev3 documentation*: https://docs.arduino.cc/hardware/uno-rev3/

4. Microchip Technology. *ATmega48A/PA/88A/PA/168A/PA/328/P Data Sheet*: https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf